11. The co-simulation with Maestro and RabbitMQFMU was added to provide simulation capabilities of the coupled cooperative motions.
12. Synchronous cooperative experiments are successful and used for deviation checking.
13. The flex-cell DT components are uploaded to the DTaaS platform (see Sect. 11.7 for further information on the DTaaS platform).

12.5.3 Summary and Future Work

The flex-cell case study provides a representative system for DT engineering in cooperative robotics, being composed of two robotic arms of different brands, with joints and grippers. It therefore gives an opportunity to explore the composition of DTs and models. The services for this case study are handled on its working space, i.e., the plate, mostly with a scope in robot positioning. There are multiple enablers for the modelling, communication and simulation aspects, such as the Robotics Toolbox and UniFMU for integrating the kinematic models of the robots, Maestro and RabbitMQFMU for running co-simulations, the DT Manager for administrating the interfaces and Unity and URDF for the visualisation components.

The most challenging aspect of this case study the integration of the models in a cooperative-synchronised setting. This is because there are multiple modelling approaches for robotic arms for different components, e.g., kinematics, dynamics, visualisation, programming. This makes it difficult to integrate all of them into a single DT if the scope is broad. Thus, it is better to narrow the scope to the applications that are needed, and hence choose the appropriate models for that scope. The current scope of the flex-cell is related to positioning, but it may be widened in the future to cover other relevant aspects.

The case study still evolves, mostly in terms of applications and services, such as collision detection, optimisation of assembly, and flexible and reconfigurable sequences. Additional models for other applications, such as payload estimation, are also possible evolution scenarios. The ongoing work of the flex-cell DT focuses on the generation of sequences for flexible manufacturing in a cooperative setting by integrating the skill-based programming approach [32].

12.6 The Research Vessel Gunnerus

The R/V Gunnerus, shown in Fig. 12.28, is a research vessel owned by the Norwegian University of Science and Technology (NTNU) that is used to support marine research and exploration. The vessel is equipped with the latest technology for a variety of research activities in biology, technology, geology, archaeology, oceanog-

raphy and fisheries [33]. In addition to research, the ship is used for educational purposes and is an important platform for marine courses at all levels and disciplines including nautical operation and bio-engineering. The main dimensions of the vessel are given in Table 12.2. The vessel is arranged with a wet lab, dry lab and a computer lab in addition to a large aft deck. Accommodation comprises three double berth scientific personnel cabins and three single berth crew cabins. The large mess hall functions as a lecture room for 25 people.

Fig. 12.28: Starboard view of the R/V Gunnerus. © 2024 Fredrik Skoglund, reprinted with permission.

This case study makes use of the Gunnerus as the test-bed to demonstrate a sound conceptual framework that uses co-simulation as a fundamental technology *towards* realising a DT for ship manoeuvring. The current state of the study does not involve automatic data transmission back to the PT, thus making it a DS as discussed in Sect. 1.2.2 or "open-loop" case study. Based on this system, however, a lot of interesting research work has been implemented, including physics-informed data-driven approach for ship docking prediction [38], co-simulation-based pre-training of ship trajectory prediction [19], and incorporating approximate dynamics into a data-driven calibrator for ship manoeuvring [37].

Table 12.2: Main dimensions of the R/V Gunnerus.

Parameter	Value
Length overall (Loa)	36.25 m
Length between pp (Lpp)	33.90 m
Waterline length (Lwl)	29.90 m
Breadth middle (Bm)	9.60 m
Breadth extreme (B)	9.90 m
Depth mld. Main deck (Dm)	4.20 m
Draught, mld (dm)	2.70 m
Deadweight	165t

12.6.1 Fundamental Characteristics

This section presents the Gunnerus case study through the 14 fundamental DT characteristics introduced in Sect. 12.2.

12.6.1.1 C1: System-under-study

The PT in this case is the R/V Gunnerus. The environment it typically operates in is the sea off the west coast of Norway, with the ship's home port located in Trondheim. Ship operations in this area are affected by rapid variations in water depth and the potential obstruction of wind flow by high mountains in local areas. These effects introduce significant danger and make the physical operation and testing of the PT challenging. The agents include the ship and its crew, other vessels, the sea traffic centre and possibly the presence of a *Remote Operating Centre* (ROC).

The ship is regularly in operation, including navigation, Dynamic Propulsion (DP) operation, docking, and lifting, etc. It is fitted with a dynamic positioning system and a HiPap 500 unit to provide optimal conditions for remote operated vehicle operations and the positioning of any deployed equipment. The onboard diesel electric system has been specially designed for low hydro-acoustic noise levels to accommodate testing and development of hydro-acoustic equipment. In C7, we discuss the DT/DS with regard to ship steering. For docking and DP operations, the ship dynamics will be different from normal speed ship manoeuvring. Otherwise, the realisation of the DT/DS will be the same.

12.6.1.2 C2: Acting Components

In order to transmit data from the PT back to shore, a special-purpose PC has been installed onboard that is able to collect key information from the various systems onboard. As the ship operates offshore, data must be transmitted wirelessly over a cellular 4G/5G connection. Thus, Message Queuing Telemetry Transport (MQTT),

a standard messaging protocol for the IoT, is used to transmit data onshore as it saves bandwidth by having a central broker relay data to potential clients onshore. This central broker needs to be reliable, so a virtual Linux server monitored and serviced by the NTNU IT department has been setup.

12.6.1.3 C3: Sensing Components

As a research vessel, there are many data acquisition systems installed onboard the Gunnerus, such as thrusters, power generators, navigational sensors and motion control systems, etc. The number of measurable sensor channels is high and for the purpose of motion prediction, we are interested in the data originating from GPS receiver, a Motion Reference Unit (MRU), compasses, a wind sensor and sensors reflecting orientation and rotational speed of thrusters. The sensor channels related to motion prediction are listed in Table 12.3. For all measurements in the data set, a sampling rate of 1 Hz is used.

Table 12.3: Data channels sensed onboard the R/V Gunnerus.

Signal	Channels	Unit
GPS	Latitude	ddmm.mmmm
	Longitude	ddmm.mmmm
	Surge velocity	knots
	Sway velocity	knots
	Course angle	deg
	Speed over ground	knots
MRU	Heading angle	deg
	Heading rate	deg/s
	Roll angle	deg
	Pitch angle	deg
	Heave displacement	m
	Roll rate	deg/s
	Pitch rate	deg/s
	Heave rate	m/s
Wind sensor	Wind direction	deg
	Wind speed	knots
Thruster	Port thruster rotational speed	%
	Port thruster angle	deg
	Starboard thruster rotational speed	%
	Starboard thruster angle	deg
	Tunnel thruster rotational speed	%

12.6.1.4 C4: Multiplicities

Currently there are two DTs implemented. One DT is for decision support for ship motion prediction and docking operations. The other DT is for system diagnostics for thruster failures detection and isolation for the ship in dynamically positioned operations for the same PT.

12.6.1.5 C5: Data Transmitted

Today's maritime engineering systems are often equipped with various sensors and operating in highly dynamic environments. All these data sources, including the status from ship, crane and engine, can be transferred via a 4G connection to an onshore control center to conduct operational planning. A PC is deployed on Gunnerus, from where the MQTT network protocol is used to publish data messages from different data sources as topics. Finally, the data is applied and analysed on the DT and could be transferred back to Gunnerus to enhance operational planning and decision-making. Relevant data is also stored to a database.

12.6.1.6 C6: Insights/Actions

Insights: The system can be used to visualise the current and historical state of the system in 2D and 3D using easily accessible web-services. Furthermore, the system can be used to provide on-board decision support for the crew and provide diagnostic services such as Remaining Useful Life (RUL) for serviceable equipment as in Fig. 12.29.

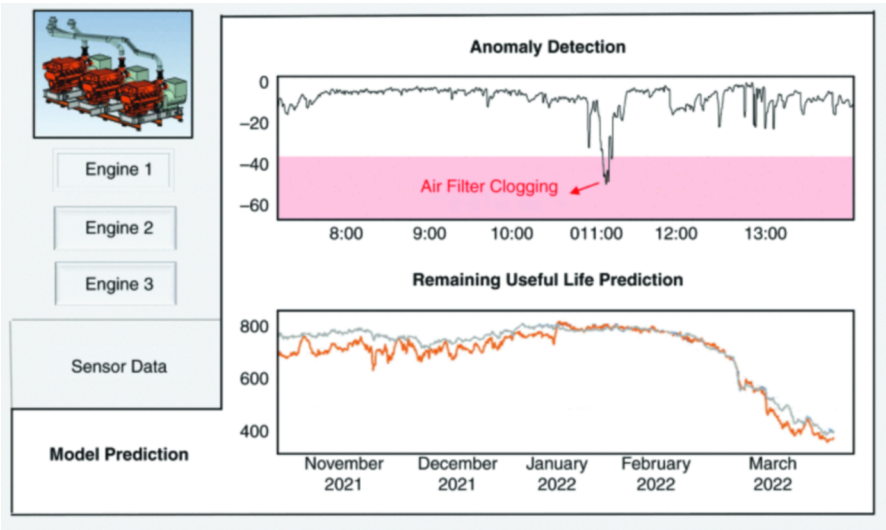


Fig. 12.29: Anomaly detection and fault prognostics of the vessel's engines. Figure taken from [39].

Actions: The PT/DT connection is currently running in an "open-loop" manner, where no automatic actions have been implemented yet.

12.6.1.7 C7: Services

Offshore vessels often perform in harsh environments with wind, waves, currents and low temperatures. These operations need to be carefully planned and rehearsed before execution. For this purpose, the DT/DS of the whole ship are used as a valuable tool for pre-operation and planning. It will allow the iterative design of operational procedures by identifying inconsistencies and blunders, exploring "what if" scenarios during onshore planning sessions based on historical methodological ocean records and raising awareness on critical phases. It can also be used in onboard "toolbox talks" just before the execution of an operation to rehearse the procedure, based on the current ship loading conditions and the short-term predictions of methodological ocean conditions.

As a short summary, the services that the DT/DS currently provides are:

1. Providing *what-if simulations* such that the user can enter in proposed conditions and examine the resulting behaviour (see Sect. 10.2).
2. Real-time *visualisation* for the stakeholders to understand the current state of the ship (see Sect. 8.4).
3. Providing extra sensory information about the current environment by comparing the simulation state with the real world state to *estimate the effect of environmental disturbances*.
4. Providing *anomaly detection* and *estimation of RUL* for the onboard equipment (see Sect. 9.4).

Different offshore operations have various features and requirements, and are quite demanding. The DT/DS currently provides valuable services for these operations. However, there are still ongoing challenges in improving the utility of these services, especially in the dimension of increased fidelity.

12.6.1.8 C8: Enablers

The case study relies on co-simulation frameworks such as the Open Simulation Platform (OSP) to coordinate the co-simulation of models [39]. A prognostics and health management system supports the RUL service, while a visualisation enabler supports the visualisation service.

12.6.1.9 C9: Models and Data

The following list briefly describes each of the FMUs used to create a DS of the Gunnerus.

Gunnerus: This model either contains historical sensor data for analytic purposes or access to live data through a MQTT connection. The data is sampled at 1Hz and includes information such as:

- Heading angle and percent-wise commanded Revolutions Per Minute (RPM) of the tunnel-thruster in the bow as well as the two azimuth thrusters in the aft.
- Longitude and latitude.
- Surge, sway, and heave.
- Yaw, pitch, and roll.
- Wind direction and speed.
- Positional and rotational velocities.

The model implements linear interpolation, which is convenient given the low sample rate of the sensors relative to the simulation, which runs at 20 Hz.

VesselModel: This model computes the vessel hydrodynamics such as the radiation forces, mass, and restoring forces as well as manoeuvring forces (resistance and cross flow drag as well as semi-empirical corrections). The equations of motions are solved by this model, summing up all the external forces acting on the vessel. The model was established during the SimVal [14] project, where the internal dynamics where calibrated based on standardised ship trails.

PMAzimuth: The hydrodynamic model of the azimuth thrusters without actuator/motor. Given a certain RPM command (issued by the *ThrusterDrive* FMU), location on the hull, azimuth angle, vessel speed, and the loss factor, the model will output the 3DoF (surge, sway, heave) force generated.

ThrusterDrive: A drive that converts force commands from the *SpeedController* into RPMs for the *PMAzimuth*.

PowerPlant: A marine power plant with two equally large gensets, including auxiliary load and circuit breakers [34].

SpeedCont: A controller used to regulate the force required by the *ThrusterDrives* so that the speed of the *VesselModel* and the vessel are aligned.

HeadingController: A controller that is used to align the heading of the *VesselModel* and the vessel by regulating the angle of the *PMAzimuths*.

VesselModelObserver: A simple model that computes the direction of travel and speed over ground of the *VesselModel* based on its current and previous position.

An overview of the models are also given in Table 12.4.

Additionally, CAD drawings and simplified 3D models as well as ample historical data in CSV format exists for the PT. This data is available for research internally in NTNU, but the data is proprietary and cannot be made public.

Table 12.4: FMUs involved in the DS for ship manoeuvring.

Component	Tool	Vendor	FMI version
HeadingController	FMI4j	NTNU	2.0
SpeedController	FMI4j	NTNU	2.0
Gunnerus	FMI4j	NTNU	2.0
VesselModelObserver	FMI4j	NTNU	2.0
ThrusterDrive	20sim	SINTEF Ocean	1.0
PowerPlant	20sim	SINTEF Ocean	1.0
VesselModel	VeSim	SINTEF Ocean	1.0
PMAzimuth	VeSim	Kongsberg Maritime	1.0

12.6.1.10 C10: Constellation

Fig. 12.30 shows the information flow between the models in the case study. As illustrated, the system is non-trivial with a total of 48 variable connections. Moreover, the VeSim and 20sim model instances cannot co-exist within the same process as other instances generated from the same tool due to shared library symbol conflicts. To overcome this challenge, execution of these must be split across processes.

12.6.1.11 C11: Time-Scale

The DS for the vessel operates at different rates:

- **Slower-than-real-time:** The anomaly alerts and dashboard information provided to the user may not be acted upon in real-time.
- **Real-time:** Dashboard updates.
- **Faster-than-real-time:** The simulator is able to simulate into the future.

As for the co-simulation time-step required to successfully run the simulation, 20Hz has been determined to be adequate as the system is fairly loosely coupled. In effect, this allows the system as a whole to run faster than real-time even on laptops with moderate specifications. The main bottleneck with regard to simulation speed is the fact that some of the models involved must communicate between processes due to implementation details related to shared library conflicts. The underlying issue is that a shared library can only be loaded once within a single process. If two models, or even two instances of the same model, needs to dynamically load the same shared library during runtime, then the code cannot be executed within one logical process.

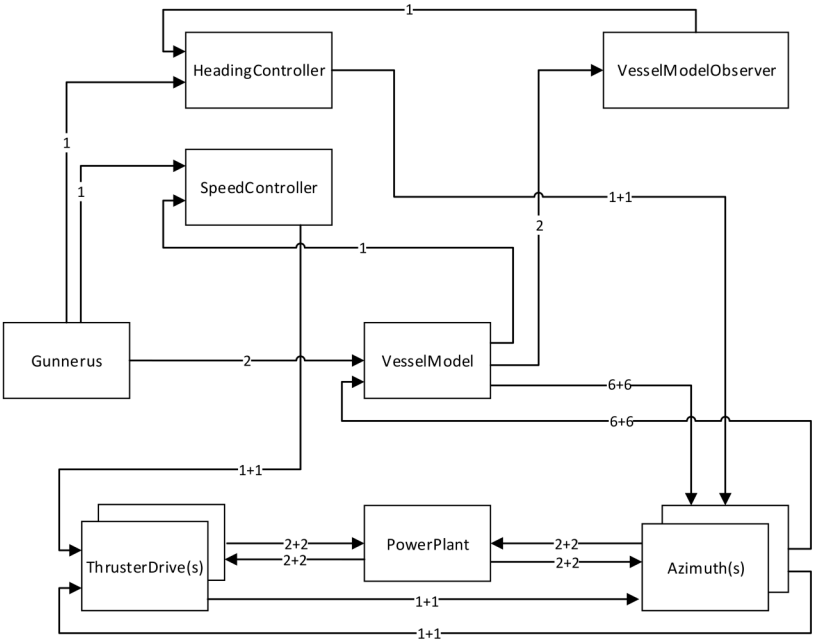


Fig. 12.30: Connection graph for the ship manoeuvring components for illustrative purposes. Numbers indicate the number of connected variables.

12.6.1.12 C12: Fidelity Considerations

The system requires high fidelity of the physics models, as the they would be used to predict anomalies and divergences between predicted and actual position of the ship. However, the complexity of the system could be decreased by removing certain behaviour models depending on the phenomena to be observed, i.e., removing the *ThrusterDrive* and *PowerPlant* models if one only wants to observe the hull characteristics.

12.6.1.13 C13: Life-cycle Stages

The DT for the ship is an open marine industrial infrastructure that serves multiple purposes. Firstly, it facilitates the overall system design by allowing system configuration and verification of operational performance. Additionally, it places significant emphasis on providing early warning capabilities, life cycle service support, and predicting system behaviour:

Design phase: in this phase, the ship could be modelled in a whole piece including all necessary important systems, so that the designer could find out if the design meets all technical requirements.

Manufacturing phase: all models and sub-models should be with high fidelity in order to make sure the single system with prefect performance. This phase should guarantee all components, systems running as expected. Depending on the complexity level, different models implementations contain different sets of properties describing the behaviour of the physical system.

Operation phase: during the operational phase, the DT becomes a system for integration, processing and analysis of the operational data. In this phase, many operational models in the DT ship could be light-weight and real-time so that ideally, the DT will provide decision support and predictions for a ship owner, subsystem manufacturers and safety training/improvement for crew and operators.

Maintenance and service: the DT tracks information on all parameters to define how each individual module and sub modules behaves over its entire useful life, including the initial design and further refinement, manufacturing related deviations, modifications, uncertainties, updates as well as sensor data from on-board systems, maintenance history and all available historical and fleet data obtained using data mining. All outcomes will be further used for ships that are either autonomous or remote-controlled for safety and reliability enhancement.

12.6.1.14 C14: Evolution

Table 12.5 details the reports on the development of this DT/DS.

Table 12.5: TwinShip articles and their topic.

Year	Topic	Citation
2019	Estimating useful life predictions	[3]
2020	Co-simulation framework to simulate Gunnerus case-study	[15]
2020	Gunnerus DT within a co-simulation environment	[16]
2021	Development of a ship trajectory predictor	[20]
2022	Improving trajectory prediction with a physics-based model	[22]
2023	Refining models between different ship size	[21]
2023	Enabling technologies for the Gunnerus DT	[39]

12.6.2 Summary and Future Work

This section has illustrated the research activity on a DT of the Gunnerus ship, especially focusing on co-simulation implementation. Key elements necessary for the success of this case-study include:

1. An onboard PC for collection and transmission of data.
2. A database with historic data for analytics.
3. High-fidelity models of the ship hull and internal systems, which is able to simulate the behaviour of the ship forward in time and perform what-if experiments.
4. Cellular 4G/5G coverage at sea as well as a reliable MQTT broker.
5. Real-time dashboard for information purposes.
6. 3D models for visualisation.
7. As some of the models involved depend on the same shared library, the simulation must be executed in part over multiple logical CPU processes. This can be solved by using a co-simulation orchestrator with distributed simulation facilities or using a wrapper solution like FMU-proxy [17] for the affected models.

Many challenges arise when trying to fully represent the physical counterpart by the twin system in a stochastic environment:

- **Developing high-fidelity models:** As seen in the Gunnerus twin ship case, the amount and quality of data is fundamental to establish high-fidelity models for the twin system, e.g., either for identification of system parameters, or for development of ML models.
- **Establishing a mechanism for model updating:** It is necessary to establish mechanisms to update models regularly, especially for these ML models. Thus, the computational cost of ML models, such as the training time for the model, must be relatively low, so that the model is able to adapt to operation/environmental changes.
- **Co-simulation:** Another challenge lies in co-simulation for DT implementation. Scalability must be considered when it comes to the industrial use of DTs. Insufficient synchronisation and tight coupling issues may arise if the system becomes complex and requires high computational effort. Domain-specific challenges in FMU implementation also need to be addressed.

The team at NTNU with our cooperated stakeholders will continue the work towards realising a full functional DT of the Gunnerus, gradually improving its accuracy. Continued development of the use-cases will provide meaningful on-board decision support for the crews of the Gunnerus.

References

1. Corke, P., Haviland, J.: Not your grandmother's toolbox–the Robotics Toolbox reinvented for Python. In: ICRA, pp. 11357–11363. IEEE (2021)
2. Ejersbo, H., Lausdahl, K., Frasheri, M., Esterle, L.: Dynamic Runtime Integration of New Models in Digital Twins. In: 2023 IEEE/ACM 18th Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), pp. 44–55 (2023). DOI: 10.1109/SEAMS59076.2023.00016
3. Ellefsen, A.L., Bjørlykhaug, E., Æsøy, V., Ushakov, S., Zhang, H.: Remaining useful life predictions for turbofan engine degradation using semi-supervised deep architecture. Reliability Engineering & System Safety **183**, 240–251 (2019)
4. Feng, H., Gomes, C., Thule, C., Lausdahl, K., Iosifidis, A., Larsen, P.G.: Introduction to Digital Twin Engineering. In: 2021 Annual Modeling and Simulation Conference (ANNSIM), pp. 1–12. IEEE. DOI: 10.23919/ANNSIM52504.2021.9552135
5. Feng, H., Gomes, C., Thule, C., Lausdahl, K., Sandberg, M., Larsen, P.G.: The Incubator Case Study for Digital Twin Engineering. URL http://arxiv.org/abs/2102.10390
6. Foldager, F., Balling, O., Gamble, C., Larsen, P.G., Boel, M., Green, O.: Design Space Exploration in the Development of Agricultural Robots. In: AgEng conference. Wageningen, The Netherlands (2018)
7. Frasheri, M., Ejersbo, H., Thule, C., Esterle, L.: RMQFMU: Bridging the Real World with Co-simulation For Practitioners. In: H.D. Macedo, C. Thule, K. Pierce (eds.) Proceedings of the 19th International Overture Workshop. Overture (2021)
8. Frasheri, M., Ejersbo, H., Thule, C., Gomes, C., Kvistgaard, J.L., Larsen, P.G., Esterle, L.: Addressing time discrepancy between digital and physical twins. Robotics and Autonomous Systems **161**, 104347 (2023). DOI: https://doi.org/10.1016/j.robot.2022.104347
9. Frasheri, M., Lumer-Klabbers, G., Kvistgaard, J.L., Hausted, J.O., Thule, C., Esterle, L., Larsen, P.G.: Building safer robots: From simulation to hardware deployment. In: 2023 27th International Conference on Methods and Models in Automation and Robotics (MMAR), pp. 63–68 (2023). DOI: 10.1109/MMAR58394.2023.10242467
10. Frasheri, M., Thule, C., Macedo, H.D., Lausdahl, K., Larsen, P.G., Esterle, L.: Fault injecting co-simulations for safety. In: The 5th International Conference on System Reliability and Safety ICSRS'21 (2021)
11. Gil, S., Mikkelsen, P.H., Tola, D., Schou, C., Larsen, P.G.: A Modeling Approach for Composed Digital Twins in Cooperative Systems. In: 2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA), pp. 1–8. IEEE (2023). DOI: 10.1109/ETFA54631.2023.10275601
12. Gil, S., Oakes, B., Gomes, C., Frasheri, M., Larsen, P.G.: Towards a systematic reporting framework for digital twins: A cooperative robotics case study. SIMULATION (2024). Accepted for publication
13. Gil, S., Schou, C., Mikkelsen, P.H., Larsen, P.G.: Integrating Skills into Digital Twins in Cooperative Systems. In: 2024 IEEE/SICE International Symposium on System Integration (SII), pp. 1124–1131. IEEE (2024). DOI: 10.1109/SII58957.2024.10417610
14. Hassani, V., Ross, A., Selvik, Ø., Fathi, D., Sprenger, F., Berg, T.E.: Time domain simulation model for research vessel Gunnerus. In: ASME 2015 34th International Conference on Ocean, Offshore and Arctic Engineering. American Society of Mechanical Engineers Digital Collection (2015)
15. Hatledal, L.I., Chu, Y., Styve, A., Zhang, H.: Vico: An entity-component-system based co-simulation framework. Simulation Modelling Practice and Theory **108**, 102243 (2021). DOI: https://doi.org/10.1016/j.simpat.2020.102243
16. Hatledal, L.I., Skulstad, R., Li, G., Styve, A., Zhang, H.: Co-simulation as a Fundamental Technology for Twin Ships. Modeling, Identification and Control **41**(4), 297–311 (2020). DOI: 10.4173/mic.2020.4.2

17. Hatledal, L.I., Styve, A., Hovland, G., Zhang, H.: A language and platform independent co-simulation framework based on the functional mock-up interface. IEEE Access **7**, 109328–109339 (2019). DOI: 10.1109/ACCESS.2019.2933275
18. Hausted, J.O., Kvistgaard, J.L., Lumer-Klabbers, G.: Towards a Digital Twin for an Autonomous Robot. Aarhus University, BSc thesis (2020)
19. Kanazawa, M., Hatledal, L.I., Li, G., Zhang, H.: Co-simulation-Based Pre-training of a Ship Trajectory Predictor. In: International Conference on Software Engineering and Formal Methods, pp. 173–188. Springer (2021)
20. Kanazawa, M., Skulstad, R., Li, G., Hatledal, L.I., Zhang, H.: A multiple-output hybrid ship trajectory predictor with consideration for future command assumption. IEEE Sensors Journal **21**(23), 27124–27135 (2021). DOI: 10.1109/JSEN.2021.3119069
21. Kanazawa, M., Wang, T., Ichinose, Y., Skulstad, R., Li, G., Zhang, H.: Bridging similar ships' dynamics for safeguarding the system identification of maneuvering models. Ocean Engineering **280**, 114874 (2023). DOI: https://doi.org/10.1016/j.oceaneng.2023.114874
22. Kanazawa, M., Wang, T., Skulstad, R., Li, G., Zhang, H.: Knowledge and data in cooperative modeling: Case studies on ship trajectory prediction. Ocean Engineering **266**, 112998 (2022). DOI: https://doi.org/10.1016/j.oceaneng.2022.112998
23. Legaard, C.M., Tola, D., Schranz, T., Macedo, H.D., Larsen, P.G.: A Universal Mechanism for Implementing Functional Mock-up Units. In: G. Wagner, F. Werner, T.I. Ören, F.D. Rango (eds.) Proceedings of the 11th International Conference on Simulation and Modeling Methodologies, Technologies and Applications, SIMULTECH 2021, Online Streaming, July 7-9, 2021, pp. 121–129. SCITEPRESS (2021). DOI: 10.5220/0010577601210129
24. Lehner, D., Gil, S., Mikkelsen, P.H., Larsen, P.G., Wimmer, M.: An architectural extension for digital twin platforms to leverage behavioral models. In: 2023 IEEE 19th International Conference on Automation Science and Engineering (CASE), pp. 1–8 (2023). DOI: 10.1109/CASE56687.2023.10260417
25. Liu, M., Fang, S., Dong, H., Xu, C.: Review of digital twin about concepts, technologies, and industrial applications. Journal of Manufacturing Systems **58**, 346–361 (2021). DOI: https://doi.org/10.1016/j.jmsy.2020.06.017. Digital Twin towards Smart Manufacturing and Industry 4.0
26. Lumer-Klabbers, G., Hausted, J.O., Kvistgaard, J.L., Macedo, H.D., Frasheri, M., Larsen, P.G.: Towards a digital twin framework for autonomous robots. In: SESS: The 5th IEEE International Workshop on Software Engineering for Smart Systems. COMPSAC 2021, IEEE (2021)
27. Madsen, E., Tola, D., Hansen, C., Gomes, C., Larsen, P.G.: AURT: A Tool for Dynamics Calibration of Robot Manipulators. In: 2022 IEEE/SICE International Symposium on System Integration (SII), pp. 190–195 (2022). DOI: 10.1109/SII52469.2022.9708769
28. Oakes, B., Parsai, A., Van Mierlo, S., Demeyer, S., Denil, J., De Meulenaere, P., Vangheluwe, H.: Improving Digital Twin Experience Reports. In: Proceedings of the 9th International Conference on Model-Driven Engineering and Software Development, pp. 179–190. SCITEPRESS - Science and Technology Publications. DOI: 10.5220/0010236101790190
29. Oakes, B.J., Gomes, C., Larsen, P.G., Denil, J., Deantoni, J., Cambeiro, J., Fitzgerald, J.: Examining model qualities and their impact on digital twins. In: 2023 Annual Modeling and Simulation Conference (ANNSIM), pp. 220–232. IEEE (2023)
30. Oakes, B.J., Parsai, A., Meyers, B., David, I., Van Mierlo, S., Demeyer, S., Denil, J., De Meulenaere, P., Vangheluwe, H.: A Digital Twin Description Framework and its Mapping to Asset Administration Shell. In: Model-Driven Engineering and Software Development, vol. 1708, pp. 1–24. Springer Nature Switzerland. DOI: 10.1007/978-3-031-38821-7_1
31. Oakes, B.J., et al.: Structuring and accessing knowledge for historical and streaming digital twins. In: 1st Workshop on Ontology-Driven Conceptual Modeling of Digital Twins, pp. 1–13
32. Schou, C., Andersen, R.S., Chrysostomou, D., Bøgh, S., Madsen, O.: Skill-based instruction of collaborative robots in industrial settings. Robotics and Computer-Integrated Manufacturing **53**(June 2016), 72–80 (2018). DOI: 10.1016/j.rcim.2018.03.008

33. Skjetne, R., Sørensen, M.E., Breivik, M., Værnø, S.A., Brodtkorb, A.H., Sørensen, A.J., Kjerstad, Ø.K., Calabrò, V., Vinje, B.O.: AMOS DP research cruise 2016: Academic full-scale testing of experimental dynamic positioning control algorithms onboard R/V Gunnerus. In: International Conference on Offshore Mechanics and Arctic Engineering, vol. 57632, p. V001T01A080. American Society of Mechanical Engineers (2017)
34. Skjong, S., Pedersen, E.: A real-time simulator framework for marine power plants with weak power grids. Mechatronics **47**, 24–36 (2017)
35. Talasila, P., Gomes, C., Mikkelsen, P.H., Arboleda, S.G., Kamburjan, E., Larsen, P.G.: Digital Twin as a Service (DTaaS): A Platform for Digital Twin Developers and Users. In: 2023 IEEE Smart World Congress (SWC), pp. 1–8 (2023). DOI: 10.1109/SWC57546.2023.10448890
36. Van Mierlo, S., Oakes, B.J., Van Acker, B., Eslampanah, R., Denil, J., Vangheluwe, H.: Exploring validity frames in practice. In: Proceedings of the First International Conference, ICSMM 2020, Bergen, Norway, June 25–26, 2020, pp. 131–148. Springer, Cham. DOI: 10.1007/978-3-030-58167-1_10
37. Wang, T., Li, G., Hatledal, L.I., Skulstad, R., Æsøy, V., Zhang, H.: Incorporating approximate dynamics into data-driven calibrator: A representative model for ship maneuvering prediction. IEEE Transactions on Industrial Informatics **18**(3), 1781–1789 (2021)
38. Wang, T., Skulstad, R., Kanazawa, M., Li, G., Æsøy, V., Zhang, H.: Physics-informed data-driven approach for ship docking prediction. In: 2022 IEEE International Conference on Real-time Computing and Robotics (RCAR), pp. 111–117. IEEE (2022)
39. Zhang, H., Li, G., Hatledal, L.I., Chu, Y., Ellefsen, A., Han, P., Major, P., Skulstad, R., Wang, T., Hildre, H.P.: A Digital Twin of the Research Vessel Gunnerus for Lifecycle Services: Outlining Key Technologies. IEEE Robotics and Automation Magazine **30**(3), 6–19 (2023). DOI: 10.1109/MRA.2022.3217745

Part V
Advanced Topics

This final part covers advanced or research-based topics that relate to our ability to deliver dependable cyber-physical systems enabled by Digital Twins (DTs). Given the ability of a DT to exercise some control over a Physical Twin (PT) and the capacity to develop DTs that act with a measure of autonomy, it is important to consider the verification of key safety and security properties. Chapter 13 considers the balance between the benefits of DTs and the risks that come from networked communications between the real and virtual worlds. Chap. 14 takes us forward to the stage where the DT is able to reconfigure the PT autonomously. Finally, Chap. 15 rounds off the book by identifying a range of possible directions for future research and innovation addressing challenges in DT dependability, formal verification, platforms and novel applications.

Chapter 13
Security and Privacy-related Issues in a Digital Twin Context

Tomas Kulik, Zahra Kazemi and Peter Gorm Larsen

Abstract Digital Twin (DT)-enabled systems depend on communications and networking, and so face a range of threats to their security. This chapter reviews approaches to managing or mitigating these threats. Approaches that are already well established in the software industry, techniques based on formal methods and finally, control theory approaches using features such as state estimation ensure that the developers of DT-enabled systems are faced with a plethora of options. The chapter also presents a standardised notation for communication of security features between engineers working within the DT area.

13.1 Introduction

This chapter introduces security-related issues relevant to Digital Twins (DTs). The data exchanged between the PT and its DT is typically communicated close to real-time, and in some cases can carry critical signals. Thus, the need for cyber security assurance within the systems employing the DT technology becomes important. Recently, an increase in attack surfaces within the industrial control systems has been observed [27], largely due to the additional connectivity that is being added to these systems in order to satisfy increasing requirements for data availability. Utilising a DT further extends the attack surface because the models underlying the DS/DT might also become attack vectors.

It is important to consider the nature of the DT itself. In order to provide a good representation of the PT it needs to be provided with a multitude of signals from the PT. Furthermore the DT can often be used as a reconfiguration tool for the PT, sending a signal to the PT on how to reconfigure itself. To this extent the DT is often seen as a solution to the security problem rather than as an increasing attack vector [15]. While a DT can often act as a security component, a question of what happens if the DT itself is compromised needs to be asked [19]. This chapter attempts

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

J. Fitzgerald et al. (eds.), *The Engineering of Digital Twins*,
https://doi.org/10.1007/978-3-031-66719-0_13

to answer the question of how we can trust a DT within critical systems. It does this by introducing example attacks within a DT environment, considering the impacts of these attacks and discussing the protections against such attacks.

A final consideration within a DT enabled systems is sharing of the data from the DT between multiple types of users with different requirements. For example, some users require read access to a DT while others need write access. It is often necessary to ensure that different entities can access different parts of a DT, requiring strong isolation in order to prevent disclosure of sensitive information. This becomes an important challenge in cases where a DT is hosted by a service provider separate from the system and the DT designers as well as the system operators. Protection of different stakeholders' privacy is also important, especially in systems where user interactions with the DT critical, for example in healthcare applications [11, 24].

Security and privacy in DT ecosystems have not been studied for long as an integrated discipline, but fall into the areas of security of digital communication and control systems. Currently the trends are emerging where DT-based solutions merit specific consideration [3]. It could be argued that, with increasing complexity of DT-enabled systems, security topics are expected to become one of the most important design aspects of these systems. Different kinds of model checking technologies are relevant in this context [18].

This chapter starts with an overview of the general security architecture for DTs in Sect. 13.2. This is followed in Sect. 13.3 with approaches that can be used for mitigating different security and privacy attacks. Section 13.4 considers Intellectual Property (IP) protection approaches before Sect. 13.5 finally considers security concerns in the real world.

13.2 DT Security Architecture

In order to discuss the security of DT-enabled systems one must consider their architecture. Often the PT carries out actions that impact the physical environment, while the DT is executed remotely. The PT and its DT are then interconnected using one or more communication protocols, depending on the complexity of the system. A simple architectural model for the incubator is presented in Fig. 13.1. Even a simple system like this can expose multiple DT services potentially running on several independent computers, requiring a network connection.

The presented architecture uses the RabbitMQ message broker and the Advanced Message Queuing Protocol (AMQP) protocol flowing from the DT to the PT. In order to secure this communication several questions have to be asked. What is the nature of the data transmitted? How could the PT deal with communication delays? Does DT communicate any commands back to the PT? For example, if it is only essential to protect the data flowing from DT to the PT, data encryption might be considered sufficient. The question then becomes about the responsibility of carrying out this

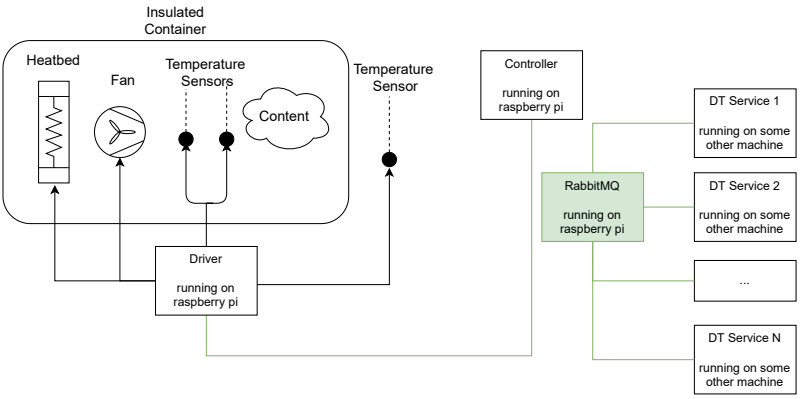


Fig. 13.1: Diagram of the communication among different DT services of the incubator.

encryption, as the DT can encrypt the data and provide it over an open channel, or the AMQP could be setup with network encryption using Transport Layer Security (TLS), which would furthermore let the AMQP client know it is connecting to the expected broker. If only a specific PT can provide data to a DT's services, as well as only specific DT services having access to this data, a mutual authentication using TLS can authenticate the PT and the DT service AMQP clients as well as provide a network encryption [38]. The decision on use of different security mechanisms often has to be taken considering the impact on data availability and potential latency it would introduce.

In this chapter we consider five cyber attacks applicable to DT-enabled systems. Four of them, the *man-in-the-middle*, *command injection*, *denial of service and data modification* are well known from communication and control systems, while the final attack, *model malware*, is known from the area of industrial automation. These attacks present a good base for discussion of cyber security within DTs, especially since they can have a significant impact on the sensing capabilities of DTs as presented in Chap. 7. It has to be noted that these attacks do not present an exhaustive list of possible cyber attacks. The security mechanisms described in this chapter are applicable to a wide variety of potential cyber attacks. In order to provide a full overview the attacks are described in detail as:

Man-in-the-middle attack: An attack in which a malicious actor joins a network channel used to transport data between the PT and the DT. The malicious actor is not seeking to actively disturb the operations of the DT-enabled system but rather obtain data from it in order to extract potential value from this data as shown in Fig. 13.2. The value obtained by the malicious attacker can be utilised for learning more about the DT-enabled system and subsequently using this knowledge for further attacks or simply directly obtaining secret information from the data transmitted.

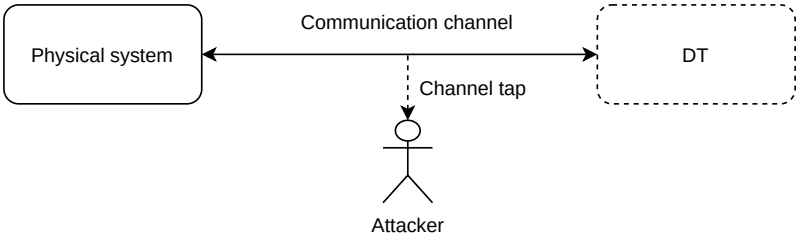


Fig. 13.2: Man-in-the-middle attack within a DT-enabled system

Command injection attack: An attack in which a malicious actor joins a network channel used to transport commands from the DT to the PT and issues malicious commands. This attack is aimed directly at disturbing operations of the PT in order to damage its intended operation. The positioning of the attacker and the DT-enabled system components is shown in Fig. 13.3. Attacks of this type are known from the field of industrial control systems [37] where, if undetected, they could result in the destruction of the physical equipment that the control system is operating. In order to be able to inject the commands, the attacker needs to have knowledge about the DT-enabled system and formatting of the command data packets. This could for example be obtained by utilising the man-in-the-middle attack prior to attacking it using a command injection attack.

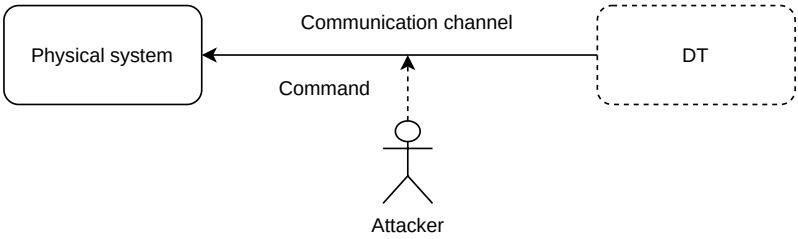


Fig. 13.3: Command injection attack within a DT-enabled system

Denial of Service (DoS) attack: An attack in which a malicious actor attempts to disturb the DT-enabled system by either delaying or altogether preventing the data transfer either from the PT to the DT or from the DT to the PT. Both options are explored in Fig. 13.4. Depending on the nature of the DT-enabled system this can have critical impact as the DT can lose overview of the state of the PT and hence mislead a potential operator. The PT might not receive important parameter updates or commands from the DT leading to potential damage of the system or the system might simply shut down due to the unavailability of the necessary data. In order to execute this type of attack, the attacker needs to flood the communication channel

with messages to such an extent that it overwhelms the processing capabilities of the affected communication endpoints.

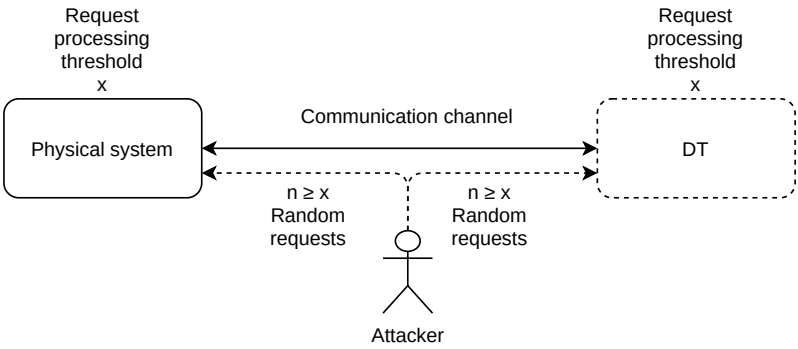


Fig. 13.4: Denial of Service attacks within a DT-enabled system

Data modification attack: An attack in which a malicious actor attempts to modify the data transferred from the PT. This attack is aimed on providing false information about the state of the PT to the operator of the DT-enabled system as well as to the DT. If the DT acts as a supervisory controller in the loop, modified data might cause wrong regulation input and subsequent damage to the PT. The original data flow is often intercepted by the man-in-the-middle attack and then modified by the attacker [49] as shown in Fig. 13.5. Depending on the nature of the DT-enabled system, small modifications to the values would lead to incorrect regulation if left undetected for a longer time period.

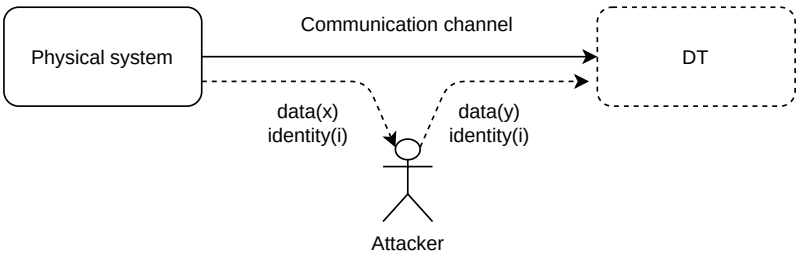


Fig. 13.5: Data modification attack within a DT-enabled system

Model malware attack: An attack in which a DT is targeted directly. The attacker aims to inject malware to a DT model, in order to impose malicious behaviour on a DT. In order to execute the attack, the attacker either introduces malware directly to the content of the model by gaining access to the repository or indirectly by either

introducing malware to some of the libraries that DT is based upon or even if a DT is capable of side loading executable code, by injecting it directly as presented within Fig. 13.6. In order to succeed the attacker must have prior knowledge of the DT and its execution. The infected model can then either mislead the system operator or even issue commands to the system causing operational problems. Similar attacks are known from the area of industrial automation, specifically control loop programs executed by Programmable Logic Controllers (PLCs) [58].

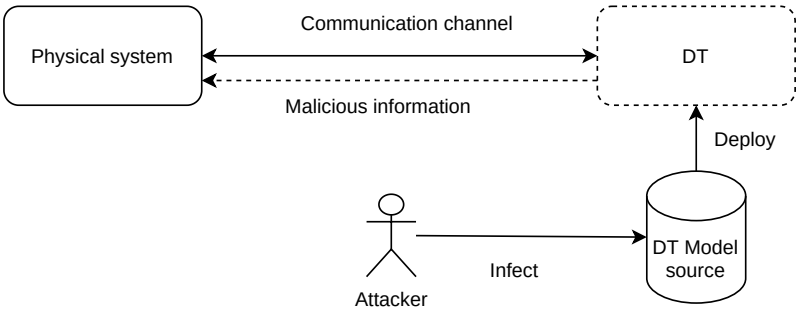


Fig. 13.6: Model malware attack aimed at the DT within a DT-enabled system

Security Attacks on the Incubator

Consider how these attacks may impact the DT-enabled incubator system. Considering the *Man-in-the-middle* attack, the attacker might dump data packets transmitted over the AMQP protocol and gather information about the system by observing the exchanged data. With the *Command injection* attack, the attacker can send a command to the PT, for example to have the heater on continuously, leading to overheating of the system. With the *denial of service* attack, the attacker can simply prevent the communication between the incubator and its DT, leaving the DT without knowledge of the system state. In case that the DT acts as a primary regulator, this would practically stop the regulation of the active elements within the incubator. The *Data modification* attack would allow the attacker to change the content of the value provided by the incubator to the DT, where the regulation provided by the DT would not correspond to the real situation of the incubator and would lead to too low or high temperature. Finally, the *Model malware* attack would affect the DT services running on separate machines, considering for example the DT service responsible for the heater once again the malware in the model might simply offset the target temperature and hence keeping it too low or too high. The attacker would simply provide an innocent library for example logging library that if permitted by the programming stack might gain access to global and local variables within different scopes [44].

These are only some of the attacks that could be anticipated by systems engineers for the overall DT-enabled system. It is the aim of this chapter to show how to mitigate these attacks as well as provide a toolkit for understanding how other types of attack could be mitigated. The different attacks presented share several properties, and utilisation of access control technologies and encryption provides good starting points for a secure DT-enabled system.

13.3 Approaches to a DT Security and Privacy

Having identified some possible attacks, we now look at how to defend against them. It has to be noted that DT engineering is an evolving field and so it could be expected that tooling specifically for security analysis of DTs will become available. An interesting approach is the use of frameworks based on the game theory, where a non-cooperative game model is used to assess behaviour of different nodes within the system, including DTs [12]. This chapter considers several different approaches to security and privacy challenges: *Standard* approaches, which are widely used within the software industry not necessarily specifically aimed at DTs, *Formal Methods based* approaches, that utilise formal (logical and mathematical) analysis to discover and address security challenges, and finally *Control theory based* approaches utilising control theory and state estimation to address the potential security challenges.

13.3.1 Standard approaches to cyber security in a DT context

The standard approaches are those that have been widely used in the software industry. This chapter considers the security by design [8, 17] approach, security reviews and risk assessments in this group. In addition, concepts such as encryption and authentication play a significant role within a secure DT ecosystem. Finally, all of the approaches need to consider operational trade-offs between flexibility and security.

The **Security by Design** approach is built upon a principle that the foundation of security starts within the design of the system. This specifically means that the design process is not only concerned with the features and operations of the DT but from the very start of the design process considers how to make sure that the features of the DT are secure. One way to carry out this process is using the Confidentiality, Integrity and Availability (CIA) framework, taking into account the basic attributes of *Confidentiality, Integrity* and *Availability* of the information within the DT enabled systems. The attributes are complementary, as indicated in Fig. 13.7. In the case of the DT-enabled system, further attributes that need to be considered include *Safety*

and *Accountability*. Security by Design employs security patterns (often based on best practices) to satisfy the CIA and other attributes. Within the area of a DT enabled system, the considerations could be:

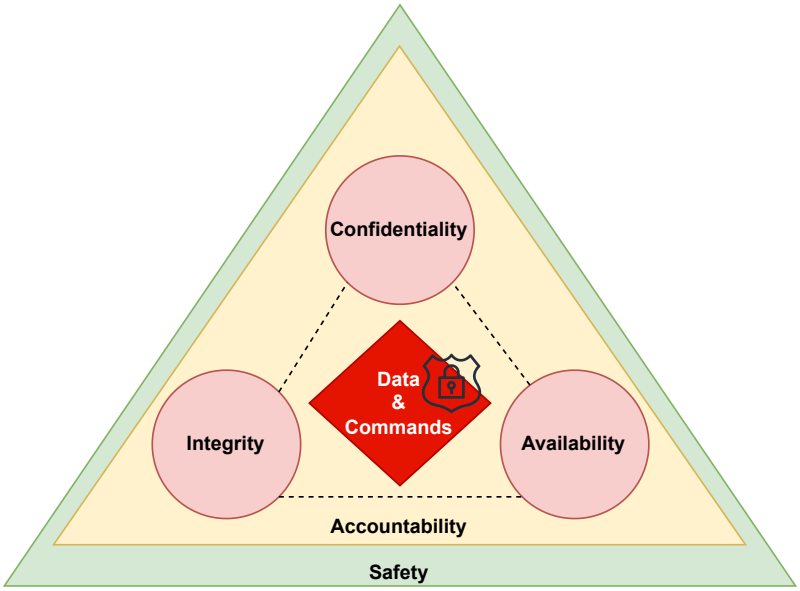


Fig. 13.7: The enhanced Confidentiality, Integrity and Availability framework

Confidentiality making sure the information transmitted on the channel stays secret. From the best practices, an approach with channel encryption or data encryption could be used, with the specific encryption algorithm adapted to the DT use case. However, a foundational requirement for secrecy of exchanged information shall be considered within the DT-enabled system as even data which seemingly does not need to be considered secret would provide a lot of information out the system to a potential attacker.

Integrity ensuring that the information exchanged within the system is accurate and consistent, i.e., the information carries the true value within an expected range of values. To achieve this, a so called semantic integrity it is necessary for the DT to have a mechanism that can establish what the correct range of values is. The strategies for information integrity could be input validation, error detection and cryptographic signing of data blocks carrying the information providing tampering protection.

Availability making sure the information is available within the system even under degraded conditions (for example, if the system is under attack). The availability requirements again depend on the use case for the DT-enabled system,

however generally the information needs to be available between the PT and the DT in order for the DT to operate correctly. Different strategies for ensuring information availability are use of redundancy and avoiding single points of failure, for example a DT can consist of multiple redundant processes where these processes use independent communication channels.

Safety ensuring the system remains in a safe operation state or shut downs safely in case of a disturbance. The safety requirements again depend on the use case of the system, however prevention of the unsafe condition of the PT is often the priority. In case of a DT-enabled system, the DT itself could be used to detect unsafe conditions. In order for this, the CIA attributes must be satisfied. One approach to increasing PT safety is utilisation of what-if analysis performed on the DT [40], i.e. executing potential commands against the DT with a current system state and only pushing these commands to the PT if the DT does not execute unsafe behaviour.

Accountability making sure that the process or an operator that has executed a specific command or series of commands can be identified and the identifying information is readily available. This is especially important in complex environments where a DT might consist of multiple processes with the ability to issue commands to the PT. One strategy for providing accountability is wide use of logging of operations and collecting the logs to a central system (e.g., inside the DT).

One standard approach to security is use of an island mode, or an isolation of the system from external systems. This approach has been used for a long time within the area of industrial control systems. Currently, industrial control systems are becoming more interconnected which increases their utilisation but also poses additional security risks [53]. The isolation approach could also be applied to DT-enabled systems, where both the DT and the PT would be in close proximity, connected on a local network without any connectivity to wider external networks, which would however in several ways decrease the utility of the DT. For example remote monitoring or model updates would not be possible. This is why isolation is regarded as an option to secure the system, but to be used sparingly. A possible use is a fallback mode where, in case of a cyber attack (or even an increased threat of a cyber attack), the connectivity to a remote network is deactivated and the DT-enabled system operates in an island mode.

The possible utilisation of island mode leads to an important consideration of trade-offs between flexibility and security. The security options that the system designers face need to be tailored to the system at hand. Several trade-off considerations are:

1. Remote monitoring versus local monitoring only
2. Remote DT updates versus local updating only
3. Remote commanding versus local commanding only
4. Direct operations versus network analysis

One of the most important points for a system designer is to decide whether the security requirements are strong enough to utilise a DT. In addition, the direct operations (not analysing the network packets) could lead to lower latency within the system. This can be important for close-to-real-time systems.

The concepts often used to satisfy the cyber security requirements are *encryption*, *authentication* and *authorisation*.

The concept of encryption provides secrecy to the data within the system. This could be for data in transit, data at rest (in a data store), data loaded to the system memory and in some cases even data that is being manipulated [1]. The basic concept considers cleartext data d, cryptographic key (secret) k and a cryptographic function f. The cipher-text c (encrypted data) is created as $c = f(d, k)$ and then d can be obtained back as $d = f^{-1}(c, k)$, in this case a symmetric cryptography has been used where a single key is used for both encryption and decryption and it needs to be transported securely between the entities that are using it. Another option is use of asymmetric cryptography that utilises key pairs as public and private key. In this case a public key pk and a private key sk are forming a key pair (pk, sk). The public key does not contain secret information and can be freely shared. The data is then encrypted as $c = f(d, pk)$ and can be decrypted as $d = f^{-1}(c, sk)$. It is important that sk remains protected. It is beyond the scope of this book to discuss individual encryption functions and advanced mechanisms, but in the case of a DT-enabled system it is important to determine how to use encryption to protect sensitive data while ensuring that the encryption overhead does not interfere with the operation of the system.

The concept of authentication is the ability to prove the identity of a specific principal. This could for example be a computing service or a user. The authentication practice ensures that only entities possessing a specific piece of information can be allowed to interact with the system. For example, this could be a password, a cryptographic token or some other secret. The DT-enabled system then knows the entities that are allowed to interact with it and checks their secret before allowing interaction. An example is a *basic* authentication known from use with HTTP, where a username and password (credential) is simply sent as clear text. The DT-enabled system then receives the credential and checks it against a database of known users. This database typically stores the username and cryptographic hash of a password. The system computes the hash of the received password, and if it matches the user is allowed to interact with the system. This is just a simple example as it is possible to utilise many different authentication mechanisms utilising different types of credentials. The Simple Authentication and Security Layer (SASL) provides evolving definitions for multitude of mechanisms and protocols [21] and is often

used within computing systems. The DT-enabled system designers need to consider authentication mechanisms in order to ensure that only the DT services with valid authentication can interact with the physical system.

Finally, the concept of authorisation builds on top of authentication by adding granularity, stating which entities have access to what resources. This can be accomplished by use of Access Control Lists (ACLs) where a specific resource such as a module within the physical system can only be accessed by a specific module within the DT. In this case ACL defines the necessary access permissions, very often in categories or as read and write permission from and to the resource. For example an entity x wants to access the resource R, the access control list would define: $x \rightarrow rR$ or $x \rightarrow wR$ or $x \rightarrow rwR$ for x is allowed to read R, x is allowed to write R and x is allowed to read and write R. This is a basic example that works on a *deny by default* principle, i.e., if no ACL rule is defined, x is denied all operations on R. This does not need to be the case as ACLs can also be used to specify deny rules. The system needs to be able to store and enforce ACLs. This leads to the need to manage the ACLs on a system level, often on individual components carrying the resources. Another option is utilisation of Role Based Access Control (RBAC), which allows for centralised management of roles and assignment of component principals (users) to specific groups. As RBAC is often managed centrally, it is suitable for larger fast evolving systems [4]. These are the two most utilised approaches that span from networked systems to operating systems. The DT-enabled system designers need to consider the scale of the system and its evolution when choosing the authorisation strategy as well as consider use of authorisation frameworks such as for example OAuth 2.0 [6] (this was also used in Sect. 11.9).

During the design process of a DT-enabled system it is important to undertake *Rrisk Assessment* and *Security Review*. Risk assessment is a process that identifies security risks to a DT-enabled system, which then are to be mitigated. This process usually follows a specific model for identification of security threats. One commonly used model (thanks to the support of modelling tools) is the STRIDE model covering threats from areas of Spoofing, Tampering, Repudiation, Information disclosure, Denial of Service and Elevation of Privilege [29]. The security review is a process that is used to review the proposed mitigations and determine if the security the provide is sufficient by identifying the residual risks and deciding if these are to be accepted. If the risks are too high to be accepted, specific actions are taken, such as redesign or mitigation improvements. The system can only be built after a successful security review. This process then starts over for new features added to the system. The full process is shown in Fig. 13.8. It is important that the system features are built with minimum known vulnerabilities and accepted risks are not of a critical nature.

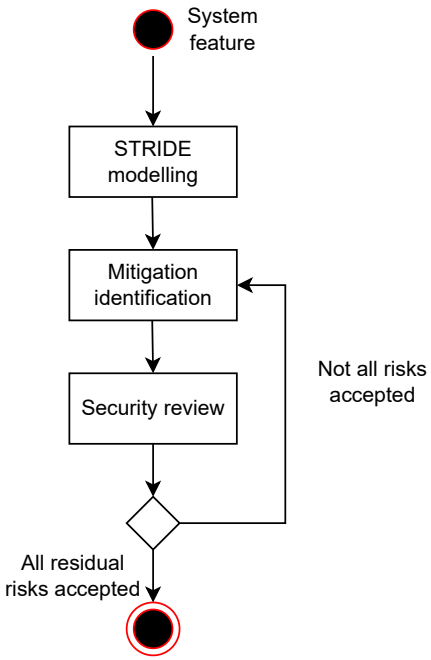


Fig. 13.8: Security process in the design stage

13.3.2 Formal Methods-Based Approaches to Cyber Security for DTs

In some cases the DT-enabled system is of a critical nature and designers must provide formal (in the sense of being highly mathematically rigorous) guarantees of certain attributes of the system [56]. Formal methods have been used to analyse security aspects of different types of systems and provide security guarantees on multiple levels of abstraction [31]. An example is model checking, a type of formal analysis where a state space of a formal model is explored for violation of given properties also expressed in a specific formal notation. Another approach is the use of theorem proving, or expressing the system as a set of theorems and proving their validity. Finally, an approach sometimes used is guided formal validation, where a formal model of the system is created and then many scenarios are executed on the model level (this was already introduced in Sect. 5.6 and in Sect. 9.2).

In all cases, the system is formally specified and then subsequently analysed. The guarantees provided by this approach are considered strong because of the rigour of the mathematical assessment. It needs to be noted that the use of formal methods presents its own challenges, for example a learning curve for extra set of techniques as well as assurance that the implemented system follows the model used for formal analysis.

To demonstrate the approach, let us consider a simple high level model of a DT model update for the incubator. This example is based on a model checking and formal specification system called Temporal Logic of Actions (TLA+) [35]. Consider a system requirement that the DT model is considered potentially malicious if it does not pass a series of checks. This could be checking the validity of a cryptographic signature, checking the validity of the integrity of the model and checking the expected size of the model. This can be expressed more compactly for a model m as: $is_malicious(m) = \neg\, sigValid(m) \vee \neg\, integValid(m) \vee size(m) > limit$ where the $limit$ is an arbitrary size limit for a valid model. Furthermore let us consider a system with a simple set of state transitions in order to transport the model to the DT. This set of transitions is shown in figure Fig. 13.9. The set of transitions can then be encoded in the formalism known as TLA+, utilising an algorithm modelling language PlusCal [36]. The property checking that the received model is not malicious can then be noted as $is_valid(m) = \neg is_malicious(m)$. In order for the model checker to be able to search for this property it needs to be expressed in a Linear Temporal Logic (LTL) formalism (also used for monitoring purposes in Chap. 9). In summary, several components need to be used together to carry out the formal analysis, these are:

1. Formal model
2. Property to verify
3. TLC model checker

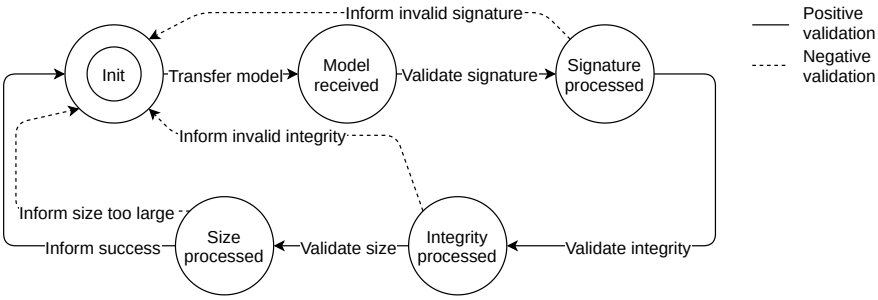


Fig. 13.9: Model delivery state transitions

Model checking suffers from the problem of state space explosion, which refers to the fact that the number of states in a system grows exponentially with the number of variables in the system [10] (recall Sect. 5.6.3). Consequently, it is regarded as being feasible for small systems or abstract models of a complicated system, although research and innovation in model checking is ongoing.

The formal model represents the behaviour of the system. The system model created in PlusCal is shown in Listing 13.1. The model covers the functionality of receiving the model and validating it against the specified conditions.

Listing 13.1: Processing of a DT model update (PlusCal)

```
fair process dt = DT
begin
DTBegin:
   skip;
   DTReceiveModel:
      if ModelHasValidSignature(dtModel) then
         DTModelValidSignature:
            \* Perform integrity check
            if ModelIsGenuine(dtModel) then
               \* Check for size
               DTModelGenuine:
                  if ModelFitsStorage(dtModel) then
                     DTModelSpaceAvailable:
                        \* Store model
                        assert dtStorage = NO_MODEL;
                        dtModelStatus := MODEL_STATUS_OK;
                        dtStorage := dtModel;
                  else
                     DTModelSpaceNotAvailable:
                        \* Security log: wrong size
                        dtModelStatus :=
                            MODEL_STATUS_INSUFFICIENT_STORAGE;
                  end if;
            else
               DTModelNotGenuine:
                  \* Security log: not genuine
                  dtModelStatus := MODEL_STATUS_NOT_GENUINE;
                  skip;
            end if;
      else
         DTModelInvalidSignature:
            \* Security log: invalid signature
            dtModelStatus := MODEL_STATUS_INVALID_SIGNATURE;
            skip;
      end if;
   DTHandlePendingRequests:
         goto DTBegin;
end process;
```

The property to verify is the property the model of the DT-enabled system is analysed against. This property can be expressed in LTL is as in Listing 13.2. The property simply states that it is expected that the model is always valid.

Listing 13.2: TLA property related to the model validation

```
DTModelAlwaysValid ≜
 □(ModelValid(dtInstalledModel) ∧
  (dtStorage ≠ NO_MODEL ⇒ ModelValid(dtStorage)))
```

The TLC model checker is used to build the entire state space of the system given by the model and tries to find a path in which the property does not hold. It can then provide a counterexample showing at which state of the system model the property does not hold.

These methods make it possible to explore the security properties of the system deeply. While the example only assesses very high-level functionality, it can be broken down and expressed in more detail, depending on the needs of the system designer. Often formal methods are used alongside standard approaches. For example the output of the risk assessment could be used to determine the specific security properties that need to be analysed by use of formal methods. In this way the approaches are complementary.

It may be necessary to specify system models, potential attacks and mitigations even without the need for formal analysis. This can be accomplished if the system design is to be communicated among a wide variety of personnel working on different aspects of the system. An example approach is the use of semi-formal notation that provides a specific way of describing a system or a problem while not necessarily utilising rigorous mathematics. In the area of DTs a specific notation has been proposed to communicate about their security aspects [32]. The notation is based on context diagrams from the Problem Frames notation, and is meant to provide a clear way of communicating between DT engineers, minimising (in many cases complementing) the use of natural language that can lead to misunderstandings.

The context diagrams are composed of domains that describe the key aspects and participants, while not describing the system architecture. In fact, they only contain one machine domain representing "the software" or "the computer". In our context diagrams it is always called the *Digital Twin*. Some domains can be controlled like the *CPS* (whose behaviour in general is predictable and therefore called *causal* and marked with a C). It is the objective of the DT to re-configure or augment the CPS. The behaviour of the *Attacker*, whose domain is called *biddable* and marked with a B, is not predictable in causal terms. Another relevant kind of domain is called *lexical*, representing some form of data. Domains that are *designed* are marked with one vertical bar on the left and machine domains with two vertical bars. Domains that interact in some way are connected by edges that are annotated with the phenomena they share. Phenomena are abstractions of concepts from the real world that can be observed or measured (for a more thorough discussion see [22, 23]). In order to emphasise the role of the network, we draw all edges connecting to it by "double bars". Attacks are then described in oval dashed ellipses using natural language and reference to points of the system they interact with. An example of this is a simple man-in-the-middle attack as defined in Sect. 13.2 and is shown in Fig. 13.10, where it can be compared with the informal diagrams first presented in this chapter.

The notation is being used actively within the area of DT security and is further being expanded upon [42]. As with any discipline in digital security, clear communication of vulnerabilities, attack and mitigations is one of the cornerstones of secure systems.

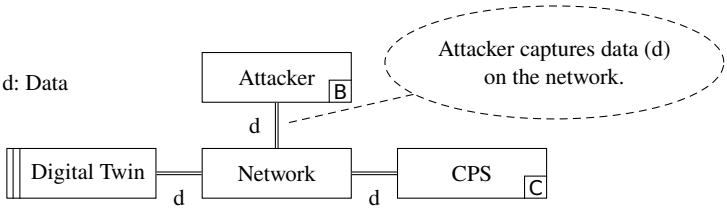


Fig. 13.10: Context diagram for man-in-the-middle attack.

13.3.3 Attack mitigations

This section will focus on mitigation of cyber attacks against a DT-enabled system. It will explore standard mitigations but also describe use of formal methods against a DT-enabled system at runtime. Several techniques have been described previously, namely use of encryption and authentication. Let us consider how these techniques could be utilised in mitigating the attacks defined in Sect. 13.2. The mitigations for most of the attacks are presented informally while an example of semi-formal mitigation definition is presented for the *Man-in-the-middle* attack.

The *Man-in-the-middle* attack can be mitigated using channel or data encryption. The difference between these two approaches is that for channel encryption a tunnel between the communication is created that encrypts all of the traffic flowing through that channel. This could for example be use of the TLS protocol. Data encryption on the other hand encrypts the data packets prior to sending them and then transfers them over an open channel, this could be used on a hardware not suitable for use of the TLS, for example in an IoT setting [50] or on network types not supporting channel encryption. An example of channel encryption is shown in Fig. 13.11. Note that the attacker is only capable of capturing the encrypted communication, given sufficiently strong encryption and key establishment protocol, the decryption of data would be beyond practical means of the attacker. This mitigation is expressed in a semi-formal notation presented in Sect. 13.3.2 as shown in Fig. 13.12. The mitigation, while simple, provides a powerful protection for the system.

The *Command injection* attack can be mitigated by enforcing authentication for any entity issuing commands. This means that the system would reject any command sent from an anonymous source. While this approach provides only limited protection against insider attacks (a potential attacker is among entities that can authenticate), it is an important starting point for securing of the system against malicious attacks. The authentication requires that the entity issuing the command submits a credential that is then evaluated; only a command with correct credentials will be executed. This approach is shown in Fig. 13.13. It is important to note that the credential needs to be protected from a potential attacker, it needs to be stored securely and encrypted during transport, for example by use of a channel encryption. In order to increase the level of protection, the system might consider authorisation to further

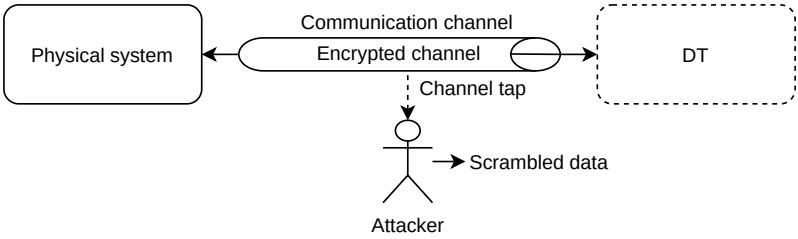


Fig. 13.11: Channel encryption mitigating Man-in-the-middle attack within a DT-enabled system

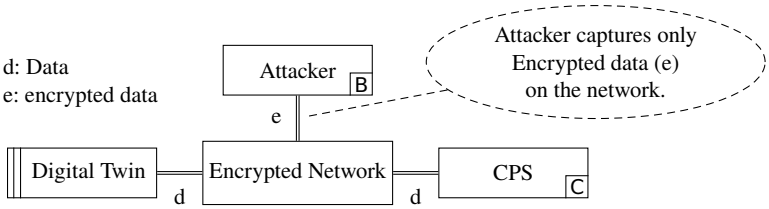


Fig. 13.12: Context diagram for man-in-the-middle attack mitigation.

limit which authenticated entities can issue what types of commands. The choice of the appropriate approach is dependent on the system complexity and the ability of the system operator to manage the authentication (i.e., keep a database of communication entities) or potential authorisation (by managing ACLs or RBAC).

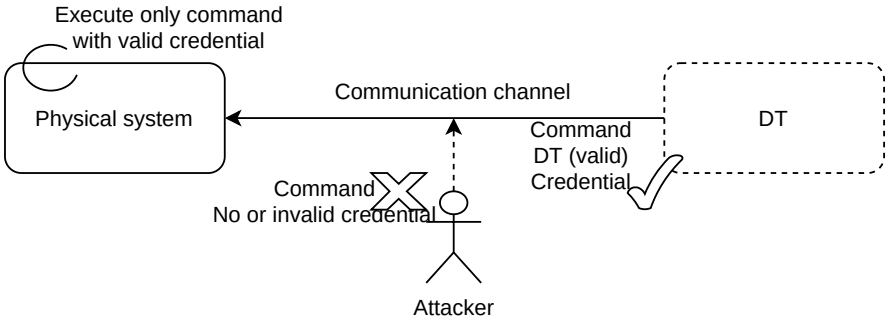


Fig. 13.13: Authentication mitigating the command injection attack within a DT-enabled system

The *Denial of Service* attack often provides a challenge for successful mitigation. One way of mitigating DoS attacks is by using request gateways [20] that can filter out traffic identified as potential DoS attack. This requires extra hardware and such

a mitigation can introduce latency, limiting its viability for some systems. This approach is shown in Fig. 13.14. The PT itself and/or the DT shall have mitigation measures implemented such as refusing traffic from a specific source if the number of data packets exceeds a specific threshold. This approach prevents the computing entities from attempting to process all of the data and become overloaded. The filtering algorithm, however, needs to be implemented directly within the system what might lead to false positives. Potential improvement within a DT-enabled system is utilisation of a DT for traffic monitoring, where the DT could issue a command to the PT to drop specific traffic if a DoS attack is suspected. As in other aspects of security, the chosen approach needs to take into account the goal of the DT-enabled system.

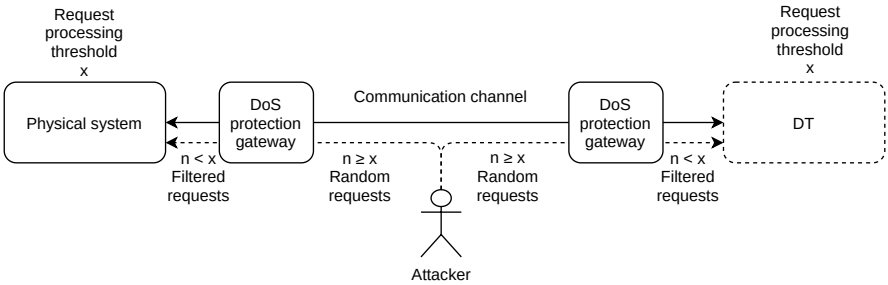


Fig. 13.14: Denial of Service mitigation using a network gateway

The *Data modification* attack can be mitigated using cryptographic means such as using signatures created from data packets computed from the identity of the system components and the data value itself. This permits the creation of a signature that cannot be simply recreated by the attacker, and any change within the data would not correspond to the signature. The mechanism should be understood as computing a hash of the value and then signing it by the private key of the entity providing this data. This approach is shown in Fig. 13.15. The entity receiving the messages needs to trust the public keys of the entity providing the messages. It is important that the legitimate private keys are protected from possible attackers as if an attacker gains an access to a private key, it is possible to modify the data, re-sign and it and the data would be accepted by the system as genuine. The protection of private keys is a wide topic beyond the scope of this book. However, approaches such as Trusted Platform Modules or software-based key storage systems should be utilised.

The *Model malware* attack can be mitigated by model integrity checks and the setup of authorisation for model contributions. This is especially important when third party libraries are used. These should be scanned for vulnerabilities and reviewed before being integrated into the DT model. The necessary depth of reviews and scanning depends on the criticality of the system. The approach is shown in Fig. 13.16. All the changes to the model should also be traceable to individual de-

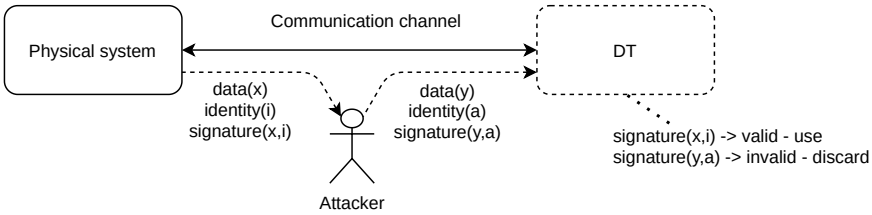


Fig. 13.15: Mitigation of data modification attack

velopers and any updates to the third party libraries should trigger the scanning and review process. This mitigation can be extended to protect against insider attacks by attaching multiple reviewers and automated scanning to the process. Furthermore, formal methods can be utilised at runtime to continuously check the system security properties are not being violated.

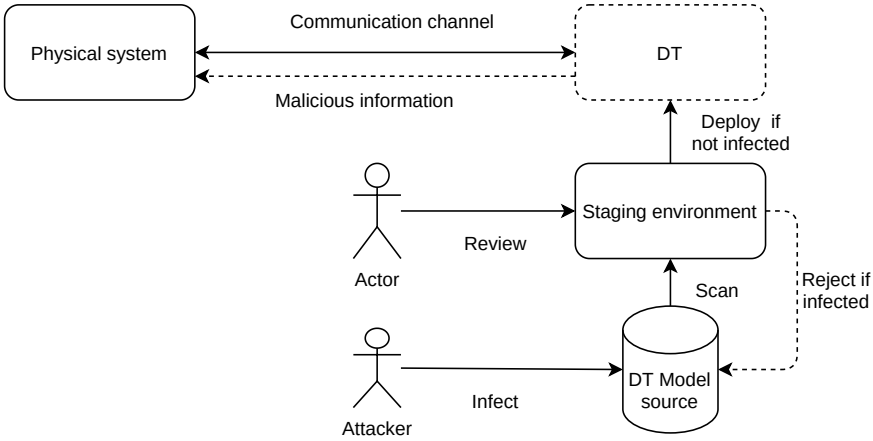


Fig. 13.16: Mitigation of model malware attack via scanning and review process

The approach of using a formal model in the loop to provide continuous assurance that specific security properties hold has been applied in industrial control systems [52] where a formalism known as Vienna Development Method (VDM) [13] has been used (see Sect. 5.6). This approach could be carried out in two ways. In the first option, the formal model understands the networking protocol between the system components and ensures that all the communication conforms to the expected protocol. In the second option, the model includes internal workings of the system and as such can monitor properties of the actual system component functionality rather than just the networking protocol. Both approaches can be combined to provide coverage of security properties affecting the network as well as the functionality.

This approach can be considered as creation of a **formal security DT** for the system, hence extending the system with an extra DT service specifically aimed at monitoring the security properties and providing formal guarantees for secure execution. The approach utilised within the industrial control systems uses a formal model m that expresses protocol and execution scenarios s_i where each scenario is bound to a security contract expressed as pre-condition *pre*, post-condition *post* and global invariant *inv*. This binding then creates a security property p_i.

During the system execution the security critical scenarios, matching the scenarios formally expressed within the model are evaluated as:

$$holds(p_i) = (m.s_i.pre \wedge m.s_i.post \wedge m.inv)$$

while $holds(p_i)$ is satisfied the system continues its execution as normal, however if the property p_i is violated the operator is informed and potential automated mitigation measures could be taken, such as disconnecting an affected component or switching to a fully isolated mode. This approach, while valuable, presents an additional learning curve for system designers and as such is more suited to security-critical functionality.

13.3.4 Attack Detection in DTs

Cyber attack detection is the process of identifying and recognising malicious activities or unauthorised actions in the CPS and the DT. The goal of the cyber attack detection mechanisms is to discover and respond to security breaches promptly in different layers of the CPS including the DT, communication channels, sensors, actuators, etc., and identify malicious activities before they can compromise the integrity of a DT and cause significant damage. Attack detection is a critical component of a robust DT cybersecurity strategy in addition to prevention and mitigation measures like firewalls, encryption and access controls that are discussed further in Sect. 13.3.3. Rapid identification and response to cyber attacks has the following advantages:

Early intervention: Fast detection and swift response to a cyber attack significantly reduces the potential impact on safety and performance [14]. It also prevents attackers from escalating their actions to cause severe impact.

Reduced damage: Identification and neutralisation of cyber attacks reduces the extent of damage such as service interruptions, financial losses, safety risks, etc [43].

Protecting critical infrastructure: In industry sectors such as transportation, energy, agriculture, or health care, cyber attacks have a great impact on public safety and national security [51] and thus, detecting cyber attacks is very important to protect these vital sectors.

Preserving and enhancing the DT integrity: Attack detectors reveal the security weaknesses of the DT and this knowledge can be used to cover security gaps and to strengthen DTs' security posture and integrity [19].

Protection strategies such as those discussed above can be devised to reduce the probability of experiencing successful attacks. However, if such attacks are successful, they can have expensive or catastrophic consequences for the PT. Given the risks, effective detectors should be devised and embedded within the DT to identify security breaches in the DT infrastructure.

13.3.4.1 Design considerations for DT attack detectors

The design consideration of a DT attack detector is briefly discussed in the following.

High accuracy: The accuracy of the DT attack detectors should be high to minimise false positive alarms which may endanger DTs security [19]. This is important to avoid unnecessary disruption and downtime, and reduce the burden on cybersecurity teams. False negatives should also be minimised to ensure the reliability of the detector and DT as a whole. The detector should also be robust against disturbances and noise that could negatively affect the accuracy [46].

Fast detection: The detection speed should be high so that cyber attacks can be detected soon after they are launched, so as to minimise the disruption and time needed to respond and limit damage.

Low complexity: The attack detector mechanism should not be complex so as to maintain its responsiveness and speed and also to avoid imposing additional burdens on DT computations [25].

Low impact on nominal performance: The attack detector should be designed to have a minimal impact on the performance of the DT and ensure smooth operations [46]. As will be discussed in Sect. 13.3.4.3, some detection methods against replay attacks such as Linear–Quadratic–Gaussian (LQG)-based techniques [41] are based on slight perturbation or violation of nominal system performance, which will cause non-optimal operation.

Regular updates and calibration: The nature of a DT requires it to be continuously updated based on the conditions of the PT, which creates possibilities for emerging threats [30]. Detectors should be regularly updated and/or calibrated (either manually or automatically) to ensure they remain effective over time.

Configurability: The attack detector should be configurable to adapt to the unique requirements of the DTs in different applications, systems and processes [25].

Scalability: The attack detector should be scalable to handle large volumes of data and traffic in dynamic DT environments [25].

It should be noted that the exact requirements and design considerations of the DT attack detectors depend on the type of DT Sect. 3.6, the application area and the corresponding security requirements. Attack detectors should be strategically

placed at key points within the DT to ensure comprehensive coverage. Examples may include the DT application layer (API and software interfaces, etc.), communication channels (e.g., for detecting man-in-the-middle attacks), databases (e.g., to identify unauthorised access to data), middleware, etc.

13.3.4.2 Motivational Examples Showing Significance of Attack Detection

Cyber attacks, if they remain undetected, can lead to catastrophic failures in DTs. Several past cyber attack incidents have been reported in the literature emphasising the critical need for effective detection mechanisms. For example, on August 2003, a cyber attack was launched against power system controllers that led to an electric power outage in the Midwest United States and Ontario, Canada, resulting in an estimated economic loss of about $6 billion [28]. A cyber attack was also launched against electricity distribution companies in Ukraine in December 2015, which impacted almost 225,000 customers [7]. A large-scale attack on SCADA systems was also reported in 2010 which directly exploited targeted control loops of centrifuges in uranium-enrichment plants in Iran [34]. The plant was first compromised to record its information silently and use it to manipulate sensing and actuating data, destabilise and destroy the controlled processes. During the attack, the supervisory controllers were provided with false data, to mislead plant operators into thinking that processes are running in normal conditions. This type of attack is termed a *replay cyber attack*.

These examples show that designing effective attack detectors is critical to ensuring DT integrity, availability and confidentiality. It is therefore appropriate to consider attack detection mechanisms in DTs from a control engineering perspective.

13.3.4.3 Attack Detection Strategies

Different attack types and scenarios have been reviewed in Sect. 13.2. The variety of attack types and scenarios means that special and dedicated attack detectors may need to be used for the detection of different attack types against DTs. For example, a detector for the identification of a False Data Injection (FDI) attack may not be effective for the detection of a replay attack. This subsection studies different attack detection methods but multi-attack detection (i.e., detecting multiple attacks with one detector) is beyond the scope. Likewise, we clarify that the methods reviewed for detecting attacks may be different from tools for the identification of faults which are unintentional incidents and rather deal with technical issues, software bugs, hardware failures, or misconfigurations.

In the following we consider attack detector design for three types of cyber attack applicable to DT-enabled systems introduced in Sect. 13.2, including command injection, DoS and data modification. To this end, we first need to formulate the

attack models and their impact on the system parts including DT and PT. Fig. 13.17 shows the positioning of the DT, the PT and the attacker. The PT is formed of the physical plant, related sensors and actuators. In Fig. 13.17, u_p and y_p are the input and output of the PT, respectively. Likewise, u is the command signal calculated by the controller and transmitted to the PT and y is the PT output received by the DT-enabled system. It should be noted that the input signal received by the PT (u_p) is equal to the command signal calculated by the controller in the DT (u) under normal conditions without any cyber attacks manipulating the command signal. An attack detector is placed in the DT-enabled system and can read the command signal (u) and signal received from the PT output (y). Signal transmission between the PT and the DT parts are accomplished through communication channels. Three main types of attacks against communication channels illustrated in Fig. 13.3 to 13.5 can be represented as switches that can disconnect the data transmission in case of DoS attacks, attack signals a_u injected to command signal u in case of command injection and attack signals (a_y) injected to the data transferred from the PT (y_p) in case of data modification. The command injection and data modification attacks are considered as two different types of attacks in the FDI attack category. In the following we discuss the design process of FDI attack detectors in the general case.

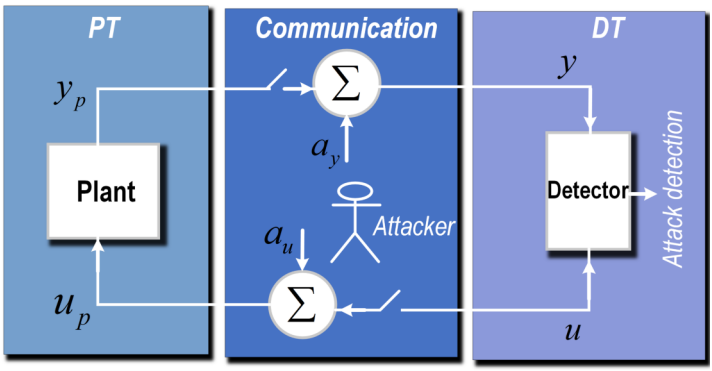


Fig. 13.17: Block diagram of a DT-enabled system in the presence of cyber attacks.

According to Fig. 13.17, the physical plant in the PT can be modelled using a state space representation to describe the system's dynamic behaviour by defining its state variables, inputs, outputs and the relationships among them. Considering the system matrix $\mathbf{F} \in \mathbb{R}^{n\times n}$, the input matrix $\mathbf{B} \in \mathbb{R}^{n\times p}$, the output matrix $\mathbf{C} \in \mathbb{R}^{m\times n}$ and the attack matrices $\mathbf{H} \in \mathbb{R}^{n\times q}$ and $\mathbf{D} \in \mathbb{R}^{m\times q}$, the state space representation of the physical plant can be written as follows:

$$\begin{aligned} \mathbf{x}(k+1) &= \mathbf{F}\mathbf{x}(k) + \mathbf{B}\mathbf{u}(k) + \mathbf{H}\mathbf{a}(k) + \mathbf{w}(k) \\ \mathbf{y}(k) &= \mathbf{C}\mathbf{x}(k) + \mathbf{D}\mathbf{a}(k) + \mathbf{v}(k) \end{aligned} \tag{13.1}$$

where $\mathbf{x}(k) \in \mathbb{R}^n$ is the state vector. The plant is assumed to be subjected to process and measurement noises $\mathbf{w}(k) \in \mathbb{R}^n$ and $\mathbf{v}(k) \in \mathbb{R}^m$, respectively. Vector $\mathbf{a}(k) \in \mathbb{R}^q$ also models a specific cyber attack and can be composed of a_u and a_y (see Fig. 13.17) as $\mathbf{a}(k) = \begin{bmatrix} a_u \\ a_y \end{bmatrix}$.

FDI Attack Detection

The FDI attack against physical plant can be modelled considering $\mathbf{a}(k) \neq 0$ in Eq. (13.1). Three different FDI cases can be considered [47]:

1. command injection ($a_u \neq 0, a_y = 0$), in which the attack injects false data into the command signal u;
2. data modification ($a_y \neq 0, a_u = 0$), in which false data is injected into the PT output vector y_p;
3. coordinated FDI ($a_y \neq 0, a_u \neq 0$) in which both u and y_p are subjected to the attack.

Such attacks can influence the process states, control and measurements of the sensors.

The FDI attack detection can be done by determining the value of the attack vector, i.e., to decide whether $\mathbf{a}(k) \neq 0$ or $\mathbf{a}(k) = 0$. This helps operators to determine the attack type and affected communication links and to subsequently activate an alarm and possibly a contingency plan. Some methods have been proposed to estimate the attack vector entries $a(0), a(1), \ldots, a(k)$, which will precisely reveal the nodes or data that were impacted by the FDI attack [47]. A recursive generalised likelihood-ratio test [55] can also be applied on real-time transmission data from physical plant to/from the controller to detect the onset of the cyber attack. Another approach based on the so-called Cumulative Sum (CUSUM) test has been discussed in [5]. The CUSUM test is categorised into residue-based attack detection approaches which are based on the calculation of a residual vector. The residual vector (denoted as $\mathbf{r}(k)$) is the difference between the PT output signal received by the DT ($\mathbf{y}$ in Fig. 13.17) and its estimated value using a dynamic state estimation mechanism such as a Kalman filter. The Kalman filter utilises the same system, input and output matrices of $\mathbf{F}$, $\mathbf{B}$ and $\mathbf{C}$, respectively, as the physical plant model (defined in Eq. (13.1)) and estimates the system state vector ($\mathbf{x}(k)$ in Eq. (13.1)) as follows:

$$\begin{aligned} \hat{\mathbf{x}}(k+1) &= \mathbf{F}\hat{\mathbf{x}}(k) + \mathbf{B}\mathbf{u}(k) + \mathbf{K}\mathbf{r}(k) \\ \mathbf{r}(k) &= \mathbf{y}(k) - \mathbf{C}\hat{\mathbf{x}}(k) \end{aligned} \tag{13.2}$$

where $\mathbf{K}$ is the Kalman gain that can be tuned with the data from non-attack conditions and $\hat{\mathbf{x}}(k) \in \mathbb{R}^n$ is the estimated state vector. In the presence of the attack vector ($\mathbf{a}(k) \neq 0$), the estimated values by the dynamic state estimator Eq. (13.2) deviates from the actual values and therefore the value of the residual vector $\mathbf{r}(k)$ increases. In each step, the residual vector $\mathbf{r}(k)$ is calculated using Eq. (13.2).

The attack detection is fulfilled based on determining a proper detection criterion. For instance, in case of the χ^2 detector [16], the following detection criterion is defined:

$$\begin{cases} |\mathbf{r}(k)|^2 < \tau & \text{no alarm} \\ |\mathbf{r}(k)|^2 \geq \tau & \text{alarm} \end{cases} \tag{13.3}$$

Despite their simplicity, the residual-based detection tests may be challenged to detect some attack types such as undetectable and stealthy attacks. For instance, for the χ^2 detector, deterministic stealthy attacks can be designed by attackers to bypass bad data detectors satisfying the following constraint for $k \geq 0$ [47]:

$$|\mathbf{r}(k; y, x, u, a)|^2 \leq \tau + \sigma \tag{13.4}$$

where $\mathbf{r}(k; y, x, u, a)$ is the residual signal under the attack condition. By choosing $\sigma = 0$, the attack will not generate an alarm signal for the χ^2 detector.

The foregoing problem has been addressed in several studies in the literature involving techniques based on Kullback–Leibler divergence [9], interleaved hop-by-hop authentication [57] and generalised likelihood ratio [39] and most recently methods based on state observers and estimators.

Replay attack detection

In replay attacks, delayed versions of the real sensor measurements (y_p) are transmitted to the controllers in the DT, and at the same time, the command signal ($\mathbf{u}$) is manipulated to degrade the DT-enabled system's performance. An example of a replay attack has been discussed in [41] which collects online sensory measurements between the sensors and the controllers, and changes the correct measurements to the values that were previously recorded. Assuming the replay attack starts at time $k = k_0$ for T steps, one can write:

$$y_a(k) = \mathbf{y}(k - T), \quad 0 \leq k \leq T - 1, \tag{13.5}$$

After this step, the cyber attack tries to change the control actions or control commands transmitted from the controller by injecting an external signal $u_a(k)$ for $0 \leq k \leq T - 1$. Replay attacks are most dangerous when the system is at steady-state conditions when the measurement values correspond to well-regulated system states [47].

It is difficult to detect replay attacks with statistical test methods that are based on a sequence of measurements. To improve the detectability of such attacks, a basic technique is to change the controller output purposefully and randomly to make the system deviate slightly from the steady-state conditions. Such variations are not reflected in the injected or fake measurements applied by a replay attack which will reveal the occurrence of the attack.

As a typical example, one can assume a system described by Eq. (13.1) with $\mathbf{a}(k) \neq 0$. Assume the original controller is based on LQG control objective minimising the quadratic cost and the optimised control command denoted as $\mathbf{u}^*$ is obtained. The

replay attacker collects the system responses to the control command $\mathbf{u}^*$. Once the attacker injects false data to the system, it changes the actual responses to the recorded values to remain undetectable from residual-based detectors such as ones discussed in Eqs. (13.2) to (13.4). To assure replay attack detectability, the control signal can be redesigned as follows:

$$\mathbf{u}(k) = \mathbf{u}^*(k) + \Delta\mathbf{u}(k). \tag{13.6}$$

where $\Delta\mathbf{u}$ is considered as an authentication signal independent of $\mathbf{u}^*$ and is derived from an i.i.d. Gaussian distribution with zero mean. The system response to the redesigned control signal Eq. (13.6) deviates from the values recorded and injected into the system by the attacker and thus the attack will be detected. While this approach improves detectability, the control performance will need to be sacrificed to some extent since the control signals are no longer fully optimised in the LQG sense [41].

DoS Attack Detection

DoS attacks usually have severe impacts on the operation of the system since they will cause temporary or continuous interruptions in the feedback control loops which may destabilise a complete plant, system, or process [47]. In the control sense, DoSs can be modelled by introducing switches as shown in Fig. 13.17. Whenever the DT-enabled system is under attack conditions, the corresponding switch will transit to the open state. The attack can be represented as:

$$u_p(k) = \gamma(k)\mathbf{u}(k), \quad \mathbf{y}(k) = \eta(k)y_p(k) \tag{13.7}$$

where u_p is the command signal applied to the PT and y_p is the PT output. The binary variables $\gamma(k)$ and $\eta(k)$ represent DoS attacks, such that $\gamma(k) = 0$ if the control command communication is interrupted by an attack and $\gamma(k) = 1$ otherwise, and $\eta(k)$ is defined in a similar way. Detection methods for DoSs are diverse and the choice of method depends on several parameters such as anomaly type, data types and sensitivity, complexity constraints and the required safety levels that must be assured [45]. The existing techniques can be classified into Signature-based, Anomaly-based and Hybrid detection as described in [2]. Likewise, Artificial Intelligence-based methods have become popular in the detection of DoS attacks [48]. A full review of DoS attack detection methods can be found in [26].

13.4 Intellectual Property Protection

Protection of IP is an important aspect of any commercial system. In the DT context the IP regarding the inner workings of the system might be inferred from a DT without the need to reverse engineer the physical system itself. This leads to the need to protect the DT model. It is of course important to manage access to the repositories where the model source is being stored, but precautions also need to be taken when the model is deployed within an active DT. Technical means such as obfuscation and encryption at rest can be applied at relatively low cost. Access to the system shall be monitored for all users, and a clear licence needs to be issued for use of the model. An interesting aspect of a DT-enabled system is that a system could be represented by a multitude of DTs that need to be interconnected.

Let us consider a DT of an engine, where the mechanical parts have a DT model representing them, fuel injection control has its own DT model and the oil filtration subsystem has its own DT model, see Fig. 13.18. These different models would often be provided by different enterprises that are sub-suppliers of the engine builder. While the engine builder would be able to access the state of the entire engine, deep diagnosis of the injection profile could for example only be visible to the supplier of the injection subsystem. This means that clear isolation of data access and model access needs to be available within the system. Similarly, the oil filtration device manufacturer might be interested in the performance of the oil filtration. However they should not learn about inner workings of the fuel injection subsystem. This is partially achievable by use of authorisation frameworks, where a set of authorisation rules guides specific user access only to the parts that should be available to those users. To this end, the DT can implement ACLs, where engine builder users would get overall engine read privileges that would include the sub-supplier subsystems on a high level. The fuel injection sub-system manufacturer would then have a diagnostic (detailed) read privilege within the fuel injection DT providing data that are usable to infer the inner workings of the subsystem.

The platform on which the DT is composed also needs to prevent IP disclosure between different parties. Specifically, the hardware, operating system and a potential hypervisor need to provide functionality for component isolation. On the operating system level this can be achieved by process isolation techniques, where different processes executed on top of the operating system forbid access to each other resources, most often the memory allocated by the process. The techniques could further be enforced by using cryptographic hardware such as trusted platform modules or in specific cases (if supported by CPU) per thread memory encryption. Recently, isolation techniques became available even for processes requiring fast domain switching, such as creation of network facing cryptographic sessions [54], though research within this area is rapidly advancing. In cases where fast switching is not necessary, a hypervisor (a virtual machine monitor and executor) can provide its own isolation, which is often easier to achieve as it can allocate a specific memory area for the virtualised context it is executing and hence can encrypt this entire

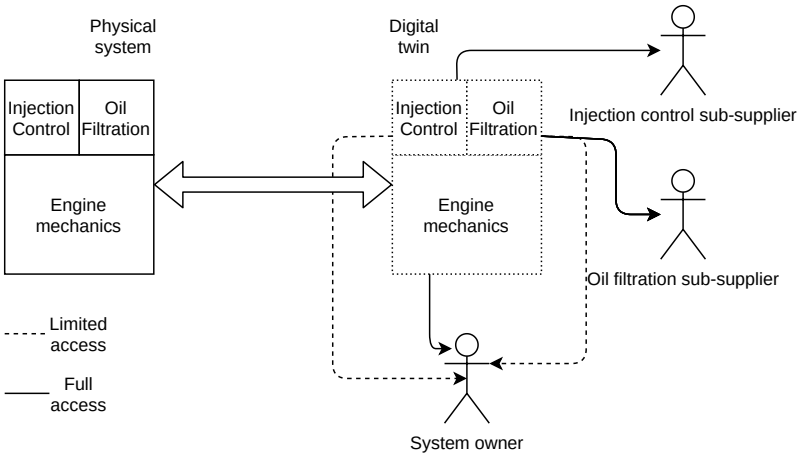


Fig. 13.18: Composite system with corresponding DT and multiple users

memory space under a specific key (rather than dynamically creating keys for every resource of every thread within the memory). Similar measures are often utilised by hypervisors for data-at-rest protection as well as network components especially within multi-tenancy environments such as cloud based platforms [33]. If there is a need to be stricter than software-based isolation within cloud platforms, it is often possible to utilise single tenant hosting, where the hosting hardware is assigned to a single tenant, what however often comes at a higher financial cost.

The primary driver for the decision on where and how to deploy a composite DT is the consideration of trust between the different entities as well as mission criticality of the DT.

13.5 Security in the Real World

An important aspect of any secure system is the use of correct security approaches to meet the security goal for the system. It is never utilisation of just one technique or approach; different security techniques need to work together in order to provide the overall security. Let us consider the composite DT once more. Systems of this type would need to consider secure network, secure user access, isolation of components and correct execution. This means that approaches such as network and data encryption, use of authentication and authorisation for users, isolation techniques such as use of hypervisors, security analysis by use of formal methods, and detection and mitigation mechanisms to prevent performance degradation in the presence of attacks would be used at the same time. This leads to more detailed decisions, such as how to encrypt the network traffic, what authorisation model should be used, what

level of isolation is needed, whether the use of formal analysis is warranted within the specific system, and how to deal with complexity of implementation and execution of Kalman filter-based detectors in real world applications where assumptions of system model, process and measurement noises are not met. There is not a single answer as it depends on the system under consideration. For example, restarting a device could be a simple attack mitigation strategy for an IT system, but interrupting an autonomous vehicle or an infrastructure system could be impossible in most practical situations. However, it is important that the system designer poses these questions and considers security as a guiding principle within the system design.

This chapter has presented some of the basic concepts applicable within secure development, but only at an introductory level. The use of encryption, for example, comes with caveats in cases where potential real-time deadlines are to be considered. It is also important that the system designers have a good understanding of primitives of encryption and authorisation in order to make the correct decisions. There are often considerations such as speed of execution, user friendliness, reconfiguration flexibility and many more that can clash with the security design. It is therefore important to focus on the risk assessment and decide on the baseline for the security of the system. The baseline dictates the minimum amount of security mechanisms for the system to be considered production secure. In case of a DT-enabled system, specific consideration needs to be given to the DT itself and its communication with the physical system, however tried and trusted approaches to secure control and computing as presented here could be utilised in the DT paradigm.

It must be said that only a system that does nothing can be considered truly secure. In the real world it is important to have systems secured to meet their designated goals. A final note for a secure system is the need not only to design and deploy the system securely, but also continuously review the threat landscape and update the system with new mitigations against vulnerabilities that were not known at a design time. As such the risk assessment and threat analysis are activities that shall be maintained during the lifetime of the system.

References

1. Acar, A., Aksu, H., Uluagac, A.S., Conti, M.: A Survey on Homomorphic Encryption Schemes: Theory and Implementation. ACM Comput. Surv. **51**(4) (2018). DOI: 10.1145/3214303
2. Agarwal, B., Mittal, N.: Hybrid approach for detection of anomaly network traffic using data mining techniques. Procedia Technology **6**, 996–1003 (2012)
3. Alcaraz, C., Lopez, J.: Digital twin: A comprehensive survey of security threats. IEEE Communications Surveys & Tutorials **24**(3), 1475–1503 (2022). DOI: 10.1109/COMST.2022.3171465
4. Alramadhan, M., Sha, K.: An overview of access control mechanisms for Internet of Things. In: 2017 26th International Conference on Computer Communication and Networks (ICCCN), pp. 1–6 (2017). DOI: 10.1109/ICCCN.2017.8038503

5. Basseville, M., Nikiforov, I.V., et al.: Detection of abrupt changes: theory and application, vol. 104. prentice Hall Englewood Cliffs (1993)
6. Bihis, C.: Mastering OAuth 2.0. Packt Publishing Ltd (2015)
7. Case, D.U.: Analysis of the cyber attack on the Ukrainian power grid. Electricity Information Sharing and Analysis Center (E-ISAC) **388**(1-29), 3 (2016)
8. Casola, V., De Benedictis, A., Rak, M., Villano, U.: A novel Security-by-Design methodology: Modeling and assessing security by SLAs with a quantitative approach. Journal of Systems and Software **163**, 110537 (2020). DOI: https://doi.org/10.1016/j.jss.2020.110537
9. Chaojun, G., Jirutitijaroen, P., Motani, M.: Detecting false data injection attacks in AC state estimation. IEEE Transactions on Smart Grid **6**(5), 2476–2483 (2015)
10. Clarke, E.M., Klieber, W., Novácek, M., Zuliani, P.: Model checking and the state explosion problem. In: B. Meyer, M. Nordio (eds.) Tools for Practical Software Verification, LASER, International Summer School 2011, Elba Island, Italy, Revised Tutorial Lectures, *Lecture Notes in Computer Science*, vol. 7682, pp. 1–30. Springer (2011)
11. De Maeyer, C., Markopoulos, P.: Future outlook on the materialisation, expectations and implementation of digital twins in healthcare. In: 34th British HCI Conference 34, pp. 180–191 (2021)
12. Feng, H., Chen, D., Lv, H., Lv, Z.: Game theory in network security for digital twins in industry. Digital Communications and Networks (2023). DOI: https://doi.org/10.1016/j.dcan.2023.01.004
13. Fitzgerald, J.S., Larsen, P.G., Verhoef, M.: Vienna Development Method. Wiley Encyclopedia of Computer Science and Engineering (2008). Edited by Benjamin Wah, John Wiley & Sons, Inc.
14. Gandi, V.P., Jatla, N.S.L., Sadhineni, G., Geddamuri, S., Chaitanya, G.K., Velmurugan, A.K.: A comparative study of AI algorithms for anomaly-based intrusion detection. In: 2023 7th International Conference on Computing Methodologies and Communication (ICCMC), pp. 530–534 (2023). DOI: 10.1109/ICCMC56507.2023.10084186
15. Gehrmann, C., Gunnarsson, M.: A digital twin based industrial automation and control system security architecture. IEEE Transactions on Industrial Informatics **16**(1), 669–680 (2020). DOI: 10.1109/TII.2019.2938885
16. Giraldo, J., Urbina, D., Cardenas, A., Valente, J., Faisal, M., Ruths, J., Tippenhauer, N.O., Sandberg, H., Candell, R.: A survey of physics-based attack detection in cyber-physical systems. ACM Computing Surveys (CSUR) **51**(4), 1–36 (2018)
17. Guggenmos, F., Häckel, B., Ollig, P., Stahl, B.: Security First, Security by Design, or Security Pragmatism – Strategic Roles of IT Security in Digitalization Projects. Computers & Security **118**, 102747 (2022). DOI: https://doi.org/10.1016/j.cose.2022.102747
18. Hensel, C., Junges, S., Katoen, J.P., Quatmann, T., Volk, M.: The probabilistic model checker Storm. International Journal on Software Tools for Technology Transfer **24**(4), 589–610 (2022). DOI: 10.1007/s10009-021-00633-z
19. Holmes, D., Papathanasaki, M., Maglaras, L., Ferrag, M.A., Nepal, S., Janicke, H.: Digital twins and cyber security – solution or challenge? In: 2021 6th South-East Europe Design Automation, Computer Engineering, Computer Networks and Social Media Conference (SEEDA-CECNSM), pp. 1–8 (2021). DOI: 10.1109/SEEDA-CECNSM53056.2021.9566277
20. Huang, X., Du, X., Song, B.: An effective DDoS defense scheme for SDN. In: 2017 IEEE International Conference on Communications (ICC), pp. 1–6 (2017). DOI: 10.1109/ICC.2017.7997187
21. Iqbal, Z., Mehmood, A., Ghafoor, A., Ahmed, H.F., Shibli, A.: Authenticated service interaction protocol for multi-agent system. In: 2007 International Symposium on High Capacity Optical Networks and Enabling Technologies, pp. 1–5 (2007). DOI: 10.1109/HONET.2007.4600276
22. Jackson, M.: Software Requirements and Specification: a lexicon of practice, principles and prejudices. Addison-Wesley (1995)

23. Jackson, M.: Problem Frames: Analysing and Structuring Software Development Problems. Addison-Wesley, New York (2001)
24. Jeske, S.J.: Digital twins in healthcare. Paris Lodron University of Salzburg and Aalborg University in Copenhagen (2020)
25. Kandasamy, N.K., Venugopalan, S., Wong, T.K., Leu, N.J.: An electric power digital twin for cyber security testing, research and education. Computers and Electrical Engineering **101**, 108061 (2022). DOI: https://doi.org/10.1016/j.compeleceng.2022.108061
26. Kaur, P., Kumar, M., Bhandari, A.: A review of detection approaches for distributed denial of service attacks. Systems Science & Control Engineering **5**(1), 301–320 (2017)
27. Kayan, H., Nunes, M., Rana, O., Burnap, P., Perera, C.: Cybersecurity of industrial cyber-physical systems: A review. ACM Comput. Surv. **54**(11s) (2022). DOI: 10.1145/3510410. URL https://doi.org/10.1145/3510410
28. Kazemi, Z., Safavi, A.A., Setoodeh, P.: Efficient resilient dynamic co-estimation framework for cyber-physical systems under sensor attacks. IET Control Theory & Applications **14**(20), 3526–3536 (2020)
29. Khan, R., McLaughlin, K., Laverty, D., Sezer, S.: Stride-based threat modeling for cyber-physical systems. In: 2017 IEEE PES Innovative Smart Grid Technologies Conference Europe (ISGT-Europe), pp. 1–6 (2017). DOI: 10.1109/ISGTEurope.2017.8260283
30. Klein, M., Carpentier, T., Jeanclaude, E., Kassab, R., Varelas, K., de Bruijn, N., Barbaresco, F., Briheche, Y., Semet, Y., Aligne, F.: AI-augmented multi function radar engineering with digital twin: Towards proactivity. In: 2020 IEEE Radar Conference (RadarConf20), pp. 1–6 (2020). DOI: 10.1109/RadarConf2043947.2020.9266455
31. Kulik, T., Dongol, B., Larsen, P.G., Macedo, H.D., Schneider, S., Tran-Jørgensen, P.W.V., Woodcock, J.: A Survey of Practical Formal Methods for Security. Formal Aspects of Computing (2022). DOI: 10.1145/3522582
32. Kulik, T., Gomes, C., Macedo, H.D., Hallerstede, S., Larsen, P.G.: Towards secure digital twins. In: T. Margaria, B. Steffen (eds.) ISoLA 2022, *Lecture Notes in Computer Science*, vol. 13704. Springer (2022). DOI: 10.1007/978-3-031-19762-8_11
33. Kumar, V., Rathore, R.S.: Security issues with virtualization in cloud computing. In: 2018 International Conference on Advances in Computing, Communication Control and Networking (ICACCCN), pp. 487–491 (2018). DOI: 10.1109/ICACCCN.2018.8748405
34. Kushner, D.: The real story of Stuxnet. IEEE Spectrum **50**(3), 48–53 (2013)
35. Lamport, L.: Specifying systems: the TLA+ language and tools for hardware and software engineers. Addison-Wesley Longman Publishing Co., Inc. (2002)
36. Lamport, L.: The PlusCal algorithm language. In: International Colloquium on Theoretical Aspects of Computing, pp. 36–60. Springer (2009)
37. Langner, R.: Stuxnet: Dissecting a cyberwarfare weapon. IEEE Security & Privacy **9**(3), 49–51 (2011). DOI: 10.1109/MSP.2011.67
38. Leines-Vite, L., Arriaga, J.C.P., Limón, X.: Confidentiality and Integrity Mechanisms for Microservices Communication. CoRR (2021). URL https://arxiv.org/abs/2111.01218
39. Li, S., Yılmaz, Y., Wang, X.: Quickest detection of false data injection attack in wide-area smart grids. IEEE Transactions on Smart Grid **6**(6), 2725–2735 (2014)
40. Löcklin, A., Müller, M., Jung, T., Jazdi, N., White, D., Weyrich, M.: Digital twin for verification and validation of industrial automation systems – a survey. In: 2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA), vol. 1, pp. 851–858 (2020). DOI: 10.1109/ETFA46521.2020.9212051
41. Mo, Y., Sinopoli, B.: Secure control against replay attacks. In: 2009 47th annual Allerton conference on communication, control, and computing (Allerton), pp. 911–918. IEEE (2009)
42. Naseri, F., Kazemi, Z., Larsen, P.G., Arefi, M.M., Schaltz, E.: Cyber-physical cloud battery management systems: Review of security aspects. Batteries **9**(7) (2023). DOI: 10.3390/batteries9070382

43. Negrete-Pincetic, M., Yoshida, F., Gross, G.: Towards quantifying the impacts of cyber attacks in the competitive electricity market environment. In: 2009 IEEE Bucharest PowerTech, pp. 1–8 (2009). DOI: 10.1109/PTC.2009.5282237
44. Pfretzschner, B., Othmane, L.b.: Dependency-based attacks on node.js. In: 2016 IEEE Cybersecurity Development (SecDev), pp. 66–66 (2016). DOI: 10.1109/SecDev.2016.023
45. Raut, A.S., Singh, K.R.: Anomaly based intrusion detection-a review. International Journal on Network Security **5**(3), 7 (2014)
46. Saad, A., Faddel, S., Youssef, T., Mohammed, O.A.: On the implementation of IoT-based digital twin for networked microgrids resiliency against cyber attacks. IEEE Transactions on Smart Grid **11**(6), 5138–5150 (2020). DOI: 10.1109/TSG.2020.3000958
47. Sandberg, H., Gupta, V., Johansson, K.H.: Secure networked control systems. Annual Review of Control, Robotics, and Autonomous Systems **5**(1), 445–464 (2022). DOI: 10.1146/annurev-control-072921-075953
48. Singh, N., Hans, A., Kumar, K., Birdi, M.P.S.: Comprehensive study of various techniques for detecting DDoS attacks in cloud environment. International Journal of Grid and Distributed Computing **8**(3), 119–126 (2015)
49. Sridhar, S., Manimaran, G.: Data integrity attacks and their impacts on SCADA control system. In: IEEE PES General Meeting, pp. 1–6 (2010). DOI: 10.1109/PES.2010.5590115
50. Thakor, V.A., Razzaque, M.A., Khandaker, M.R.A.: Lightweight Cryptography Algorithms for Resource-Constrained IoT Devices: A Review, Comparison and Research Opportunities. IEEE Access **9**, 28177–28193 (2021). DOI: 10.1109/ACCESS.2021.3052867
51. Thakur, K., Ali, M.L., Jiang, N., Qiu, M.: Impact of Cyber-Attacks on Critical Infrastructure. In: 2016 IEEE 2nd International Conference on Big Data Security on Cloud (BigDataSecurity), IEEE International Conference on High Performance and Smart Computing (HPSC), and IEEE International Conference on Intelligent Data and Security (IDS), pp. 183–186 (2016). DOI: 10.1109/BigDataSecurity-HPSC-IDS.2016.22
52. Ubys, L., Vancea, V.N., Kulik, T., Larsen, P.G., Boudjadar, J., Aranha, D.F.: Formal model in-the-loop for secure industrial control networks. In: 18th International Conference on Formal Aspects of Component Software. Oslo (virtual) (2022)
53. Upadhyay, D., Sampalli, S.: Scada (supervisory control and data acquisition) systems: Vulnerability assessment and security recommendations. Computers & Security **89**, 101666 (2020). DOI: https://doi.org/10.1016/j.cose.2019.101666
54. Vahldiek-Oberwagner, A., Elnikety, E., Duarte, N.O., Sammler, M., Druschel, P., Garg, D.: ERIM: Secure, efficient in-process isolation with protection keys (MPK). In: 28th USENIX Security Symposium (USENIX Security 19), pp. 1221–1238 (2019)
55. Willsky, A., Jones, H.: A generalized likelihood ratio approach to the detection and estimation of jumps in linear systems. IEEE Transactions on Automatic control **21**(1), 108–112 (1976)
56. Woodcock, J., Larsen, P.G., Bicarregui, J., Fitzgerald, J.: Formal Methods: Practice and Experience. ACM Computing Surveys **41**(4), 1–36 (2009). DOI: http://doi.acm.org/10.1145/1592434.1592436
57. Zhu, S., Setia, S., Jajodia, S., Ning, P.: An interleaved hop-by-hop authentication scheme for filtering of injected false data in sensor networks. In: IEEE Symposium on Security and Privacy, 2004. Proceedings. 2004, pp. 259–271. IEEE (2004)
58. Zonouz, S., Rrushi, J., McLaughlin, S.: Detecting industrial control malware using automated plc code analytics. IEEE Security & Privacy **12**(6), 40–47 (2014)

Chapter 14
Autonomous Reconfiguration Enabled by Digital Twins

Lukas Esterle, Mirgita Frasheri and Peter Gorm Larsen

Abstract Digital Twins (DTs) integrate models with data gathered from Physical Twin (PT) counterparts, adding value through enablers and services that allow informed decision-making about the reconfiguration of the PT, either autonomously, or by human stakeholders. We consider the potential of DTs for improving the capabilities and performance of autonomous systems through two lenses. First, we outline how different agent architectures can integrate DT models to enhance decision-making and reconfiguration. Second, we consider how DT technology can further support self-* properties within autonomous (multi-)agent systems. We identify open challenges that we argue need to be tackled in order to improve the utilisation of DT technology for autonomously reconfiguring systems at runtime.

14.1 Introduction

The important difference between a DT and a Digital Shadow (DS) as discussed in Sect. 1.2.2 is the DT's ability to support the reconfiguration of the Physical Twin (PT) with a degree of autonomy. The DT thus allows the PT to operate in dynamic environments and respond to at least some changes in the environment. In this chapter, we aim to shed light on requirements and approaches to "closing the loop" by allowing the DT to support the reconfiguration process. This provides a basis for autonomous and self-adapting Cyber-Physical Systems (CPSs) while having the advantage of a DT that is able to marshal a set of models that offer different levels of granularity and performance.

Self-adaptation procedures can be very intricate, rely heavily on domain knowledge and are application-specific (e.g., a self-adaptation procedure for our running example of the tempeh incubator is very different to the self-adaptation loop for a robot manipulator). Incidentally, the people who know best how to reconfigure the CPS are not necessarily the ones who can best implement those automated proce-

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

J. Fitzgerald et al. (eds.), *The Engineering of Digital Twins*,
https://doi.org/10.1007/978-3-031-66719-0_14

dures. Therefore these CPSs have to rely on self-reconfiguration approaches utilising DTs.

In this chapter, we first describe autonomous CPSs, their architectures and the incorporation of DTs in general terms in Sect. 14.2. Thereafter, Sect. 14.3 details the self-* properties that are core to autonomous CPSs before focussing on self-awareness, a capability required by autonomous CPSs to make decisions on their abilities and how they can be reconfigured. In order to implement self-* properties, it is necessary to consider how we describe the goals that direct the decision-making about a reconfiguration, and this is discussed in Sect. 14.4. Afterwards, Sect. 14.5 discusses the capabilities required of autonomous agents to collaborate with each other, even in the absence of human operators. The components required to exploit DT technology to facilitate collaboration are highlighted. Then Sect. 14.6 discusses safety issues stemming from autonomous CPSs and how they might be addressed using DTs. Finally, a roadmap is provided in Sect. 14.7, indicating required research to bring our vision of autonomous DT-enabled systems to fruition.

14.2 Autonomous Systems and DTs

Autonomous CPSs are governed by autonomous software *agents*, able to respond to changing conditions, allowing higher flexibility in their deployment. An agent is generally defined as a computer system that is able to sense and act, through its sensors and actuators, in the environment in which it is placed, as well as carry out actions autonomously [39, 26]. This definition can be expanded to capture intelligent agents, provided that there is a capability for flexible autonomous actions [39]. Flexibility targets three main aspects of behaviour of agents such as: (i) social ability, i.e., being able to interact with others, (ii) reactivity to external stimuli and (iii) pro-activity, or in other words being able to decide autonomously on a course of action. Wooldridge argues that a balance between reactivity and pro-activity is crucial in intelligent agents, ensuring neither blind execution of procedures, nor blind reactivity impeding the agent from actually completing any goals [40]. Similarly for CPSs, sensing mechanisms are required to identify changes in the environment and actuators to perform actions on the environment. Dedicated decision-making mechanisms allow the CPS to determine which action to perform in a given situation [38].

We differentiate between *reactive* and *deliberative* agent architectures for autonomous systems. In the following, both types of agent are explored and expanded by introducing a DT. This means the autonomous CPS has a model of itself available that is constantly updated based on sensory input about the environment and the action it takes, and is able to provide feedback to the DT-enabled system.

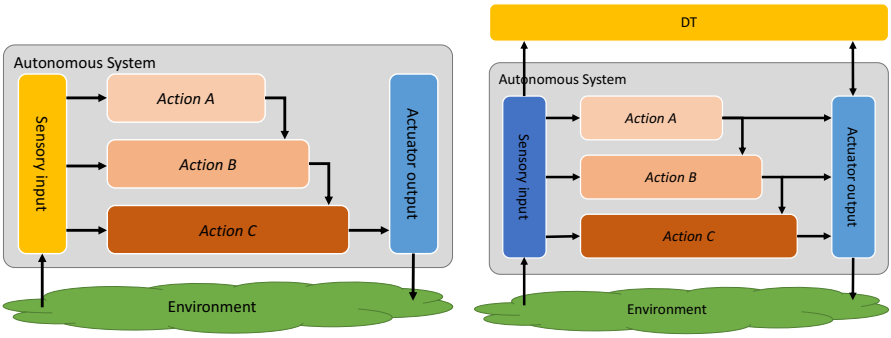
We also differentiate between *loosely* and *tightly* coupled DT-enabled systems. A tightly coupled DT-enabled system requires the DT to operate and utilise all its capabilities. Specifically, for autonomous DT-enabled systems this would mean

that all decisions are being made by the DT itself and imposed on the physical counterpart. In contrast, a *loosely* coupled DT-enabled system can operate without access to the DT and decisions can be made through other local mechanisms. This becomes relevant, when we consider the DT to be external to the system. A *loosely* coupled DT-enabled system will remain operational while a *tightly* coupled DT-enabled system will become unoperational. Loosely coupled DT-enabled systems are able to operate continuously but require synchronisation between the DT and the physical system when a re-connection is established. In the following, we will consider *loosely* coupled DT-enabled systems, replicating their decision-making approaches within the DT and the local DT-enabled system.

Reactive agent architectures operate on direct sensory inputs and immediate reactions to them. In contrast to other types of architectures, reactive agents focus on quick, reflexive responses to the current state of the environment. These architectures are designed to perform specific tasks efficiently without the need for complex planning or long-term memory.

Reactive agent architectures rely on a stimulus-response mapping to select the right actions in each situation. The agent receives sensory inputs from the environment and maps them directly to appropriate actions. This mapping is typically implemented through a set of predefined rules or behaviours which are often organised in a hierarchical or layered structure. During development, the goal is to define rules or behaviours allowing a response for each specific situation or stimulus. At runtime, the agent iterates through the possible actions based on the current sensory input and the environmental response. One of the most prominent examples of a reactive architecture is Brooks's Subsumption Architecture [10]. Fig. 1(a) illustrates a generic version of Brooks's idea. Here, lower-level actions (e.g., Action C) can be overridden by more fine-grained actions in the higher levels (e.g., Action A). With a dedicated DT involved in the reactive architecture illustrated in Fig. 1(b), the DT itself allows actions to be reviewed before generating an output, if the environment and the respective results of the actions are modelled correctly. In practice, without the DT actions would need to be executed for the CPS to understand whether the action has no result and the next action needs to be utilised. However, with the DT involved, the CPS can already exploit this information to determine if another action should be utilised. However, to remain a reactive architecture, the DT is required to model the response of the environment.

One of the key advantages of reactive agent architectures is that, due to their simplicity, they are able to work in their ability to operate in real-time, dynamic and resource-constrained CPS environments. However, since such agent architectures lack internal representations or models of the environment, they cannot reason or plan in a given situation, but only react to immediate stimuli. This means they do not consider the broader context or high-level goals. As a result, these architectures may not be suitable for complex tasks that involve strategic thinking, learning from experience, or maintaining persistent states.



(a) Traditional subsumption architecture (b) DT-enabled Subsumption architecture

Fig. 14.1: Brooks's Subsumption Architecture [10] extended with a DT. Lower level Actions can be can *subsumed* by higher level behaviour.

Deliberative agent architectures, in contrast to reactive agent architectures, focus on reasoning, planning and decision-making based on internal representations or models of the environment. This means they prioritise a more strategic and goal-oriented approach. Generally, deliberative agents consider various possible actions and their impact on long-term objectives before making a decision.

At the heart of deliberative agent architectures is a model of the environment. This model represents the agent's understanding of the environment, including its own current state, the available actions the agent can perform and deliver the expected / potential outcomes. Depending on its granularity, this model may include explicit representations of objects, relationships between itself and other objects and PTs, various rules and available goals. The agent utilises this model to simulate possible future scenarios, evaluate different courses of action and make informed decisions based on desired outcomes. This is where DTs can play a crucial role when combining autonomous agent architectures with autonomous reconfiguration of PTs.

Deliberative agents utilise algorithms for planning and reasoning in order to create sequences of actions, enabling them to achieve their goals [33, 3, 36, 35]. Proper understanding of their own state and the state of their environment as well as the impact of their actions is required. This can be achieved by introducing domain knowledge through the DT paradigm.

There are various architectures for deliberative agents. One example is the Belief-Desire-Intention (BDI) model [9], which attributes three internal states to each agent: (i) beliefs, representing the knowledge about the environment the agent is embedded, (ii) desires, the goals and preferences each agent has and (iii) intentions, a subset of desires consisting of the immediate plans and sequence of actions to achieve the goals. The overall architecture is illustrated in Fig. 14.2, extended with a DT. Here the Belief Revision Function (BRF) updates the beliefs based on experiences gained

through the sensors. In this arrangement, the BRF can additionally interact with the digital copy of the PT, and therefore reduces the degree to which actions have to be executed on the environment. Beliefs feed into the function to generate options. The DT-enabled system replicates the BDI architecture within the DT in order to extend the functionality. Sensory input is not only given to the DT but specifically to the BRF. Similarly, the Actuators not only affect the real world, but also inform the DT. The assumption is, that the generated beliefs, desires and intentions are deterministic. If they are not, a synchronisation between beliefs, desires and intentions is required. The generated goals might or might not be achievable for the agent. Again, the function of generating options can interact with the DT to refine the options more accurately. The filter function ensures that only feasible desires become intentions. Reconfiguring the agents physical presence (i.e., moving a robot arm or traversing through the environment) is based on a local reasoning approach.

Another architecture is MAPE-K, introduced by Kephart and Chess [28]. At its core, it utilises an autonomic manager consisting of the Monitor, the Analyser, the Planner, the Executer and the Knowledge base, as shown in Fig. 14.3. The elements of the architecture are as follows:

- *The Monitor* collects data about the CPS's internal state and relevant external or environmental factors. Information is directly received through the sensors of the system as well as monitoring the CPS's performance and required resources.
- *The Analyser* processes the information from the Monitor. Utilising machine learning approaches allows the CPS to create meaningful insights such as patterns, trends, anomalies or critical events. Furthermore, it can be utilised to perform diagnostics or predictions.
- *The Planner* creates appropriate plans and strategies for the CPS in order to achieve predefined goals or optimisations. It prepares necessary reconfiguration actions in order to improve the performance of the CPS towards its goals. The planning process may involve decision-making algorithms, optimisation techniques or rule-based approaches to generate an executable plan.
- *The Executer* implements and carries out the reconfiguration of the CPS based on the previously generated plans. The Executer interacts with the CPS's actuators or effectors to make the required adjustments to the CPS's configuration or behaviour.
- Finally, *the Knowledge base* contains all information and knowledge of the CPS. This ranges from domain-specific knowledge, models, policies and rules to historical data and information created by the Monitor. The Knowledge base provides the information required by the various components within the MAPE-K architecture.

When introducing a DT into the MAPE-K loop, the autonomic manager needs to be expanded and the knowledge base generated within the loop needs to be tightly coupled with the DT model as illustrated in Fig. 14.3. Feng et al. [21] have detailed the inclusion of a DT in the MAPE-K loop. As with other architectures, if knowledge

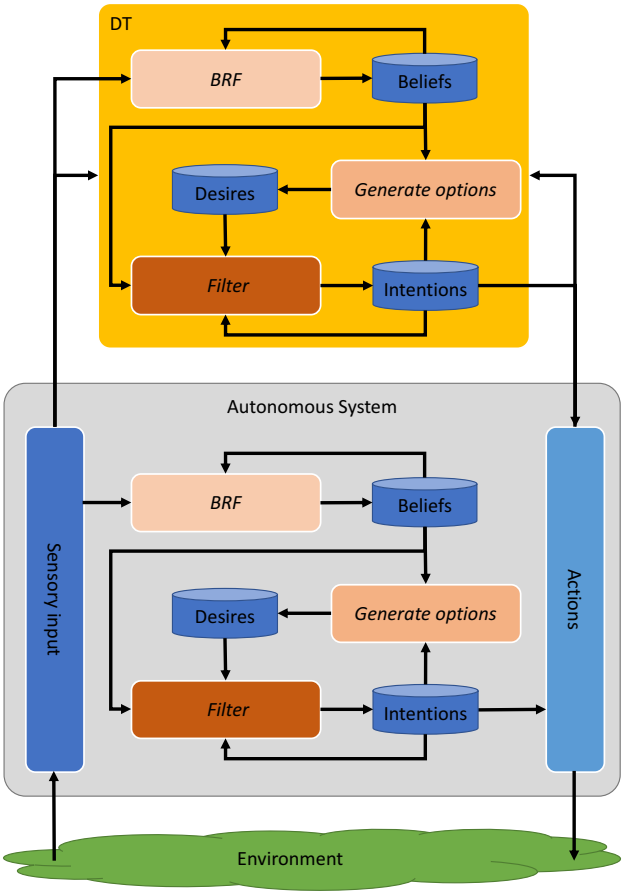


Fig. 14.2: Belief-Desire-Intention architecture for Autonomous Systems with an extension for DT following Bratman [9]. Beliefs represent knowledge about the environment shaped by BRF. Desires are goals of the CPS generated by the Generate options function and filtered by the Filter based on feasibility to create Intentions.

cannot be generated deterministically, a synchronisation between the PT and DT is required on this specific aspect.

We also consider the architecture for computational self-aware deliberative agents following [12, 20, 11, 29]. Computational self-awareness considers four fundamental levels of self-awareness and a meta-self-aware level:

- *Stimulus awareness* considers all stimuli and can differentiate between them. This is the most basic form of awareness.
- *Time awareness* allows to distinguish stimuli over time, enabling the CPS to consider patterns in the environment.

- *Interaction awareness* enables the CPS to identify different sources of stimuli and how other CPSs interact with it.
- *Goal awareness* is the last level, enabling the CPS to distinguish between its goals, allows it to adapt, select from among them and synthesise them.

In addition, *meta-self-awareness* gives the CPS an understanding of its awareness. Furthermore, the self-aware CPS architecture has a self-expression component, coordinating actions on the environment. As with all deliberative agents, models are at the core of the architecture, enabling the CPS to keep models about themselves and their environment. When incorporating a DT into the self-aware architecture, the self-aware mechanisms and algorithms are replicated in the DT. This means, similar to the MAPE-K loop, that the CPS needs to identify when the first-principles models need to be updated and calibrated with new information and learned models during runtime. With missing ground truth and in the absence of human operators, this can become very challenging even with simple CPSs [32]. Additionally, when goals are being updated within the DT-enabled PT, this needs to be reflected in the DT. In a similar way, self-expressive behaviour needs to be synchronised between the PT and the DT.

Introducing autonomous reconfiguration abilities into a DT-enabled system requires adequate models of the reconfigurable system itself and of the environment in which it operates, and the quality of the knowledge represented in those models needs to be known or maintained. As examples consider: (i) a change made by the

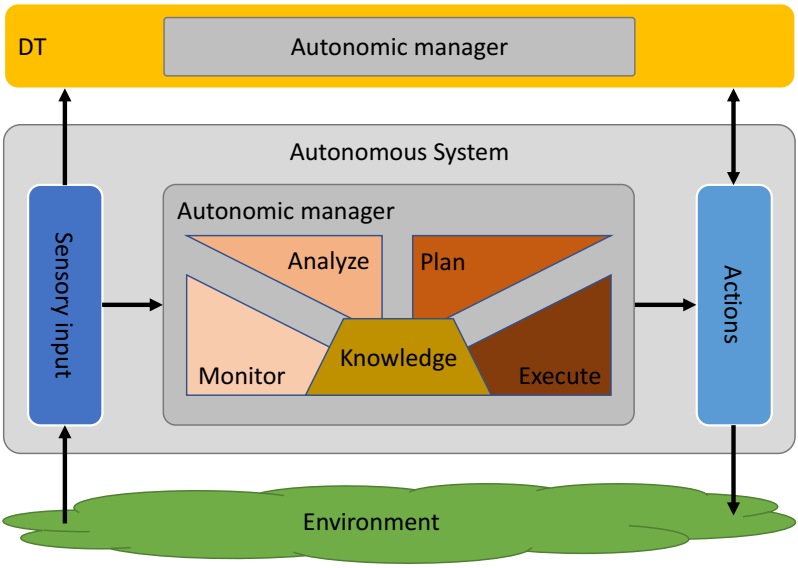


Fig. 14.3: MAPE-K architecture for Autonomous CPSs following [28] with a DT extension.

PT's actuators need to be reflected in the DT's models, (ii) DT-enabled system needs to take account of the bounds on the accuracy of data. However, through proper models of itself and the environment, so-called self-* properties can be realised.

With the proposed architectures shown in this chapter, DT-enabled systems have to ensure they can synchronise their information and state accordingly in order to profit from their DT. Additionally, explicit and secure communication channels are required. The issue of where architecturally a DT is deployed (fog, cloud, etc., as discussed in Chap. 11) clearly plays a central role in determining the quality of the system's autonomous actions.

14.3 Self-* properties

Self-* properties are core elements of autonomous CPSs and relevant for any autonomous reconfiguration. Different self-* properties do have dependencies on each

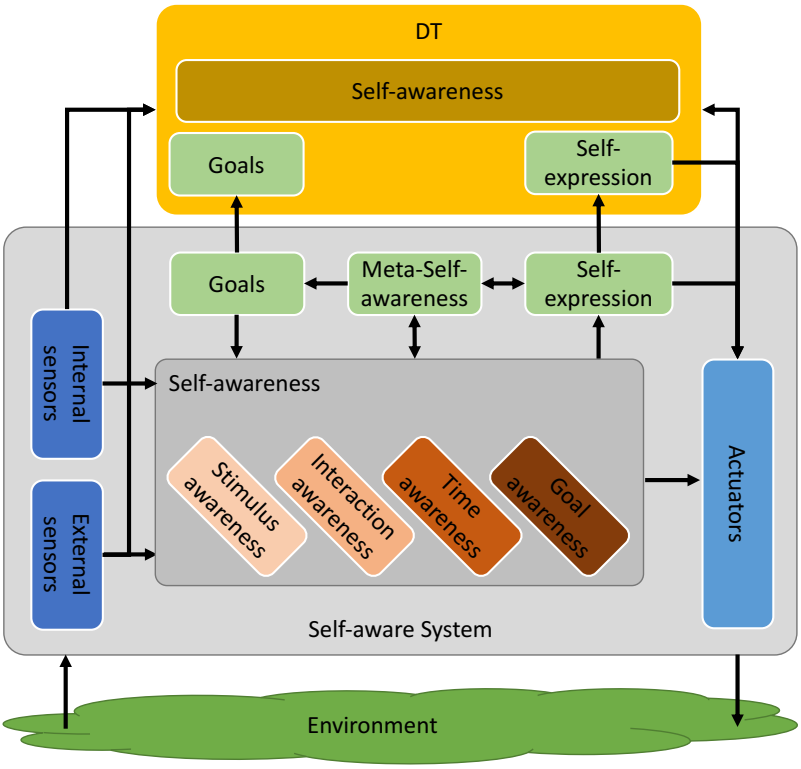


Fig. 14.4: Computational self-awareness architecture for Autonomous CPSs [11]

other in order to enable useful reconfiguration in autonomous CPSs. For example, self-adaptation requires a level of self-monitoring. In the following different self-* properties are discussed.

Self-monitoring: CPSs able to observe and assess their own behaviour, performance and state are considered self-monitoring. The process involves collecting, analysing and processing of data about the CPS's own components as well as interactions with the environment. Through insights into its own operation, identified patterns, anomalies or deviations from the expected behaviour, the CPS can detect issues or areas for improvement. This makes self-monitoring a crucial capability for self-maintenance, self-improvement and optimisation, self-healing, self-organisation with others and computational self-awareness. A Digital Shadow (DS) as discussed in Sect. 1.2.2 brings together data and models to deliver a monitoring capability. This can be used to increase the level of self-monitoring in the DT-enabled system, as it can inform external stakeholders through visualisation capabilities of the kind outlined in Chap. 8.

Self-adaptation: CPSs able to adapt and change their actions, behaviour, configurations or even physical structure in response to changing conditions in the environment or the requirements, are considered to have self-adaptive capabilities. Self-adaptation builds heavily on self-monitoring, i.e. observing and understanding ones own capabilities, performance and effects on the environment. In many cases, self-adaptive behaviour will synthesise new actions and behaviours and analyse its performance either in internal simulations or directly in the physical environment. Note that, while simulations are resource intensive, they avoid risks of potential damage to the CPS through harmful actions. Self-adaptation enables systems to dynamically change their configuration, allocate resources, adjust parameters or change their operational modes. By adapting their behaviour at runtime, CPSs can maintain optimal functioning, handle uncertainties and respond effectively to evolving circumstances. Self-adaptation is crucial for other self-* properties such as self-optimisation or self-maintenance, among others.
Given the enablers and services that we discussed in earlier chapters, we would argue that a DT that has some decision-making ability and an actuation link to the PT is able to increase the extent of self-adaptation at the level of the DT-enabled system. Such a decision-making capability might come from the implementation of either a reactive or deliberative approach in the DT, exploiting the data, models, enablers and services such as what-if analysis (see Sect. 10.2), that the DT offers.

Self-healing: This property enables autonomous CPSs to recover from faults, errors or failures without the need for external intervention such as a human operator [24]. Self-healing mechanisms vary in capability: from simply applying alternative actions, behaviours or modes of operations to more complex techniques aiming to identify root causes and respective recovery operations to ensure a continuous functional state. These, more complex, mechanisms can involve error detection algorithms, fault-tolerant design, redundancy or proactive measures to prevent failures [34]. This means that self-healing requires an approach for

fault diagnosis. Self-healing CPSs aim to minimise downtime, maintain continuous operation and recover from disruptions, ensuring the CPS's reliability and availability, contributing to the overall resilience of the CPS. DTs can support self-healing activities through monitoring (Chap. 9), and fault diagnosis (Sect. 10.3) as well as taking appropriate recovery actions.

Self-maintenance: CPSs able to ensure their availability, performance and reliability proactively are considered self-maintaining [2, 31]. To maintain the CPS's operational health, preventive measures and corrective actions are applied to avoid degradation of the CPS's performance. To achieve this, these CPSs need to continuously self-monitor their performance, resources and activities. Applying approaches of self-maintenance and proactively identifying and addressing potential problems reduces downtime, extends the CPS's lifetime and potentially optimises performance under changing conditions. Self-maintenance is, in contrast to self-healing, a pro-active approach that aims to avoid failures and reduce downtime. Through the DT paradigm, predictive maintenance can be applied, allowing the exploration of various adaptation and self-maintenance strategies. The link between self-maintenance and predictive maintenance, discussed in Sect. 10.4, is natural, as in both cases techniques and data are needed to make predictions on when things will go wrong in order to apply either preventive or corrective measures. Thus, predictive maintenance can be seen as part of self-maintenance for a given CPS.

Self-protection: As discussed in Chap. 13, CPSs operating in a shared environment are prone to threats and malicious activities, and DTs arguably further increase the attack surface. CPSs with an ability to safeguard themselves against potential vulnerabilities and threats are considered self-protecting [23]. For this, the CPS is required to have proactive measures and mechanisms implemented to detect, prevent and mitigate risks that may compromise its security, integrity or availability. This can include authentication, access control, encryption, intrusion detection and firewalls. DTs can be utilised to find best defensive measures against unauthorised access, data breaches or malicious attacks. This ensures the provision of confidentiality, integrity, as well as availability of resources and information.

Self-optimisation: While continuous operation is provided by self-healing, self-maintenance, self-protection and self-adaptation, the property of self-optimisation goes one step further and aims at improving performance, efficiency and effectiveness of the CPS. This is achieved by runtime exploration and exploitation of adaptive parameters in combination with continuous monitoring and analysis. In static or known environments, self-optimisation requires identification of the state and matching with optimal parameters. In highly dynamic environments, the CPS might also be required to predict the future states in order to perform adaptation in a timely manner. As discussed in Sect. 10.2, the DT can be used in both cases, (i) to explore optimal parameters for all given configurations of the CPS and the environment by performing simulations and (ii) by making predictive

simulations of the DT to identify potential future states and apply self-adaptive activities pro-actively. By autonomously adapting and fine-tuning its operation, a self-optimising CPS can continually enhance its performance and adapt to evolving requirements. Exploration of better parameters can be performed within a single, established DT. However, recently, Ejersbo et al. [14] have proposed an approach that allows CPSs to replace entire models and change their internal structure at runtime. This can be utilised for exploration and optimisation of behaviour in co-simulations.

Self-management: In contrast to self-adaptation, CPSs with self-management abilities are also able to regulate and control their own behaviour and resources [13]. Self-management is often used as an umbrella term as it covers self-configuration, self-optimisation, self-healing and self-protection capabilities. By taking responsibility for tasks that would traditionally require human intervention, self-management allows CPSs to operate efficiently, adapt to changing conditions and maintain desired performance levels. In contrast to simple self-adaptation, the CPS cannot only dynamically adjust the its own behaviour, but also regulate and control itself as a whole. Therefore, self-adaption is only a specific aspect of self-management, making adaptations of individual aspects of the CPSs part of the overall self-management process [25]. Specifically, self-management in CPSs can be concrete implementation of a DT when the management aspects are applied immediately.

Self-organisation: When multiple autonomous CPSs collaborate, structure, patterns and order may emerge from their concurrent operation in the absence of external or centralised controller. This means, the interaction of individual CPSs gives rise to collective behaviour and organisational structures. As a result, the system of CPSs appears to be controlled even though it is only governed by individual rules. These rules might even differ from system to system. In the same way that self-organisation is found in biological systems [27] and social networks [4], it has also been utilised to coordinate multiple autonomous agents towards a common goal [40]. Utilising a DT in this setting can enable individuals to improve their individual actions towards mutual or personal goals. Therefore the self-optimisation is not necessarily only for the personal gain but may also benefit the greater good of the social community of other systems. It is tempting to draw a connection between this possibility and the potential for self-organising fleets of DT-enabled systems that function effectively as systems-of-systems (see Sect. 11.11), although realising this remains a research goal.

Self-awareness: Computational self-awareness enables autonomous CPS to understand their own internal state, behaviour, capabilities and interactions. Self-monitoring plays a crucial role in such CPSs, allowing it to analyse and interpret data about itself, including information about its resources, performance and interactions with the environment [30]. Furthermore, it is argued that, through computational self-awareness, an autonomous CPS can reason about its own limitations, strengths or goals. This enables informed decision making about adaptation pro-

cesses [8]. Expanding the concept in more detail towards the environment of the CPS, concepts of social awareness [5, 7] and networked self-awareness [16, 17] enable the autonomous CPS to consider their interactions with other CPSs in more detail, allowing for better performance in their interaction. Simple forms of self-awareness can be achieved with DSs as discussed in Sect. 1.2.2. Full computational self-awareness, as discussed in [11], also allowing to apply actions based on self-aware knowledge, will require a DT as a basis of the CPS.

14.4 Goals

In order to implement any of the self-* properties and perform meaningful autonomous reconfiguration, a CPS either needs to have an explicit goal or an awareness and understanding of goals. In contrast to properties of a CPS, goals describe states and properties that are not currently reached. Goals, whether defined and described explicitly or not, indicate and guide actions in order to achieve them. These actions might affect internal properties as well as external aspects and environmental states, including other CPSs.

Having goals: Having a simple goal function enables the CPS to adjust, reconfigure, self-optimise its actions in order to achieve this goal more efficiently and effectively. Utilising a DT, enables the exploration of optimal adaptations for the physical counterpart.

Knowing goals: With additional knowledge, the autonomous agent cannot only *have* goals but also reason about them. This means the agent has additional information related to the goals such as requirements from the environment or resources. With this the agent can select the most appropriate goals during runtime in the event that prerequisites for goals change. For example, a resource, that heavily affects the efficiency of achieving a goal has been removed from the environment. With this an alternative goal yields higher expected performance utility and is therefore selected by the agent [15]. Utilising a DT enables us to explore these differences in performance utility in order to maximise the overall performance of the autonomous CPS.

Dividing goals: Reconfiguration of CPSs may not only entail changing parameters for actions or even replacing one action with another, but also includes adapting goals themselves while keeping the underlying goal function. For example, let us consider the construction of a complex object, requiring various parts that need to be prepared beforehand. In this context, if some of the tools are sporadically provided externally, the CPS would not need to tool them and in doing so would waste valuable resources. Utilising a DT would allow the exploration of potential sub-goals and split up the original goal into individual steps with its own goals. With *knowing these sub-goals* the CPS could choose which goals are most profitable for their performance.

New goals: The last step, when it comes to dealing with goals, enables a CPS not only to work with existing goals, but also generate new goals based on observations. This might initially seem counter-intuitive. However, consider a CPS that would be idle and not generate any revenue. To overcome this, the DT could be utilised to explore potential new revenue streams defined by new goals. More so, a CPS could identify tasks and underlying goals it could perform in order to support another CPS, leading to better performance of the other CPS and an overall better utilisation of resources across all CPSs. This kind of collaboration is discussed further below.

14.5 Collaboration between Systems

When CPSs operate in the environment, they may interact with other CPSs. Integrating these other CPSs with one's own actions can lead to emergent and unintended behaviour and results [37]. Introducing more reliability and control in such integration processes, leading to collaboration between individual CPSs, is needed and desired. Utilising DT-enabled systems provide additional benefits when enabling CPSs to collaborate reliably [18]. In previous work, several steps have been outlined to achieve this:

1. *Detection:* A CPS that has the potential to interact with others in a (potentially mutually) beneficial way, needs first to be able to detect the other systems whether they are highly controlled or relatively autonomous. Additionally, the environment as a whole should be considered, as it may be affecting the chosen actions as well as interfering with the reconfiguration towards specific goals [6].
2. *Refinement:* Knowing that there is a *something else*, is not enough. In addition, the system also needs to be able to understand what actions do affect it and how. An accurate representation of the causes might not be relevant. Consider a situation in which a resource required by an agent has been displaced. The fact that the resource is displaced might require a reconfiguration. However, the cause of the displacement may or may not be relevant, dependent on whether one wants to simply use the displaced resource or alter the displacement in itself.
3. *Utilisation:* Finally, systems have to utilise their potential collaboration with other systems in order to achieve their individual goals more efficiently. This might require individual reconfiguration to ensure the best integration of the actions of others within the set of actions required to reach the goal.
4. *Repetition:* For best results, steps 2 and 3 need to be repeated in regular intervals. This ensures that changes to the configuration in one system do not interfere with the actions of another system. In addition, parallel reconfiguration may lead to a perpetual effect that requires continuous reconfiguration in order to perform well. With this, systems may spend more time on reconfiguration than actually performing well towards their goal.

To achieve these steps, a DT-enabled architecture has been proposed for collaboration and integration of new systems [18]. Fig. 14.5 illustrates this architecture. At its core, there is the DT manager, aiming to generate, modify, handle, but also destroy DTs of other systems locally. These DTs can then be utilised with its own DT model in the *Planner* to achieve most useful actions. While it would be preferred if all DT models are complete and shared by all CPS willingly, in reality DT models will be limited by the requirements of the CPSs they reflect. If now another CPS wants to use the DT, the relevant model needs to be adapted and adjusted to the requirements of the new CPS. Additionally, since the DT model does not affect the actual CPS it represents anymore, it is downgraded to a DS.

Specifically, the following components are proposed:

Creator: By performing a DT creation, a CPS can generate a DT from another CPS. This can be achieved by either applying first-principles observations to the model or by exploring the available DT models from the database and utilising the Mapper to find the best match for the observations.

Mapper: By performing a DT mapping, a CPS can identify similarities, differences or overlaps between models and facilitate the exchange of information or knowledge. It enables the integration of heterogeneous models, the conversion of models from one representation to another and the synchronisation of data and semantics across diverse modelling languages or systems. DT mapping plays a crucial role in areas such as data integration, model-driven engineering, ontology alignment and semantic interoperability.

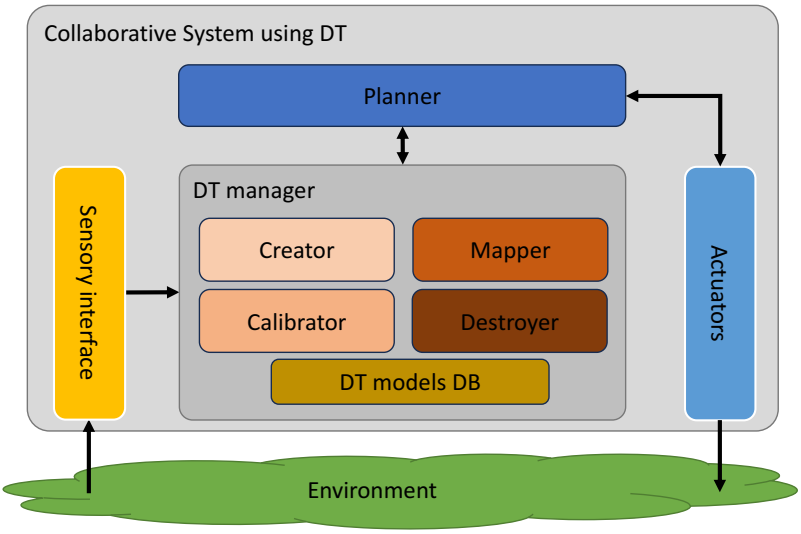


Fig. 14.5: Components required for a collaborative autonomous CPS using DT [18]

Calibrator: By performing a DT calibration, a CPS can modify the DT itself, making it more accurate to the actions and tasks performed. It allows the integration of new information into the DT at runtime while the actual CPS is being observed (see Sect. 6.2). A challenge arises as observation might not be accurate or obstructed and reliable calibration becomes difficult.

Destroyer: By performing a DT destruction, a CPS can free resources by not further utilising a DT of another CPS. In such a situation, the DT needs to be destroyed and removed from memory. Before doing so, the component needs to decide whether the underlying model should be stored in the DT models database for future reference. This should only be done if it is not a comparable DT model already. Again, the Mapper can be used to make such a decision.

14.6 Safety and uncertainty in reconfiguration

The interplay of DT-enabled systems and their changing environment lead to rapidly unfolding situations that may not have been foreseen when the individual DT-enabled system elements were designed and created. Nevertheless, CPSs operating along with other systems, including humans or other animals, have to ensure safety even in unexpected situations. Due to the emergent nature of the CPSs, not all safety measures, i.e., reconfigurations to ensure safety, can be defined at design-time, requiring autonomous approaches to establish them at run-time. This requires the CPS to identify similar situations and map those towards the required safety measures.

The monitoring of requirements itself is dependent on the architecture. In MAPE-K, this would be achieved in the Planner with information from the knowledge base while BDI agents would take those requirements into consideration during the filtering process. Almeaibed et al. [1] have discussed safety framework incorporating a DT explicitly for autonomous vehicular systems.

However, measurements of a CPS might be prone to uncertainty, either be due to insufficient knowledge and experience (epistemic uncertainty) or due to the inherent randomness of the environment itself (aleatoric uncertainty) [19]. While we often need to estimate the degree of uncertainty (i.e., uncertainty quantification), we are also interested in *uncertainty classification*, i.e., deciding whether an experienced randomness is epistemic or aleatoric. Approaches to identify and classify various uncertainties can apply what-if simulations as discussed in Sect. 10.2 utilising fault injection [22] and model-swapping [14] to explore the affects of various uncertainties.

14.7 Roadmap

While initial steps have been made towards meaningful reconfiguration utilising DTs, there are still several aspects that remain to be resolved in order to improve autonomous reconfiguration for CPSs. These aspects include, but are not limited to:

Mapping: At this point, mapping individual models against each other can only be done based on their natural language description. There are no dedicated techniques that allow us to compare the semantic levels of two arbitrary models. In order to achieve this, machines are required to build up meaning from the execution or observation of models directly. Even harder is the problem of generalisation from one model to another. A model that seems to be similar to another model by 95%, may still have a vastly different goal and purpose.

Uncertainty classification: Deciding whether an observed uncertainty is aleatoric or epistemic is inherently hard without a fixed point of reference. Only by having comparable and reliable information can estimations about the source of uncertainty be made.

Observation-based calibration: The task of generating and calibrating models based on observations faces a dilemma around deciding the nature of the uncertainty. For example, when a robotic arm is observed to be jittering, it is not possible to know if this jitter is due to some inaccuracies in the driving motor or if the robot is deliberately jittering due to some reasons only known to the robot itself. Even without the issue of external uncertainty classification, calibration of models based on observation requires complete information about the actions of the system and its effect on the environment. Overall, the external calibration represents a magnitude harder problem than direct calibration which has already been identified as an ongoing research direction in itself.

Micro/Macro models: Particularly when operating in a self-organised manner and in collaboration with numerous other systems, models of individual systems might result in a level of fidelity that is higher than is required for the goals of interaction with the collective. In such situations, it is required to abstract away information from individual models and generate collective models, representing the overall system-of-systems. Conversely, it is necessary to refine collective models down to individual ones when required. The former seems to be straight-forward, requiring the combination of relevant information from individuals within a single model. The latter might be impossible without explicit information exchange from the CPS represented by the model.

References

1. Almeaibed, S., Al-Rubaye, S., Tsourdos, A., Avdelidis, N.: Digital Twin Analysis to Promote Safety and Security in Autonomous Vehicles. IEEE Communications Standards Magazine,

5(1), 40–46 (2021). DOI: 10.1109/MCOMSTD.011.2100004
2. Asami, S., Talagala, N., Patterson, D.A.: Designing a self-maintaining storage system. In: Proceedings of the Symposium on Mass Storage Systems, pp. 222–233. IEEE (1999)
3. Badran, O., Mekhilef, S., Mokhlis, H., Dahalan, W.: Optimal reconfiguration of distribution system connected with distributed generations: A review of different methodologies. Renewable and Sustainable Energy Reviews **73**, 854–867 (2017)
4. Barabási, A.L.: Linked: The New Science of Networks. Perseus Pub. (2003)
5. Barnes, C.M., Ekárt, A., Lewis, P.R.: Social action in socially situated agents. In: Proceedings of the International Conference on Self-Adaptive and Self-Organizing Systems, pp. 97–106. IEEE (2019)
6. Barnes, C.M., Esterle, L., Brown, J.N.: "When you Believe in Things that you don't Understand": the Effect of Cross-Generational Habits on Self-Improving System Integration. In: 2019 IEEE 4th International Workshops on Foundations and Applications of Self* Systems (FAS* W), pp. 28–31. IEEE (2019)
7. Bellman, K., Botev, J., Hildmann, H., Lewis, P.R., Marsh, S., Pitt, J., Scholtes, I., Tomforde, S.: Socially-sensitive systems design: Exploring social potential. IEEE Technology and Society Magazine **36**(3), 72–80 (2017)
8. Bellman, K., Landauer, C., Dutt, N., Esterle, L., Herkersdorf, A., Jantsch, A., Taherinejad, N., Lewis, P.R., Platzner, M., Tammemäe, K.: Self-aware cyber-physical systems. ACM Transactions on Cyber-Physical Systems **4**(4), 1–26 (2020)
9. Bratman, M.: Intention, Plans, and Practical Reason. Cambridge, MA: Harvard University Press, Cambridge (1987)
10. Brooks, R.: A robust layered control system for a mobile robot. IEEE Journal on Robotics and Automation **2**(1), 14–23 (1986)
11. Chen, T., Faniyi, F., Bahsoon, R., Lewis, P.R., Yao, X., Minku, L.L., Esterle, L.: The handbook of engineering self-aware and self-expressive systems. arXiv preprint arXiv:1409.1793 (2014)
12. Diaconescu, A., Bellman, K.L., Esterle, L., Giese, H., Götz, S., Lewis, P., Zisman, A.: Architectures for collective self-aware computing systems. In: Self-Aware Computing Systems, pp. 191–235. Springer (2017)
13. Diao, Y., Hellerstein, J.L., Parekh, S., Griffith, R., Kaiser, G., Phung, D.: Self-managing systems: A control theory foundation. In: Proceedings of the International Conference and Workshops on the Engineering of Computer-Based Systems, pp. 441–448. IEEE (2005)
14. Ejersbo, H., Lausdahl, K., Frasheri, M., Esterle, L.: Dynamic Runtime Integration of New Models in Digital Twins. In: 2023 IEEE/ACM 18th Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), pp. 44–55 (2023). DOI: 10.1109/SEAMS59076.2023.00016
15. Esterle, L.: Goal-aware team affiliation in collectives of autonomous robots. Proceedings of the International Conference on Self-Adaptive and Self-Organizing Systems, pp. 90–99 (2018). DOI: 10.1109/SASO.2018.00020
16. Esterle, L., Brown, J.N.: Levels of networked self-awareness. Proceedings of the International Workshops on Foundations and Applications of Self* Systems (FAS*W), pp. 237–238 (2018). DOI: 10.1109/FAS-W.2018.00054
17. Esterle, L., Brown, J.N.A.: I Think Therefore You Are: Models for Interaction in Collectives of Self-Aware Cyber-Physical Systems **4**(4) (2020). DOI: 10.1145/3375403
18. Esterle, L., Gomes, C., Frasheri, M., Ejersbo, H., Tomforde, S., Larsen, P.G.: Digital twins for collaboration and self-integration. In: 2021 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C). IEEE (2021). DOI: 10.1109/ACSOS-C52956.2021.00040
19. Esterle, L., Porter, B., Woodcock, J.: Verification and uncertainties in self-integrating system. In: 2021 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C). IEEE (2021). DOI: 10.1109/ACSOS-C52956.2021.00050

20. Esterle, L., Rinner, B.: An Architecture for Self-Aware IoT Applications. In: Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing, pp. 6588–6592 (2018). DOI: 10.1109/ICASSP.2018.8462005
21. Feng, H., Gomes, C., Gil, S., Mikkelsen, P.H., Tola, D., Sandberg, M., Larsen, P.G.: Integration of the MAPE-K Loop in Digital Twins. IEEE (2022). DOI: doi:10.23919/ANNSIM55834.2022.9859489
22. Frasheri, M., Thule, C., Macedo, H.D., Lausdahl, K., Larsen, P.G., Esterle, L.: Fault injecting co-simulations for safety. In: The 5th International Conference on System Reliability and Safety ICSRS'21 (2021)
23. Frincke, D., Wespi, A., Zamboni, D.: From intrusion detection to self-protection. Computer Networks **51**(5), 1233–1238 (2007). DOI: j.comnet.2006.10.004
24. Ghosh, D., Sharman, R., Rao, H.R., Upadhyaya, S.: Self-healing systems—survey and synthesis. Decision support systems **42**(4), 2164–2185 (2007)
25. Hinchey, M.G., Sterritt, R.: Self-managing software. Computer **39**(2), 107–109 (2006)
26. Jennings, N.R., Sycara, K., Wooldridge, M.: A roadmap of agent research and development. Autonomous Agents and Multi-agent Systems **1**(1), 7–38 (1998)
27. Kauffman, S.: The Origins of Order: Self-organization and Selection in Evolution. The Origins of Order: Self-organization and Selection in Evolution. Oxford University Press (1993)
28. Kephart, J.O., Chess, D.M.: The vision of autonomic computing. Computer **36**(1), 41–50 (2003)
29. Kephart, J.O., Diaconescu, A., Giese, H., Robertsson, A., Abdelzaher, T., Lewis, P., Filieri, A., Esterle, L., Frey, S.: Self-adaptation in collective self-aware computing systems. In: Self-Aware Computing Systems, pp. 401–435. Springer (2017)
30. Kounev, S., Lewis, P., Bellman, K.L., Bencomo, N., Camara, J., Diaconescu, A., Esterle, L., Geihs, K., Giese, H., Götz, S., et al.: The notion of self-aware computing. In: Self-Aware Computing Systems, pp. 3–16. Springer (2017)
31. Mamei, M., Zambonelli, F.: Self-maintaining overlay data structures for autonomic distributed computing. In: Proceedings of the International Conference on Autonomic Computing, pp. 376–377. IEEE (2005)
32. Nelson, P.R., Bellman, K.L., Landauer, C.: Self-modeling - a practical example of why it's hard. In: 2022 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C), pp. 88–94 (2022). DOI: 10.1109/ACSOSC56246.2022.00039
33. Piciarelli, C., Esterle, L., Khan, A., Rinner, B., Foresti, G.L.: Dynamic reconfiguration in camera networks: A short survey. IEEE Transactions on Circuits and Systems for Video Technology **26**(5), 965–977 (2015)
34. Psaier, H., Dustdar, S.: A survey on self-healing systems: approaches and systems. Computing **91**, 43–73 (2011)
35. Thalamy, P., Piranda, B., Bourgeois, J.: A survey of autonomous self-reconfiguration methods for robot-based programmable matter. Robotics and Autonomous Systems **120**, 103242 (2019)
36. Tipaldi, M., Glielmo, L.: A survey on model-based mission planning and execution for autonomous spacecraft. IEEE Systems Journal **12**(4), 3893–3905 (2017)
37. Tola, D., Madsen, E., Gomes, C., Esterle, L., Schlette, C., Hansen, C., Larsen, P.G.: Towards easy robot system integration: Challenges and future directions. In: 2022 IEEE/SICE International Symposium on System Integration (SII), pp. 77–82 (2022). DOI: 10.1109/SII52469.2022.9708846
38. Watson, D.P., Scheidt, D.H.: Autonomous systems. Johns Hopkins APL technical digest **26**(4), 368–376 (2005)
39. Wooldridge, M.: Intelligent agents: The key concepts. In: ECCAI Advanced Course on Artificial Intelligence, pp. 3–43. Springer (2001)
40. Wooldridge, M.: An introduction to multiagent systems. John Wiley & Sons (2009)

Chapter 15
Future Directions and Challenges

Peter Gorm Larsen, John Fitzgerald, Cláudio Gomes, Jim Woodcock, Stylianos Basagiannis, Alessandro Ulisse, Lukas Esterle, Daniel Enrique Lucani Rötter, Simon Thrane Hansen and Bentley James Oakes

Abstract Although digital twins have been around for decades, there are many areas of research that need to be pursued in order to enable trustworthy predictions. The existing challenges and the future directions to take can be organised in many different ways. In this chapter, the foundational issues that we still think require clarification comes first. This is followed by research challenges in the different platforms needed to be able to produce digital twins in a cost-efficient manner. Afterwards the research changes for enabling autonomy of digital twins (uncertainty in many areas may hinder an accurate estimation of the state). Finally, research in composition of digital twins is considered and this really goes into system of systems considerations where an eco-system of stakeholders needs to agree about the means of composition.

15.1 Introduction

Although the concept of the Digital Twin (DT) is very well established, the rapid growth in interest in the technology is remarkable. Although many tools are emerging that enable the construction of DT-enabled systems, and the wave of new applications is almost overwhelming, we seem hardly to have scratched the surface of this technology. The potential of this combination of models, data, computation and communication is enormous and yet to be realised. In this final chapter, we identify directions for research and innovation towards the goal of a well-founded and established "discipline" of DT engineering that helps us to realise more of the benefits in the future and deliver DT-enabled systems on which people and businesses can rely.

There are many potential directions for research and innovation on DTs. It is striking that established organisations like the International Council on Systems Engineering (INCOSE) have visions for the future of digital transformations that include DTs [34]. In general, developing a DT research and innovation community

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

J. Fitzgerald et al. (eds.), *The Engineering of Digital Twins*,
https://doi.org/10.1007/978-3-031-66719-0_15

with a joint research agenda that cuts across application domains is perhaps needed to outcomes that are focussed on achieving such a vision. After the process of developing the content of this book, we propose four strands for future work:

Foundations: This includes the development of a basis for placing well-understood levels of reliance on the services offered by DTs. Key to this is mastering the uncertainties that arise from elements of DT-enabled systems and their environments, so that a DT's predictions are fit for purpose and can safely and securely be used to inform decisions that affects DTs' corresponding Physical Twins (PTs). We discuss directions for future work here in Sect. 15.2.

Platform: It can be costly and challenging to develop and maintain a DT, and so there is a line of valuable work to be done in making the engineering of DTs for applications both more affordable and sustainable. Goals here include the automation of DT operation through offering platforms and tools that have potentially wide user bases. We consider this further in Sect. 15.3.

Autonomy: Can DTs present a route towards dependable autonomy for Cyber-Physical Systems (CPSs)? As shown in Chap. 14, there are decisions to be made in either trusting DTs to influence the physical world directly or in augmenting existing autonomy. Research and innovation here also involves ways to provide a "learned cognitive capability" inside the DT by deductive and inductive means. We discuss this further in Sect. 15.4.

Composition: The work on a platform for DTs as a Service discussed in Chap. 11 leads towards the possibility of the composition of DTs and by extension the composition of their DT-enabled systems. Such compositions range from simple fleets of near-identical and non-interfering products to more heterogeneous Systems of Systems (SoSs) in which reliance comes to be placed on the qualities of the emergent collaboration. We consider this subject further in Sect. 15.5.

We provide a short overview of novel and perhaps controversial applications of DTs beyond the area of engineering CPSs in Sect. 15.6 and end the book with concluding remarks in Sect. 15.7.

15.2 Firm Foundations for Digital Twin Engineering

Although DTs arise in many existing fields (see Sect. 1.3.3), there is still a lack of formal foundations that justify placing appropriate reliance on the outcome of some DT services. Below, we argue for work on the overall limits of the predictability of DTs (Sect. 15.2.1). This is closely linked to the need to quantify and cope with uncertainties at different levels (Sect. 15.2.2), and to support verify properties of DTs (Sect. 15.2.3). As discussed in Chap. 13, security-related properties are a particular concern in the context of DTs, and so we discuss this further in Sect. 15.2.4, alongside

safety in Sect. 15.2.5. Finally, Sect. 15.2.6 considers our reliance on good calibration and the challenge of calibrating multiple parameters simultaneously.

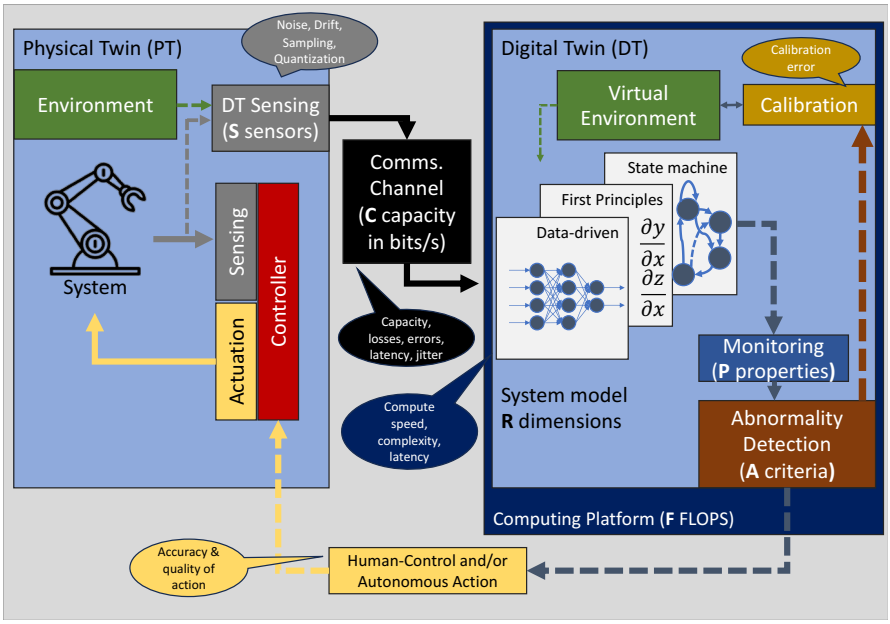


Fig. 15.1: Aspects affecting the prediction limitations from a DT.

15.2.1 Understanding the Limits of Predictions

To be able to rely on the predictions made by a DT, we must understand the limits on its accuracy. These limits include the high dimensionality and scale of the DT's different variables and stages of its operation, from data capture and communication to modelling and predictions. Preliminary work has proposed examining key qualities of models, namely relevance, verifiability, substitutability and faithfulness [51]. Earlier work in has focused on the model validity frame, that is, the set of inputs for which the models holds its relevance, verifiability and substitutability [18, 63]. The DT services that depend on models will naturally depend on, and in some sense propagate, these qualities, but it is currently unclear how they are affected by the algorithms employed in these services.

By understanding the limits on model-based analyses in the DT context, we can answer questions such as "What is the prediction accuracy of a DT with a model with complexity order **R** and a set of **P** properties testing **A** abnormality criteria

using information from **S** sensors sharing a communication channel of capacity **C** and a computational resource of **F** flops?". An overview of the connections between these different aspects is shown in Fig. 15.1. From left to right, we have the PT operating in its environment. The data from the sensors on the PT may have noise and suffer from other inaccuracies. That data needs to be communicated to the DT over a channel that can have limitations in its capacity, time delay and potentially lose information. When the data comes over to the DT on the right, it needs to be used both by the models of the PT and the monitors that check for abnormalities. Such models may need to be reduced to work within timing requirements and the capabilities of the underlying computing platform. In communicating back to the PT, the predictions' accuracy will be affected by the uncertainties of the different aspects here. The "formula" for this has not been scientifically established for DTs. Network calculus can be used for making upper- and lower-bound estimates to performance estimates [29].

Further questions arise from this:

- What are more appropriate models of computational resources?
- What is the prediction accuracy time horizon of a DT and how does it depend on the evolution of the PT?
- Can the over-approximations or over-conservative assumptions from network calculus be improved with control of the uncertainties?
- What are the bounds and feasible operating regions on prediction latency and accuracy?

Various effects characterise the limits of prediction accuracy. Naturally, when we wish to act on the predictions we need to communicate to the actuators, and here too there may be uncertainties in the form of inertia, accuracy and failure rates. If we think of this metric as an error in prediction, we could visualise it as a result of four sources: sensing, communication, modelling and monitoring.

15.2.2 Uncertainty: Quantification and Propagation

Extensive work has been done on traditional uncertainty configuration and propagation for CPS models. There is a need to continue the research in this area, but at the DT level, to provide valuable input for the challenges mentioned in Sect. 15.2.1. Different application domains have different ways to represent and propagate uncertainty. This includes reachability analysis [21], Monte-Carlo simulation [68], stochastic differential equations [49] and sensitivity analysis [14]). The novelty of this challenge is how to represent that uncertainty in the models used by the DT, how this propagates through the DT layers and whether that propagation affects the operation of those layers themselves. A DT must be able to cope with substantial changes to the environment of the PT [68]. DT-enabled systems must continue to

make decisions in the presence of uncertainty [16]. To do this, it must clearly distinguish between aleatoric and epistemic uncertainty as explained in Chap. 14. The propagation of the uncertainties of different aspects is essential to understanding how to determine the tolerances that can be allowed. The actions to be taken if such tolerances are exceeded will depend on the specific DT-enabled system in particular in relation to the criticality to the environment for the PT.

There is a need to work on the challenge of modelling and characterising the effects of sources of uncertainty that are not considered in the existing state of the art from a theoretical perspective, but which are known to be critical to the prediction accuracy and, thus, operation of DTs from a practical standpoint. For example, a physical model that incorporates friction effects usually considers it a known constant. This could be measured beforehand for controlled conditions with some reasonable measurement error. However, a DT aims to uncover the value of this constant from prior measurements (model calibration) and direct interactions with the environment. Thus, it will be able to adapt to changes in the environment not envisaged when the PT was originally engineered. This is related to the notion of "validity frames" indicating when the DT is able to provide acceptable predictions [2].

15.2.3 Towards Verified Digital Twins

Formal methods provide mathematical specifications, designs and realisations that can be analysed using simulation (see Sect. 5.6). Formal methods systematise and introduce rigour into all the phases of the development of software artefacts to be used in a DT context. The usual workflow is as follows:

- Write a specification of the PT from a system perspective.
- Write the desirable properties the PT should have and capture the assumptions that are necessary for its environment.
- Directly "test" the specification to check the properties. This testing may involve conventional validation techniques or more rigorous verification with model checkers and theorem provers.
- If this finds a problem, fix it without building the wrong PT.
- Once the system passes the test against the properties, start realising it while being confident that it will satisfy the needs.
- If necessary, the documentation for this process can be used for the certification of the PT.

If we deal with a DT-enabled system, we can go further. A DT enables the development of a self-adaptive CPS that reconfigures its behaviour in response to novel environments. These self-adaptations must be formally verified against the properties at runtime to avoid expensive re-certification of the reconfigured CPS [69]. A formal definition of DTs would enable the following:

- Develop a digital twinning research community that clarifies and agrees on a universally accepted concept.
- Continue to develop the CPS foundations that underpin DT engineering. Formalise these foundations in agreed tools, such as Isabelle [56] and Coq [8] on the discrete side and incorporate differential equations using tools such as KeYmaera X [58]. In the case of probabilistic models it makes sense to use model checkers such as Storm [17].
- Build user communities for application domains to increase industrial adoption and commercialisation.

The formalisation of DTs will enable use cases. For example, probabilistic DTs will be augmented with Markov and Bayesian models, particularly probabilistic risk models. They will be used for online risk assessment to support operational decisions.

15.2.4 Protection against Security and Privacy Attacks

We may need to add sensors and actuators to the PT so that the DT can perform appropriately. As we indicated in Chap. 13, malicious stakeholders can misuse these features [3, 65]. So, we must consider how to protect the DT-enabled system from attacks [40]. The objective is to increase security with the availability of the DT. How we do this will depend on the specific circumstances, but patterns of potential mitigation strategies need to be explored [39]. If we operate a Digital Twin as a Service (DTaaS) platform including many DTs, it will be essential to guarantee that each of their data spaces is separated and protected against intruders. As we discuss further in Sect. 15.5, organisations will be very eager to protect their Intellectual Property (IP).

Further research directions for secure DTs include the following:

- Developing a taxonomy of security properties for DTs across different application domains.
- Identifying DT vulnerabilities and how they can be exploited.
- Developing and specialising methods and tools for verifying security in DTs.
- Compiling a catalogue of DT attack models and mitigation strategies.
- Developing low-overhead security protocols for bidirectional communication between DTs and PTs.
- Agreeing on community-based, domain-specific case studies and benchmarks.

15.2.5 Synthesising Safe Scenarios

It can be advantageous to request the PT to perform specific actions that can be used to calibrate the models in a DT (e.g., determining the friction affecting a movement). However, before carrying out such experiments, it is necessary to ensure that the PT is in a state where it is possible (and not harmful) to carry out such an experiment. We imagine having a fundamental exploration layer inside the DT which can synthesise possible scenarios to be conducted to clarify something the DT thinks has changed. This is comparable to what is known as *design of experiments* for which a methodology was proposed by Fisher [30].

An exploration layer in the DT would require information about the PT and its interaction with the environment to create models. During the operation of the PT, it may have to devise physical experiments that the PT can conduct, the results of which will enable adjustments of the models (in particular of the environment). An example is an agricultural robot driving in a straight line. This movement does not give enough information about the mass distribution of the robot. So, the exploration layer needs to command the robot to experiment. It tells it to turn through 90° at a certain speed. It can then estimate the mass distribution from the experiment's results. Naturally, the exploration layer must know that this action is safe and will not interfere with the normal operation of the PT. There is extensive research in system identification [35] and anomaly detection [12] that tackles this problem. Scenarios can also provide predictive analytics for accidents by identifying poor or unsafe procedures in domain-specific workplaces. However, all of this research needs to be raised to the DT-enabled systems level and it needs to be possible to determine when it is safe to carry out such "experiments".

15.2.6 Mutual Calibration

Following a PT for extended periods will be necessary to track the PT and estimate the PT's state constantly. The uncertainties mentioned above mean that the perception of the PT's current state and the PT's real state will not always be fully aligned, and in addition, there may be a time offset inside the DT. The DT layers use multiple models for different aspects of the system. Many of these may overlap. For example, the PT model contains the plant updated by the PT tracking and state estimation layer [27]. When the plant model parameters are updated, the PT model must also receive those updates because other layers depend on it. The PT tracking and state estimation layer will use different state estimators (as in interacting multiple model filters [46]) that rely on overlapping models. Therefore, we need a way to synchronise them so they do not interfere.

When a DT detects some discrepancy, its root cause may either be some abnormality or the need to perform a (re-)calibration since uncertainties have accumulated

to the extent that deviation from prediction is now too great. If it is possible to carry out designs of experiments like those introduced in Sect. 15.2.5, this may be a fast way to get back in sync and reestablish the correct estimation of the current state of the PT. The fact that a DT contains multiple models complicates such calibrations further. In the long term, the vision is that it will be possible to shift autonomously between these models (see Sect. 1.4.2).

15.3 Digital Twin Platforms

Although DT frameworks and platforms already exist, there is scope for increasing automation in the engineering of DTs to reduce the development time and risk, as well as maintenance and operational costs. A visionary goal might be to reach the level of automation achieved for computer-based languages with scanners and parsers. Compiler construction has become one of the most mature examples of domain engineering. A similar standardisation of frameworks and tools may allow the DT community to move away from hand-built DTs and to focus efforts on more interesting avenues of research and development.

In this section, we first consider the potential for automating the production of DTs using frameworks and platforms (Sect. 15.3.1). Such improved automation entails using modelling languages that could be used to specify DTs and the services we require from them (Sect. 15.3.2). For these to be effective we need to have confidence that concepts are shared consistently, for which ontologies and knowledge graphs are critical in providing a common semantic backbone (Sect. 15.3.3).

As we implement and deploy more capable but computationally demanding DTs, there is a growing need to improve the ways in which work, especially simulation, should be distributed among the computing resources available (Sect. 15.3.4), and to ensure scalable yet accurate data linkage between DT and PT (Sect. 15.3.5).

Beyond automation of DT production and operation, we consider the wider engineering life cycle, and the need to keep an assured grasp on the large range of artefacts generated and maintained through a DT's existence. There is a need for research and innovation promoting dependable reuse and for methods that allow combined Development & Operations (DevOps) for DTs that are in service (Sect. 15.3.6), gaining the most value from the presence of multiple models in a DT (Sect. 15.3.7) and offering the right levels of assurance for DT applications (Sect. 15.3.8).

15.3.1 Automating Digital Twin Production

There are good arguments for bringing experience from Model-Driven Engineering (MDE) to bear on challenges in DT engineering. These include the need for management of heterogeneous models in DTs, synchronisation of the DT with the PT, and support for multi-stakeholder and multi-discipline engineering [9]. Any model-based framework to support DT engineering should be defined in terms of a basic architecture and set of language concepts and services, for instance, for integrating existing heterogeneous models that can be adapted and extended for specific DT developments. Among other things, the framework must enable the connection of the DT with system data on concrete platforms. The current frameworks (see Sect. 11.2) are mainly derived from corresponding Internet of Things (IoT) frameworks, and as a consequence the majority of the current automation is related to the communication of data from the PT to the DT. There is a need to at least introduce the same amount of automation for both simulations of models and checking inside monitors.

Such a framework requires identifying the information needed to deploy and configure key algorithms such as state estimation and monitoring automatically from a given set of models and metadata. Recent work suggests that the Functional Mockup Interface (FMI) – which provides an interface to interact with models regardless of the tool that built them – has potential here [26]. We need to evaluate the suitability of the FMI standard as the preferred way to interact with both the models and the monitors. Starting points could be implementing particle filter state estimators, which work with a wide range of continuous time and discrete event models and understanding their theoretical limitations [67].

15.3.2 Modelling Languages for Digital Twins

As outlined above, the MDE community has produced theories and techniques to combine and manage such heterogeneous languages but in the context of software systems. Moyne et al. identify DT requirements on DT frameworks in the medium and long term, including the capacity to use languages for DT definition, supporting reuse and portability, interoperability and interchangeability of DTs, and having the capacity to support validation and verification [47]. One of the languages that may be useful as a starting point for this strand of research is the Digital Twin Definition Language (DTDL)[1] targeted at describing Azure DTs. It is open-source, and may be a candidate for exploring where the limitations are for automation in DT construction [59]. DTDL is being extended with features such as DT configuration data. We imagine that these and other extensions to DT services can benefit the DT platforms using this.

[1] https://azure.github.io/opendigitaltwins-dtdl/

15.3.3 Incorporation of Ontologies and Knowledge Graphs

To engineer DTs effectively, we need techniques and tools to capture the *domain knowledge* required to reason correctly about and control these systems. One approach is to use *ontologies* to formalise these concepts and establish "a shared understanding of a domain" [62] in the form of a rich, semantic, and machine-readable expression of the domain concepts and relationships [33]. We foresee the use of ontologies in future DT engineering to provide the "semantic backbone" for aligning, integrating, and describing the components of the DT itself[2]. That is, ontologies can provide rich and traceable connections between the models, data, functions, and services of the DT, as well as the methodologies used to construct the DT itself. This backbone may aid integration of new information into DT platforms (Chap. 11), and can be used for (semi-)automatically producing high-quality experience reports [52].

Once ontologies have been constructed for a domain to provide the high-level concepts, the DT can also interact with a *Knowledge Graph* (KG) which aligns with these ontologies. The DT accesses these KGs as data repositories to provide *semantic* information, knowledge, and wisdom about the meaning of the data [53]. For example, the KG may contain temperature values which are not just *float* values, but can also be explicitly typed as *degree Celsius* values provided by a particular *sensor* component. The increased traceability enhances the DT's quality, as the KG "acquires and integrates information into an ontology and applies a reasoner to derive new knowledge" [22]. This reasoning can ensure the consistency and quality of the DT and its data [51], and can reduce the need to perform experimentation.

While the mathematical underpinnings of ontologies provide formal reasoning, they also bring practical challenges. These include scalability concerns, especially as KGs could contain millions of elements. Another factor is the expertise and tooling required to construct a domain ontology. The tool Protégé is widely used, even though it has been noted to "require significant experience in knowledge representation and semantics" [33]. This may change for future ontology construction, as the openCAESAR ontology-based modelling language framework [64] reduces the accidental complexity of ontological construction to aid the development process.

Another avenue for integrating domain knowledge is through the use of Large Language Models (LLMs). Once trained on a suitably-large corpus, LLMs could be used in the DT construction process by recommending DT components [70]. LLMs could also be employed as a natural language interface to the DT itself, where the user would provide questions as prompts. The LLM could then act as an agent to coordinate the KG search, the running of experiments, or executing simulations to obtain the answer. New research questions for LLMs come up, e.g., can we train LLMs to provide the level of robustness needed when composing unreliable or untrusted DT interfaces.

[2] Ontologies may work best in eco-systems in certain domains, such as space `https://mb4se.esa.int/`.

15.3.4 Distributed Simulation and Workload Distribution

Operating and executing DTs at high fidelity requires many resources and an appropriate workload distribution for simulating the DT models and monitors, for example in a High-Performance Computing (HPC) context. One possible research strategy is splitting the DT into components, distributing it to several devices and co-simulating the components. However, in such co-simulations it is also challenging to couple models with different levels of fidelity in a manner which is faithful in relation with the PT. Dynamic resource allocation will ensure optimal performance between edge, fog and cloud (see Sect. 11.8). Correct execution requires time-sensitive delivery of information and time-triggered networking and interaction. Improved workload distribution may require changing the fidelity of models (see Sect. 15.3.7). Relevant work includes serverless computing [55] and the cognitive compute continuum [28].

In particular, when the DT operator wants to carry out some of the more advanced DT services, such as what-if simulations (see Sect. 10.2), it will be necessary to use a distributed underlying hardware infrastructure. Research is needed to determine the optimal ways to use a distributed architecture based on the needs for the services requested from the DTs. For example, what-if analysis is only relevant if it can be conducted faster than real-time.

15.3.5 Bi-directional synchronisation with the actual system

The architecture of a DT and its PT requires automatic, bi-directional data linkage between the virtual and physical worlds [50]. Synchronisation is crucial for the DT to follow the changes in the PT's state. However, control, anomaly detection and behavioural optimisation require data to pass from the DT to the PT. The main challenge is in providing scalable solutions to support this bi-directional communication and synchronisation throughout the life cycle of the DT-enabled system. In particular, if errors of some kind start appearing at a PT, this will likely increase traffic to the DT, so here one can research alternative approaches for transferring/compressing the data. The possible time delay from PT data arriving at the DT can be so significant that it would only make sense to operate as a Digital Shadow as discussed in Sect. 1.2.2 and principles for ensuring this need to be researched.

In general, for configuring the data to be communicated and the capacity of the communication media used to a DT innovation in how to automate the set-up for potential compression/decompression of data if necessary (depending upon where the bottlenecks are present). It will also be interesting to research the extent to which it could be possible to change this dynamically and autonomously, since it is by no means obvious how this will best be accomplished.

15.3.6 Full Life Cycle Management

Establishing a DT-enabled system involves a multiplicity of artefacts, including DT assets and the associated documentation and assurance information. Innovation is necessary to ensure that all artefacts from the entire lifecycle are managed appropriately through version, and traceability control. The governance of models, data, and the relationships between them will be paramount. What is essential in this support is that whenever it makes sense, artefacts are reused in different phases (see Chap. 4).

The requirement to support collaborative development for DT-enabled systems applies throughout the life cycle. Thus, platforms must be able to support collaboration efficiently. When the DT-enabled system is deployed and operational, it is also possible to update services while the DT-enabled system is in use. In connection with DevOps it is essential that the PT that updates are pushed to is in a state where it is safe to provide the updates.

15.3.7 Leveraging Multiple Levels of Abstraction

A DT may use models that operate at different levels of abstraction, may have been intended for different purposes (see Sect. 1.4.2), and may offer different levels of computational performance. For example, a kinematic and a dynamic model of a mechanical system may provide similar predictions under some conditions and very different predictions under others. How can we take advantage of this wealth of models, tailoring them to the needs of each individual situation? To autonomously change between such alternative models, it would be necessary to capture the circumstances under which one wishes to swap between them. Ejersbo et al. [24] have proposed an approach allowing dynamic change of models at runtime. This includes replacing models as well as changing the structure of the entire DT (i.e., how individual models interact) while providing safety guarantees during the process.

Jones [37] poses the following research question: what level of fidelity is appropriate and realistic for a given use case in a DT setting? The answer must maximise benefit while minimising expense and the technical difficulty of implementation. The abstract level of fidelity raises some significant challenges. Can we twin before finalising the physical and virtual design? Can we create design prototype twins while the physical entity varies substantially in fidelity? Can we even twin a design concept or idea itself? The creation of abstract and early-stage DTs may have substantial benefits that are worth further exploration.

Models within the DT may operate at different levels of fidelity depending on the underlying requirements. Usually, the developer defines the degree of fidelity, fixing it for the model's lifetime. In this research area, we also want to investigate different fidelity levels to adjust models' fidelity at runtime. There will naturally

be a tradeoff between simulation speed and precision so that if the deadline for performing a simulation is tight, a model with less precision may be advantageous. In general, model swapping poses significant challenges due to the dynamics of physical systems, so it is important to be able to identify the situations in which it is possible to swap between models of different fidelity. This ability may prove valuable in workload distribution (Sect. 15.3.4).

15.3.8 Certification of Digital Twins

How much assurance can we offer for a DT-enabled system? Much of the value in a DT comes from enabling services that include monitoring, decision support and predicting maintenance needs. All these capabilities potentially increase dependability. Whether as an advisory system or a partially autonomous controller, a DT may need to be trusted. Validation, Verification and Accreditation (VV&A) processes and methodologies which focus on models and simulations are a starting point here [19]. However, these need to be implemented, extended and assessed at the DT level. Here the process to be followed for the modelling and simulation capability needs to be credible and that can be documented following a specific Software Process Improvement and Capability Determination (SPICE) [23].

Building on this, the methodology of validating and verifying the services inside a DT needs attention for cases where the DT-enabled system requires assurance of its dependability properties. This will likely also entail semantic traceability between different artefacts as discussed in Sect. 15.3 above [44]. Research in this area will necessarily involve certification authorities.

15.4 Increasing the Level of Autonomy for Digital Twins

Part of the vision shared by the participants in our opening round-table discussion (Chap. 2) was of DTs having a degree of autonomy inclosing the loop to the PT. Such autonomous real-time decision-making requires a high degree of trust in the predictions and actions taken in the DT-enabled system. This way of potentially reducing the human supervision of the PT performance also allows more rapid decision-making so long as this does not compromise safety. However, providing guarantees of safety of the DT-enabled system can be very challenging because of uncertainties at all levels, and ever-changing environmental conditions in some DT application domains. Even specifying trustworthiness is challenging [1].

We consider human involvement in the feedback loop to the PT (Sect. 15.4.1) and the integration of a cognitive layer into DT-enabled systems (Sect. 15.4.2). Then Sect. 15.4.3 describes research challenges related to closing the reality gap between

the current state in a PT and the perception within its DT, leading to a need to derive predictions even in the presence of inconsistencies. Sect. 15.4.4 considers research challenges for the perception of the state of the PT and its environment. Finally, Sect. 15.4.5 considers the possibility of adding robustness to the predictions performed inside the DT based on the uncertainty quantification bounds.

15.4.1 Reducing Human Supervision

As indicated in Sect. 2.5, it may make sense to initially have a human in the loop when a DT makes predictions. Research is needed into how one can gradually change the balance between the human and DT supervision when the level of trust in predictions is sufficiently high compared to the consequences of making wrong predictions.

A potential PT adaptation layer inside the DT can be responsible for reconfiguring the PT in response to novel environments while ensuring that the PT's performance is kept at the same level or improved. This layer, however, needs to set some requirements for it to operate without human supervision. We must verify new configurations against everything known about the PT and its environment [66, 69]. If these new configurations result from a design space exploration process (which is common), then one challenge is how to push the validation into the exploration process itself rather than just doing it at the end. Another challenge is representing and learning the safety properties we use to verify the new configuration. Signal temporal logic is an obvious candidate, but learning such properties is still in its infancy [36]). Furthermore, we need automation to create a runtime monitor to check whether the assumptions used to generate the new configuration remain valid as a PT operates [4]. If this is no longer true, we should roll back the system to a previously known safe configuration, as is commonly done in micro-service architectures [13]. Unfortunately it is often not feasible to roll back for many PTs, such the circumstances under which this is possible needs to be identified.

15.4.2 The Cognitive Digital Twin

A "Cognitive" DT has some autonomous capabilities, reasoning, learning and autonomously triggering reconfigurations as adaptations. Such a DT will typically apply deductive and inductive principles, the former based on formal rules, the latter on data science [71, 61]. To make such a vision a reality, a cognitive DT is defined to dynamically recognise complex and previously unpredicted behaviours with optimisation strategies. The expectation is that semantic technologies (such as the ontologies mentioned in Sect. 15.3.3), artificial intelligence, Industrial IoT and ubiquitous sensing technologies will make it possible to realise cognition capabil-

ities at a certain level. Research on perception and prediction is needed to make decisions autonomously and react to the PT adaptively if we have sufficient trust in the perception/predictions being correct.

15.4.3 Awareness of the Reality Gap

Understanding uncertainties and considering their quantification allows users to adjust DTs and PTs accordingly (see also Sect. 15.2.2). Consequently, there is a need to investigate the capability of the autonomous system to utilise this information and obtain an awareness of the "reality gap" between the world as the DT perceives it, and reality. With this awareness, the system can potentially respond to operational shortcomings and explore options to mitigate its effects. Kounev et al. [38] has provided foundational work and an initial architecture for computational self-awareness while Bellman et al. [7] investigated self-aware properties specifically for CPSs. More generally, lessons learned from the autonomy research community will be an excellent starting point (see Chap. 14). In general, the DT will need to operate in the presence of inconsistent data, managing these inconsistencies in its assessment of the current situation. There is as yet only limited experience in this form of "situational awareness" in the DT setting (see also Sect. 1.3.3).

15.4.4 Capture and Representation of Causal Relations

The primary responsibility of a potential perception layer (above the data layer) is to gather information about the different modes of operation of the PT and environment. There is thus a need for work on how to represent the different system operation modes, and what the conditions are for the system to transition between those modes. Hybrid Automata [31] and Hidden Markov Models [5] and the theoretical framework thereof, can be used as starting points.

To enable scalable reasoning at the higher layers, it is crucial that the different operating modes can be clustered into classes of equivalence. For example, we must evaluate a new system configuration in all possible operating modes learned from the historical data (Sect. 15.4.1). A research objective is to understand what modifications we need to carry out in the representations so that we can merge similar operating modes without losing the ability to verify the properties.

15.4.5 Increased Robustness

As discussed in Sect. 15.2.5, a goal for future work is to have uncertainty bounds available for outputs from models in the DT. Decisions based on those models can also produce uncertainty bounds. Consequently, PT reconfigurations will include a level of certainty that can help in autonomous decision-making. Robustness is also required for controlling how much resource is dedicated to solving a particular task. The attempt to find an optimal configuration will involve considering many potential states and parameter combinations. There is therefore a trade-off between the search for an optimal solution and the benefits that the optimal solution may be expected to deliver.

Kulkarni et al. [41] present an Enterprise Simulation Language to develop DTs to support the decision-making process in complex systems. Oquendo [54]. Thelen et al. [60] give a good overview over recent methods and approaches to deal with the quantification of uncertainty in both areas, aleatoric and epistemic uncertainty. The discussed methods can be categorised in *parameter calibration*, *bias correction*, and the *KOH framework*, constructing a Gaussian process regression model to simultaneously quantify several sources of uncertainty. While uncertainty is present in any CPS, uncertainty quantification enables systems to identify them and with appropriate mitigation strategies, the robustness of the system can be improved.

15.5 Supporting Composition of Digital Twins

Part of our vision of the future of DT engineering (outlined in Sect. 1.4.2), is the composition of DT-enabled systems that may have been produced by different manufacturers. This gives rise to several challenges, typically because the organisations involved may be to varying degrees both collaborators and competitors. Can we enable an ecosystem in which organisations are not so scared of sharing data that they forego the potential benefits of collaborating DTs?

Broadly speaking, we anticipate that the more high-fidelity the models used inside a DT-enabled system, the closer the model is to its owner organisation's core IP. Consequently, the need for infrastructure that enables the establishment of DTs across different organisations *while securing such IP* becomes paramount. Here the recent developments for the smart home ecosystem with what is called thread[3] and matter[4] is an interesting and promising development. This is a particularly interesting initiative since it is led by a number of the "tech giants", and so it may be able to move more rapidly than traditional standardisation processes.

[3] https://threadgroup.org/

[4] https://csa-iot.org/all-solutions/matter/

In Sect. 15.5.1 we focus on the need to enable standardised interfaces for DTs as step towards an ecosystem of DT-based systems. In Sect. 15.5.2 we discuss how collaboration between DTs can be established, and in Sect. 15.5.3 consider the need for transparency between organisations in an SoS setting.

15.5.1 Need for Standardised DT Interfaces

Complete automation of interoperability between DTs requires consensual standardisation of DT interfaces, but there is a lack of standards, frameworks and regulations for DTs. Practical standards often emerge from well-established communities of developers and users. Such communities are starting to form: for example, the Digital Twin Consortium[5] is a part of the Object Management Group (OMG). This needs to be continued with different stacks of technologies so that the competition between the companies can be more on the services inside their DT-enabled systems rather than depending on the specific technology stack and alliances. In this context it is worth mentioning the efforts conducted for Version 2 of the SysML standard[6] since this is a significant progress for Model-Based Systems Engineering (MBSE).

Other standardisation efforts are underway (see Sect. 1.3), but such initiatives can be relatively slow, requiring considerable alignment. of baseline concepts. It will take a long time for new research results to be incorporated into such standards and in particular, there is still less focus on the interoperability between DTs. Additional research is needed at a cognitive layer to determine how collaboration between DTs can be enabled there.

15.5.2 Collaborative Digital Twins

CPSs usually do not operate in isolation but are embedded in a dynamic environment and the context of other computational systems. Diverse DT-enabled systems must collaborate and interact, causing other behaviours to emerge. Studying these behaviours might not require high-fidelity models. We may be able to replicate them with simpler models representing the collective. The first step is to identify collective behaviour and automatically create DTs of these collectives. A second step will investigate approaches enabling the reverse operation, where we extract individuals from the collective. Finally, runtime analysis will permit a smooth transition from individual to collective systems. Casadei et al. [10] propose an initial version of DT for collectives.

[5] https://www.digitaltwinconsortium.org/

[6] https://github.com/Systems-Modeling/SysML-v2-Release

DTs can support predictive maintenance, runtime monitoring and what-if scenarios in the environment. We can use them to explore and define the activities of autonomous CPSs. However, at this point, DTs are rarely used to describe collaborative behaviour, particularly between DT-enabled systems provided by different organisations. We could extend to collectives if individual systems had a DT for each other. More research is needed into opportunities to exchange and use DTs of other systems collaboratively. This includes using these DTs to perform what-if scenarios, enabling autonomous systems to conduct collaborative research without being explicitly designed for them. Prior work outlining initial ideas will be continued and intensified in this research area [25].

We expect collaborative behaviour and interaction among CPSs to speed up research. But it will also make the underlying systems prone to exploitation by others (e.g., obtaining valuable IP through repeated simulation requests). Our collaborative behaviour system aims to avoid exploitation among the individual systems. A starting point for this activity is provided in the work by Bellman et al. [6] and our preliminary work in enabling distributed co-simulations [57].

15.5.3 Openness of DTs

DTs play a crucial role in sustainment operations, such as enabling the prediction of the remaining useful life of a system and the prediction of what-if scenarios to evaluate particular operational conditions and rare events. In this setting, typically, reusing design artefacts (e.g., models) to build DTs is in general not applicable as such artefacts are not intended to perform predictions. Moreover, exploiting purely data-driven approaches is typically limited by the data availability and quality (i.e., representativeness) as well, and using purely physics-based models is generally limited by a lack of knowledge regarding physics processes, the complexity of the dynamics of the system or the fact that when models can be defined, the result is too complex to scale up to industrial applications. An emerging modelling paradigm in the literature (see, e.g., [43]) is to combine data-driven approaches (e.g., ML-based) with physics-based approaches, typically resulting in surrogate models. This research direction promises to tackle the challenges mentioned above if different stakeholders are willing to share data. This would also enable a higher degree of data fusion, which will be necessary for composing DTs.

In general, there is a challenge in managing data ownership in an SoS context while still being able to share data with other organisations. Openness and sustainability are necessary to compose DT-enabled systems correctly across different organisations. Research is needed to construct user communities for different application domains where there is potential for mutual gain from collaboration through DTs. Consequently, we would expect an increase in the adoption and commercialisation of composable DT-enabled systems.

15.6 Novel Applications of Digital Twins

The concept of the DT is being taken up much more widely than in the highly engineered systems that form the subject of this book. We conclude by considering two of them: DTs for humans, and the metaverse.

The application of DTs of humans can potentially transform patient care through personalised data-driven medicine [15, 11, 32]. The DT of a human can be subjected to experimental treatments conducted virtually. Data gathered from a multiplicity of (accurate) human DTs brings the potential to help assess treatments over populations. Combined with DTs of delivery mechanisms such as hospitals, this may lead to improved resource allocation and management of treatments. It is interesting to think about mapping the concepts that we have discussed at the level of engineered DTs in this book (such as fidelity, accuracy, robustness, uncertainty) to the biological context.

The *metaverse* is perhaps a rather slippery concept. Although widely used, the term appears to have no settled definition, but, broadly speaking, has been used to refer to networks of virtual worlds offering online experiences that include a sense of immersion, real-time interactivity and user agency [20]. The idea of an *industrial metaverse* is perhaps more obviously and closely related to the types of system considered in this book. Seen as a continuation of the digitalisation trend, it is a convergence of technologies including DTs, virtual and augmented reality, IoT and AI. Such a metaverse supports interactions with industrial processes and systems including prototyping, design, simulation, monitoring and process optimisation. DTs are a pivotal technology in this [45]. Key factors in realising the technology are seen as interoperability, data and integration, automation, workforce skills, costs, regulation and security [48] reputation and identity, data security, currency and payments, law and jurisdiction, ownership and property, community and IT infrastructure. Challenges include reputation and identity, data security, currency and payments, law and jurisdiction, ownership and property, community and network and time and space [42].

15.7 Concluding Remarks

In concluding, it makes sense to reflect on the vision outlined at the outset of our book in Sect. 1.4.2 and the views of DT technologies from the roundtable in Chap. 2. The chapters following them demonstrated the breadth of engineering processes and techniques that come together to create an effective DT of a CPS. There are many trade-offs to consider, for example between models, data and communications mechanisms. Indeed, the costs associated with establishing, operating and maintaining a DT needs to be carefully balanced with the value it brings to its users.

Governance of data and models is essential to success. In some application domains it may never happen, whereas in others regulatory pressures may force the ecosystems to be established (e.g., in order to save energy). DT platforms enabling support for this are essential to make such a vision a reality. We believe that initially most DTs will be realised as DSs and add value by means of the digital services they supply, with trusted automation in reconfiguring the PT still some way off at scale.

In situations where the PT is a part of a larger ecosystem there are more challenges to be overcome as a result of the need to balance IP with the benefits of interaction. However, in situations where the supplier of a PT is able to obtain the necessary data from the PT itself (via sensors) we expect that we will see a significant increase in new DT-enabled systems.

DT engineering, as it emerges, will be a remarkable discipline in that it brings together a great diversity of technologies, competencies and challenges. At first sight this presents a complex, almost intimidating, picture, but our experience is that taking a disciplined systems-based approach may help to realise the vision of the dependable DT-enabled system. The development of DT engineering will be an exciting journey to follow!

References

1. Abeywickrama, D.B., Bennaceur, A., Chance, G., Demiris, Y., Kordoni, A., Levine, M., Moffat, L., Moreau, L., Mousavi, M.R., Nuseibeh, B., Ramamoorthy, S., Ringert, J.O., Wilson, J., Windsor, S., Eder, K.: On specifying for trustworthiness. Commun. ACM **67**(1), 98–109 (2023). DOI: 10.1145/3624699
2. Acker, B.V., Meulenaere, P.D., Vangheluwe, H., Denil, J.: Validity frame–enabled model-based engineering processes. Simulation (2023). DOI: 10.1177/00375497231205035
3. Alcaraz, C., Lopez, J.: Digital twin: A comprehensive survey of security threats. IEEE Communications Surveys & Tutorials **24**(3), 1475–1503 (2022). DOI: 10.1109/COMST.2022.3171465
4. Bartocci, E., Deshmukh, J., Donzé, A., Fainekos, G., Maler, O., Ničković, D., Sankaranarayanan, S.: Specification-Based Monitoring of Cyber-Physical Systems: A Survey on Theory, Tools and Applications. In: E. Bartocci, Y. Falcone (eds.) Lectures on Runtime Verification, *Lecture Notes in Computer Science*, vol. 10457, pp. 135–175. Springer International Publishing, Cham (2018). DOI: 10.1007/978-3-319-75632-5_5
5. Baum, L.E., Petrie, T.: Statistical inference for probabilistic functions of finite state Markov chains. The annals of mathematical statistics **37**(6), 1554–1563 (1966)
6. Bellman, K., Botev, J., Diaconescu, A., Esterle, L., Gruhl, C., Landauer, C., Lewis, P.R., Nelson, P.R., Pournaras, E., Stein, A., et al.: Self-improving system integration: Mastering continuous change. Future Generation Computer Systems **117**, 29–46 (2021)
7. Bellman, K., Landauer, C., Dutt, N., Esterle, L., Herkersdorf, A., Jantsch, A., Taherinejad, N., Lewis, P.R., Platzner, M., Tammemäe, K.: Self-aware cyber-physical systems. ACM Transactions on Cyber-Physical Systems **4**(4), 1–26 (2020)
8. Bertot, Y., Casteran, P.: Interactive Theorem Proving and Program Development. Springer Verlag (2004)
9. Bordeleau, F., Combemale, B., Eramo, R., van den Brand, M., Wimmer, M.: Towards model-driven digital twin engineering: Current opportunities and future challenges. In: Ö. Babur,

J. Denil, B. Vogel-Heuser (eds.) Systems Modelling and Management, pp. 43–54. Springer International Publishing, Cham (2020)
10. Casadei, R., Pianini, D., Viroli, M., Weyns, D.: Digital Twins, Virtual Devices, and Augmentations for Self-Organising Cyber-Physical Collectives. Applied Sciences **12**(1) (2022). DOI: 10.3390/app12010349
11. Cellina, M., Cè, M., Alì, M., Irmici, G., Ibba, S., Caloro, E., Fazzini, D., Oliva, G., Papa, S.: Digital twins: The new frontier for personalized medicine? Applied Sciences **13**(13) (2023). DOI: 10.3390/app13137940
12. Chandola, V., Banerjee, A., Kumar, V.: Anomaly detection: A survey. ACM Comput. Surv. **41**(3) (2009). DOI: 10.1145/1541880.1541882
13. Chen, L.: Microservices: Architecting for Continuous Delivery and DevOps. 2018 IEEE International Conference on Software Architecture (ICSA) pp. 39–397 (2018). URL https://api.semanticscholar.org/CorpusID:50768519
14. Chou, Y., Sankaranarayanan, S.: Bayesian parameter estimation for nonlinear dynamics using sensitivity analysis. In: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19, pp. 5708–5714. International Joint Conferences on Artificial Intelligence Organization (2019). DOI: 10.24963/ijcai.2019/791
15. Coveney, P., Highfield, R.: Virtual You: How Building Your Digital Twin Will Revolutionize Medicine and Change Your Life. Princeton University Press (2023)
16. Deantoni, J., Muñoz, P., Gomes, C., Verbrugge, C., Mittal, R., Heinrich, R., Bellis, S., Vallecillo, A.: Quantifying and combining uncertainty for improving the behavior of digital twin systems (2024)
17. Dehnert, C., Junges, S., Katoen, J.P., Volk, M.: A Storm is Coming: A Modern Probabilistic Model Checker. In: R. Majumdar, V. Kunčak (eds.) Computer Aided Verification, pp. 592–600. Springer International Publishing, Cham (2017)
18. Denil, J., Klikovits, S., Mosterman, P.J., Vallecillo, A., Vangheluwe, H.: The experiment model and validity frame in M&S. In: Proceedings of the Symposium on Theory of Modeling & Simulation, TMS/DEVS '17, pp. 1–12. Society for Computer Simulation International
19. Diem, J., Henninger, A., Youngblood, S.: VV&A in the M&S in the DoD: Moving from Art to Science. ITEA Journal (31), 324–327 (2010)
20. Digital, M.: Digital twins: The foundation of the enterprise metaverse. www.mckinsey.com/capabilities/mckinsey-digital/our-insights/digital-twins-the-foundation-of-the-enterprise-metaverse (2023). Accessed: 2023-11-12
21. Donzé, A.: Breach, a toolbox for verification and parameter synthesis of hybrid systems. In: T. Touili, B. Cook, P. Jackson (eds.) Computer Aided Verification, pp. 167–170. Springer Berlin Heidelberg, Berlin, Heidelberg (2010)
22. Ehrlinger, L., Wöß, W.: Towards a definition of knowledge graphs. SEMANTiCS (Posters, Demos, SuCCESS) **48**(1-4), 2 (2016)
23. Eichenseer, F., Heinkel, H.M., Benedikt, M., Ahmann, M., Holzner, M., Stadler, C.: Modeling & Simulation SPICE: Assessing the Capability of Credible Simulation Processes. INCOSE International Symposium **33**(1), 399–415 (2023). DOI: https://doi.org/10.1002/iis2.13029
24. Ejersbo, H., Lausdahl, K., Frasheri, M., Esterle, L.: Dynamic Runtime Integration of New Models in Digital Twins. In: 2023 IEEE/ACM 18th Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), pp. 44–55 (2023). DOI: 10.1109/SEAMS59076.2023.00016
25. Esterle, L., Gomes, C., Frasheri, M., Ejersbo, H., Tomforde, S., Larsen, P.G.: Digital twins for collaboration and self-integration. In: 2021 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C). IEEE (2021). DOI: 10.1109/ACSOS-C52956.2021.00040

26. Feng, H., Gomes, C., Sandberg, M., Macedo, H.D., Larsen, P.G.: Under what conditions does a digital shadow track a periodic linear physical system? In: A. Cerone, M. Autili, A. Bucaioni, C. Gomes, P. Graziani, M. Palmieri, M. Temperini, G. Venture (eds.) Software Engineering and Formal Methods. SEFM 2021 Collocated Workshops, pp. 143–155. Springer International Publishing, Cham (2022). DOI: https://doi.org/10.1007/978-3-031-12429-7_11
27. Feng, H., Gomes, C., Thule, C., Lausdahl, K., Iosifidis, A., Larsen, P.G.: Introduction to Digital Twin Engineering. In: Proceedings of the 2021 ANNUAL MODELING AND SIMULATION CONFERENCE. IEEE, Virtual Conference (2021). DOI: 10.23919/ANNSIM52504.2021.9552135
28. Ferrer, A.M.J., Becker, S., Schmidt, F., Thamsen, L., Kao, O.: Towards a cognitive compute continuum: An architecture for ad-hoc self-managed swarms. 2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid) pp. 634–641 (2021). URL https://api.semanticscholar.org/CorpusID:232170583
29. Fidler, M.: Survey of deterministic and stochastic service curve models in the network calculus. IEEE Communications Surveys & Tutorials **12**(1), 59–86 (2010). DOI: 10.1109/SURV.2010.020110.00019
30. Fisher, R.A.: The Arrangement of Field Experiments, pp. 82–91. Springer New York, New York, NY (1992). DOI: 10.1007/978-1-4612-4380-9_8
31. Frehse, G.: An Introduction to Hybrid Automata, Numerical Simulation and Reachability Analysis, pp. 50–81. Springer Fachmedien Wiesbaden, Wiesbaden (2015). DOI: 10.1007/978-3-658-09994-7_3
32. Geddes, L.: How digital twins may enable personalised health treatment. The Guardian URL www.theguardian.com/science/2023/nov/12/digital-twin-personalised-medical-treatment?CMP=Share_iOSApp_Other
33. Hildebrandt, C., Köcher, A., Küstner, C., López-Enríquez, C.M., Müller, A.W., Caesar, B., Gundlach, C.S., Fay, A.: Ontology building for cyber–physical systems: Application in the manufacturing domain. IEEE Transactions on Automation Science and Engineering **17**(3), 1266–1282 (2020). DOI: 10.1109/TASE.2020.2991777
34. INCOSE: Systems Engineering Vision 2035: Engineering Solutions for a better World. https://www.incose.org/2023_redesign/publications/se-vision-2035 (2021)
35. Isermann, R., Münchhof, M.: Identification of dynamic systems: An introduction with applications (2010). URL https://api.semanticscholar.org/CorpusID:45986314
36. Jha, S., Tiwari, A., Seshia, S.A., Sahai, T., Shankar, N.: Telex: learning signal temporal logic from positive examples using tightness metric. Formal Methods in System Design **54**, 364–387 (2019)
37. Jones, D., Snider, C., Nassehi, A., Yon, J., Hicks, B.: Characterising the digital twin: A systematic literature review. CIRP Journal of Manufacturing Science and Technology **29**, 36–52 (2020). DOI: 10.1016/j.cirpj.2020.02.002
38. Kounev, S., Lewis, P., Bellman, K.L., Bencomo, N., Camara, J., Diaconescu, A., Esterle, L., Geihs, K., Giese, H., Götz, S., et al.: The notion of self-aware computing. In: Self-Aware Computing Systems, pp. 3–16. Springer (2017)
39. Kulik, T., Dongol, B., Larsen, P.G., Macedo, H.D., Schneider, S., Tran-Jørgensen, P.W.V., Woodcock, J.: A Survey of Practical Formal Methods for Security. Formal Aspects of Computing (2022). DOI: 10.1145/3522582
40. Kulik, T., Gomes, C., Macedo, H.D., Hallerstede, S., Larsen, P.G.: Towards secure digital twins. In: T. Margaria, B. Steffen (eds.) ISoLA 2022, *Lecture Notes in Computer Science*, vol. 13704. Springer (2022). DOI: 10.1007/978-3-031-19762-8_11
41. Kulkarni, V., Barat, S., Clark, T., Barn, B.S.: Digital twin as an aid for decision-making in the face of uncertainty. In: 2022 Winter Simulation Conference (WSC), pp. 1371–1385 (2022). DOI: 10.1109/WSC57314.2022.10015528
42. Labs, L.R.: Top seven metaverse challenges, issues, and solutions. lucidrealitylabs.com/blog/7-challenges-of-the-metaverse#

1-reputation-amp-identity-metaverse-challenges (2023). Accessed: 2023-11-12
43. Legaard, C., Schranz, T., Schweiger, G., Drgoňa, J., Falay, B., Gomes, C., Iosifidis, A., Abkar, M., Larsen, P.: Constructing neural network based models for simulating dynamical systems. ACM Comput. Surv. **55**(11) (2023). DOI: 10.1145/3567591
44. Liu, J., Cao, X., Zhou, H., Li, L., Liu, X., Zhao, P., Dong, J.: A digital twin-driven approach towards traceability and dynamic control for processing quality. Advanced Engineering Informatics **50**, 101395 (2021). DOI: https://doi.org/10.1016/j.aei.2021.101395
45. Lyu, Z., Fridenfalk, M.: Digital twins for building industrial metaverse. Journal of Advanced Research (2023). DOI: https://doi.org/10.1016/j.jare.2023.11.019
46. Mazor, E., Averbuch, A., Bar-Shalom, Y., Dayan, J.: Interacting multiple model methods in target tracking: A survey **34**(1), 103–123. DOI: 10.1109/7.640267
47. Moyne, J., Qamsane, Y., Balta, E.C., Kovalenko, I., Faris, J., Barton, K., Tilbury, D.M.: A requirements driven digital twin framework: Specification and opportunities. IEEE Access **8**, 107781–107801 (2020). DOI: 10.1109/ACCESS.2020.3000437
48. Müller, M., Bohné, T., Jamboula, T.: Identifying key factors for the development of the industrial metaverse. In: 2023 IEEE Smart World Congress (SWC), pp. 1–6 (2023). DOI: 10.1109/SWC57546.2023.10448572
49. Mustafa Bayram, T.P., Buyukoz, G.O.: Numerical methods for simulation of stochastic differential equations. Advances in Difference Equations (17) (2018)
50. Nwogu, C., Lugaresi, G., Anagnostou, A., Matta, A., Taylor, S.J.: Towards a Requirement-driven Digital Twin Architecture. Procedia CIRP **107**, 758–763 (2022). DOI: https://doi.org/10.1016/j.procir.2022.05.058. Leading manufacturing systems transformation – Proceedings of the 55th CIRP Conference on Manufacturing Systems 2022
51. Oakes, B., Gomes, C., Larsen, P.G., Denil, J., DeAntoni, J., Cambeiro, J., Fitzgerald, J.: Examining Model Qualities and Their Impact on Digital Twins. In: Annual Modelling and Simulation Conference, pp. 220–232
52. Oakes, B.J., Parsai, A., Meyers, B., David, I., Van Mierlo, S., Demeyer, S., Denil, J., De Meulenaere, P., Vangheluwe, H.: A Digital Twin Description Framework and its Mapping to Asset Administration Shell. In: Model-Driven Engineering and Software Development, vol. 1708, pp. 1–24. Springer Nature Switzerland. DOI: 10.1007/978-3-031-38821-7_1
53. Oakes, B.J., et al.: Structuring and accessing knowledge for historical and streaming digital twins. In: 1st Workshop on Ontology-Driven Conceptual Modeling of Digital Twins, pp. 1–13
54. Oquendo, F.: Dealing with Uncertainty in Software Architecture on the Internet-of-Things with Digital Twins. In: 9th International Conference on Computational Science and Its Applications (ICCSA 2019), LNCS 11619, Springer, 9th International Conference on Computational Science and Its Applications (ICCSA 2019), pp. 770–786. Springer, Saint Petersburg, Russia (2019). DOI: 10.1007/978-3-030-24289-3_57
55. Patros, P., Spillner, J., Papadopoulos, A.V., Varghese, B., Rana, O., Dustdar, S.: Toward sustainable serverless computing. IEEE Internet Computing **25**(6), 42–50 (2021). DOI: 10.1109/MIC.2021.3093105
56. Paulson, L.C.: Isabelle: A Generic Theorem Prover. Springer-Verlag LNCS 828 (1994)
57. Pedersen, N., Lausdahl, K., Sanchez, E.V., Larsen, P.G., Madsen, J.: Distributed Co-Simulation of Embedded Control Software with Exhaust Gas Recirculation Water Handling System using INTO-CPS. In: Proceedings of the 7th International Conference on Simulation and Modeling Methodologies, Technologies and Applications (SIMULTECH 2017), pp. 73–82. Madrid, Spain (2017). DOI: https://doi.org/10.1145/3179993. ISBN: 978-989-758-265-3
58. Platzer, A.: Logical Analysis of Hybrid Systems - Proving Theorems for Complex Dynamics. Springer (2010)
59. Schmidt, C., Volz, F., Stojanovic, L., Sutschet, G.: Increasing Interoperability between Digital Twin Standards and Specifications: Transformation of DTDL to AAS. Sensors **23**(18) (2023). DOI: 10.3390/s23187742

60. Thelen, A., Zhang, X., Fink, O., Lu, Y., Ghosh, S., Youn, B.D., Todd, M.D., Mahadevan, S., Hu, C., Hu, Z.: A comprehensive review of digital twin – part 2: Roles of uncertainty quantification and optimization, a battery digital twin, and perspectives (2022)
61. Unal, P., Albayrak, Ö., Jomâa, M., Berre, A.J.: Data-Driven Artificial Intelligence and Predictive Analytics for the Maintenance of Industrial Machinery with Hybrid and Cognitive Digital Twins, pp. 299–319. Springer International Publishing, Cham (2022). DOI: 10.1007/978-3-030-78307-5_14
62. Uschold, M.: Knowledge level modelling: concepts and terminology. The knowledge engineering review **13**(1), 5–29 (1998)
63. Van Acker, B., Oakes, B.J., Moradi, M., Demeulenaere, P., Denil, J.: Validity Frame Concept as Effort-Cutting Technique within the Verification and Validation of Complex Cyber-Physical Systems. In: Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings, pp. 1–10. ACM. DOI: 10.1145/3417990.3419226
64. Wagner, D.A., Chodas, M., Elaasar, M., Jenkins, J.S., Rouquette, N.: Ontological Metamodeling and Analysis Using openCAESAR, pp. 925–954. Springer International Publishing, Cham (2023). DOI: 10.1007/978-3-030-93582-5_78
65. Wang, Y., Su, Z., Guo, S., Dai, M., Luan, T.H., Liu, Y.: A survey on digital twins: Architecture, enabling technologies, security and privacy, and future prospects. IEEE Internet of Things Journal pp. 1–1 (2023). DOI: 10.1109/JIOT.2023.3263909
66. Weyns, D., Iftikhar, M.U., de la Iglesia, D.G., Ahmad, T.: A Survey of Formal Methods in Self-Adaptive Systems. In: Proceedings of the Fifth International C* Conference on Computer Science and Software Engineering, C3S2E '12, p. 67–79. Association for Computing Machinery, New York, NY, USA (2012). DOI: 10.1145/2347583.2347592
67. Wikle, C.K., Berliner, L.M.: A Bayesian tutorial for data assimilation. Physica D: Nonlinear Phenomena **230**(1), 1–16 (2007). DOI: https://doi.org/10.1016/j.physd.2006.09.017. Data Assimilation
68. Woodcock, J., Gomes, C., Macedo, H.D., Larsen, P.G.: Uncertainty quantification and runtime monitoring using environment-aware digital twins. In: T. Margaria, B. Steffen (eds.) Leveraging Applications of Formal Methods, Verification and Validation: Tools and Trends, pp. 72–87. Springer International Publishing, Cham (2021)
69. Wright, T., Gomes, C., Woodcock, J.: Formally Verified Self-Adaptation Of an Incubator Digital Twin. In: Leveraging Applications of Formal Methods, Verification and Validation. Practice: 11th International Symposium, ISoLA 2022, Rhodes, Greece, October 22–30, 2022, Proceedings, Part IV, p. 89–109. Springer-Verlag, Berlin, Heidelberg (2022). DOI: 10.1007/978-3-031-19762-8_7
70. Wu, J., Gan, W., Chen, Z., Wan, S., Yu, P.S.: Multimodal large language models: A survey (2023)
71. Zheng, X., Lu, J., Kiritsis, D.: The emergence of cognitive digital twin: vision, challenges and opportunities. International Journal of Production Research **60**(24), 7610–7632 (2022). DOI: 10.1080/00207543.2021.2014591